



東華大學
DONGHUA UNIVERSITY

ICPC / CCPC
Standard Code Library

Gospel_rock



Coach

老黄畅谈

Contestants

ZnZrYb

xixii_

47523

Build Time: July 28, 2026 by ZnZrYb

Contents

1	梦的开始	6
1.1	ASCII 表	6
1.2	初始化模板	6
1.3	CMake 配置	7
1.3.1	Sanitizer 小技巧	7
1.3.2	Cmake Release 配置小技巧	7
1.4	本地手测时怎么发 EOF	7
1.5	CLion 常用功能与快捷键 (默认 keymap)	8
1.6	CLion 语言引擎: 赛场机上先切 Nova	8
1.7	Windows: 禁用 Shift 切换中英文输入	8
1.7.1	微软拼音 (系统自带, Win10 逐步截图)	8
1.7.2	搜狗输入法	10
2	基础数学	11
2.1	快速幂	11
2.2	矩阵快速幂	11
2.2.1	矩阵快速幂加速递推	13
2.3	自动取模类	14
2.4	异或线性基 (linear basis)	14
2.5	区间异或线性基 (prefix linear basis)	16
2.6	n 进制异或线性基	16
2.7	FFT 多项式乘法 (非递归)	18
2.7.1	高精度乘法 (基于 FFT)	18
2.7.2	FFT 的使用方法与常见应用	18
2.8	MEX (Minimum Excluded Value)	19
2.9	中位数 (Median)	19
2.9.1	中位数的双向判定不等式	19
2.9.2	± 1 编码 + 前缀和: $O(1)$ 区间中位数判定	19
2.9.3	多段奇长度划分公共中位数 = 全局中位数	19
2.9.4	L_1 最优化是中位数 (对比 L_2 平方和最优化是平均数)	20
2.9.5	流式 / 增量中位数: 对顶堆	20
2.9.6	区间第 k 大 (中位数是 $k = \lceil \text{len}/2 \rceil$ 的特例)	20
2.10	常见结论 (conclusion)	20
2.10.1	约数个数定理	20
2.10.2	Legendre 公式 ($n!$ 的质因子指数)	20
2.10.3	二次方程根与二阶线性递推	21
2.10.4	逆序对奇偶性与置换环	21
3	数论	22
3.1	线性筛	22
3.2	Miller-Rabin 与 Pollard's Rho	22
3.3	欧拉定理, 扩展欧拉定理, 欧拉函数的求解 (线性筛, 试除法)	23
3.3.1	欧拉定理与费马小定理的证明	23
3.4	gcd (迭代与递归)	24
3.5	exgcd 与逆元	24
3.6	在线快速逆元 ($O(1)$ 查询)	25
3.7	中国剩余定理 (CRT)	26
3.7.1	CRT 双射形式 (一维循环 $\leftrightarrow$ 二维网格)	26
3.7.2	扩展中国剩余定理 (exCRT)	26
3.7.3	CRT 的典型应用	26
3.8	裴蜀定理 (Bézout's identity)	27
3.8.1	裴蜀定理的典型应用	27
3.9	莫比乌斯反演 (Möbius inversion)	28
3.9.1	莫比乌斯函数 μ	28
3.9.2	反演公式	28
3.9.3	关键引理: $\sum_{d n} \mu(d) = [n=1]$	28
3.9.4	反演公式 (因数形式) 的证明	29

4	计数与组合	31
4.1	Catalan 数 (合法括号串数量)	31
4.2	求组合数 (递推法)	31
4.3	求组合数 (逆元)	31
4.4	插板法 (stars and bars) 求非负整数解的数量	32
4.5	Lucas 定理	32
4.6	组合相关公式以及技巧	32
4.6.1	离散变量概率变期望	32
4.6.2	朱世杰恒等式 (Hockey-stick identity)	33
4.7	Hook-length 公式 (树版本): 递增标号计数	33
4.8	Erdős-Szekeres 定理 (单调子序列)	33
5	博弈论	35
5.1	Nim 博弈	35
5.1.1	三角形博弈 (带约束的 Nim)	35
5.1.2	Moore's Nim-K (每回合可同时动 K 堆)	36
5.1.3	反 Nim (Anti-Nim / Misère Nim)	36
5.1.4	阶梯 Nim	36
5.1.5	Lasker's Nim (可拆分的 Multi-SG)	36
5.2	巴什博弈 (单堆有上限取)	36
5.3	SG 函数的暴力打表与 Anti / Every 变体	36
5.3.1	反 SG (Anti-SG)	37
5.3.2	Every-SG (每回合走"所有还能动的石子")	37
5.3.3	实战范例: CCPC 2025 网络赛 F「连线博弈」——XOR 哈希切子游戏	37
5.4	威佐夫博弈 (双堆联动)	38
5.5	斐波那契博弈 (前手两倍约束)	38
5.6	图上删边游戏	38
5.6.1	树上删边 (Colon Principle)	38
5.6.2	无向图删边 (Fusion Principle)	38
6	位运算相关恒等式	39
6.1	加法与位运算转换	39
6.2	或、与、异或的关系	39
6.3	减法与位运算	39
6.4	& 与 的分配律	39
6.5	lowbit 的推导	39
6.6	去除最低位的 1	39
6.7	连续异或前缀和	39
6.8	对相邻元素异或的操作转化为前缀异或交换	40
6.9	集合内两两最小异或和	40
7	基础算法	41
7.1	二维前缀和	41
7.2	二维差分	41
7.3	双指针模板	41
7.4	整数二分	42
7.4.1	partition_point 实现二分答案 (C++20)	42
7.5	三分 (实数)	42
7.6	三分整数	43
7.7	倍增法	43
7.8	LIS 最长上升子序列及其还原	43
7.8.1	Dilworth 定理 (最小链覆盖 / 偏序划分)	44
8	数据结构	45
8.1	单调栈	45
8.2	离散化	45
8.3	ST 表 (Sparse Table)	46
8.4	树状数组 (BIT / Fenwick Tree)	47
8.4.1	单点加, 区间求和	47

8.4.2	区间加, 单点查询	48
8.4.3	区间加, 区间求和	48
8.4.4	单点改, 区间最值查询 (BIT 实现 RMQ)	49
8.5	珂朵莉树 (ODT / Chtholly Tree)	50
8.6	懒标记线段树 (Tag + Info 写法)	51
8.7	带权可撤销并查集 (判二分图)	52
8.8	线段树分治	53
8.9	zkw 线段树	53
8.9.1	zkw 线段树区间加	53
8.9.2	线段树区间加乘 (递归 / zkw)	54
8.9.3	zkw 区间推平 $+-$, 区间和查询	55
8.9.4	zkwlen	56
8.9.5	线段树 RMQ (zkw / 递归式)	56
8.9.6	zkw 小白逛公园 (维护最大子段和)	57
8.9.7	zkw 线段树维护平方和	58
8.9.8	zkw 维护立方和, 四则运算, 区间推平	59
8.10	segSum (最普通的线段树) (返回第 k 个 1 的下标) (线段树二分)	60
8.11	segMxPosIntervalAdd 区间查询最大值位置以及最大值	62
8.12	segModify (扶苏的问题) (区间查询最大, 最小值, 区间赋值, 区间 $+-$)	64
8.13	segMul (区间 $+-$, 区间乘法)	66
8.14	segMxSum (小白逛公园) (最大子段和)	68
8.15	动态开点线段树 (区间加, 区间求和)	69
8.16	线段树上二分	70
8.16.1	前缀含负数时找区间第一个前缀和 $\geq x$ 的位置	71
8.17	segValueModAdd 求全局最小众数的权值线段树	71
8.18	线段树可视化	73
8.19	pb_ds 红黑树 (tree)	74
8.20	gp_hash_table (哈希表, 比 unordered_map 快)	75
8.21	rope ($O(1)$ 拷贝的可持久化序列)	76
8.22	FHQ-treap (无旋 Treap)	76
8.23	替罪羊树 (Scapegoat Tree)	77
8.24	主席树 (可持久化权值线段树)	79
8.25	分块 (单点修改)	80
8.26	回滚莫队	81
9	图论	82
9.1	倍增法求 LCA	82
9.2	虚树 (Virtual Tree)	82
9.3	并查集 (DSU) 与 DSU on next 技巧	84
9.3.1	带权并查集 (class 封装, 维护到根距离与连通块大小)	85
9.4	Borůvka 最小生成树	86
9.5	树上背包	86
9.6	Floyd 全源最短路	87
9.7	Dijkstra (堆优化)	87
9.8	二分图判定 (染色法)	88
9.9	匈牙利算法 (二分图最大匹配)	88
9.9.1	Hall 婚配定理 (完美匹配判定)	88
9.10	差分约束	88
9.11	Tarjan 缩点为边双连通分量 (EBCC)	89
9.12	Tarjan 求强连通分量 (SCC)	89
9.13	点分治	90
9.14	重链剖分 (HLD + 线段树)	91
9.15	欧拉回路与欧拉路径	93
9.16	Color Coding (随机染色 + 状压 DP)	93
9.16.1	转移骨架	94
9.16.2	剪枝技巧	94
9.16.3	复杂度	94
9.16.4	一句话总结	94

9.17	合法括号序列 $\leftrightarrow$ 有根有序树	94
9.18	网络流 (Dinic): 最大流 / 最小割容量及割边集合	95
9.19	最小费用最大流 (SPFA 势能 + Dijkstra 增广)	97
9.20	图论常见结论	98
9.20.1	树直径端点的“远点”性质	98
9.20.2	相邻点单源最短路差不超过 1	98
9.20.3	广义凯莱公式 (森林连通块版)	98
9.20.4	两条树上路径是否相交 (LCA 判定)	98
10	字符串算法	100
10.1	Manacher 算法	100
10.2	Z 函数 (扩展 KMP)	100
10.3	KMP (前缀函数 pi)	100
10.4	字符串哈希 (mod $2^{61} - 1$ 版本)	101
10.5	子序列自动机	102
10.6	Trie 树	102
10.7	01 Trie (XOR Trie)	103
10.8	字符串分词与输出技术 (大模拟读入)	104
11	动态规划 (dp)	105
11.1	二进制子集枚举技巧	105
11.1.1	枚举某个集合的所有子集	105
11.1.2	枚举固定大小的子集	105
11.2	SOS DP (Sum / Max over Subsets, 高维前缀和)	105
11.2.1	典型应用: 按位无交最大值	105
11.3	线段树维护 dp 求最大独立集	106
12	计算几何	108
12.1	点线基础模板	108
12.2	点在线上的投影点	111
12.3	对称点	111
12.4	点线位置判断 (CCW)	112
12.4.1	逆时针弧角 ccw_angle (标量 / 向量两版)	112
12.5	判断两直线是否垂直 / 平行	113
12.6	判断两线段是否相交	113
12.7	两条直线交点	113
12.8	两线段间最小距离	114
12.9	多边形面积	115
12.10	圆 Circle: 测度五件套	115
12.11	两圆位置关系 circle_relation	117
12.12	三角形内接圆 / 外接圆	117
12.13	直线与圆的关系 / 交点 circle_line_relation / getCrossPointsCL	118
12.14	两圆的关系 / 交点 circle_circle_relation / getCrossPointsCC	119
12.15	过一点圆的切点 getTangentPoints	119
12.16	两圆公切线 getCommonTangentPoints	120
12.17	两圆交集面积 two_circle_intersect_area	120
12.18	简单多边形与圆的交集面积 polygon_circle_intersect_area	121
12.19	多边形凸性检验	122
12.19.1	反例 1: 内角为反向凹角 (外角为锐角)	122
12.19.2	反例 2: 自交多边形 (蝴蝶结)	122
12.19.3	反例 3: 内角恰为 180°	123
12.20	点与多边形关系	124
12.21	极角排序	124
12.22	Minkowski 闵可夫斯基和	125
12.23	半平面交 (half_plane_cut)	126
12.24	凸包 (Andrew 算法)	127
12.24.1	对偶点技巧: 直线上半包络 $\leftrightarrow$ 对偶点上凸壳	128
12.25	凸多边形直径 (旋转卡壳)	129
12.26	最远点对 (旋转卡壳, 返回原始下标)	130

12.27最小矩形覆盖（旋转卡壳，4 指针）	131
12.28平面最近点对	132
12.29两凸多边形最近距离（旋转卡壳）	133
12.30常用公式	135
12.30.1 正多边形面积公式	135
12.30.2 三角恒等式	135
12.31附录：C++14 版本基础模板	136
13 STL 及其使用	139
13.1 string::find 的起始位置参数与 string_view	139
13.2 mt19937_64 套 splitmix64 抗 BM 攻击	139
13.3 unordered_map 自定义 hash & 结构体做 key	139
13.4 views::keys / views::values 遍历 map 的键 / 值 (C++20 ranges)	140
13.5 sort + unique 按 key 去重并保留指定值	140
13.6 lower_bound / upper_bound 自定义比较器参数顺序相反	140
13.7 __int128 读入输出（重载运算符版）	141
13.8 next_permutation 枚举全排列	141
14 常见问题与错误	143
14.1 -O2 下的 FMA 乘加融合踩崩浮点 ceil	143
14.2 __float128 使用要点	143
14.3 DP 不可达状态必须用 INF/-INF 显式标识	144
14.4 大网格 DFS 遇到障碍要跳过，不要继续标记递归	144
15 debug 技巧	145
15.1 除了打印之外，善用 assert	145
15.1.1 增量 debug	145
15.2 std::string 条件断点的陷阱	145
16 环境测试	146
16.1 试机赛与纪律	146
16.2 编译器版本与扩展头	146
16.3 __int128, __float128, long double 实测	146
16.4 评测机限制与编译选项	146
16.5 #pragma 开优化 / SIMD 实测	147
16.6 性能基准（map 跑分）	147
17 常用 Python 命令	148
17.1 启动交互解释器	148
17.2 当计算器：科学计数法速估	148
18 常用技巧（卡常，对拍，打表，调试等）	149
18.1 快读快写	149
18.1.1 读入，输出科学计数法小数	149
18.2 开大栈空间	150
18.3 C++ 中 struct 自定义比较器与 std::array 性能对比	150
18.4 计时	150
18.5 对拍	151
18.5.1 Special Judge（答案不唯一）题目怎么对拍	151
18.6 打表	152
18.6.1 打表中需要输出的东西（答案）	152
18.6.2 打表中需要输出的东西（前缀答案）	152
18.7 凸包数据生成	153
18.8 编译 shell	153

1.5 CLion 常用功能与快捷键（默认 keymap）

赛场 / 平时最常用的几个 CLion 操作与默认快捷键如下（赛场机一律 Windows / Linux，故只列这一套绑定）。**Run context configuration** 指「运行光标当前所在的那个目标」——多目标 CMake 工程里免去手动切下拉框，直接跑当前打开的这道题；**Reformat Code** 是一键格式化当前文件 / 选区；**Rename** 是语义级重命名，改变量 / 函数名时所有引用一起改，比全文替换安全（不会误伤字符串和同名的别的符号）。快捷键随 keymap 版本略有出入，忘了就用 **Find Action** (Ctrl+Shift+A) 搜功能名，弹出项右侧直接显示当前绑定。

功能	Windows / Linux
Run context configuration（跑光标所在目标）	Ctrl+Shift+F10
Run（当前选中的配置）	Shift+F10
Debug（当前选中的配置）	Shift+F9
Build Project（只编译不跑）	Ctrl+F9
Reformat Code（格式化当前文件 / 选区）	Ctrl+Alt+L
Rename（重命名符号 + 所有引用）	Shift+F6
注释 / 取消注释行	Ctrl+/
复制当前行 / 选区	Ctrl+D
删除整行	Ctrl+Y
上 / 下移当前行	Alt+Shift+Up/Down
Find Action（搜任意功能 / 快捷键）	Ctrl+Shift+A

1.6 CLion 语言引擎：赛场机上先切 Nova

CLion 有两套语言引擎：**Classic**（补全 / 高亮走 clangd）和 **Nova**（换成 ReSharper C++ 引擎，核心功能不走 clangd）。**赛场机开机先切 Nova**——最直接的好处是类模板补全带尖括号：敲 `vec` 回车，Nova 给 `vector<>` 且光标落在尖括号中间，Classic 只给 `vector`。竞赛里 `vector` / `map` 满屏都是，省下的按键很可观。

Classic 这边没有开关能打开：clangd 只对函数模板补尖括号，类模板一律不补（C++17 有了 CTAD，`vector v{1, 2}`；本身合法，判断不出要不要那对尖括号）。

切换：右侧工具栏齿轮 → **Switch to Nova Engine** → **Enable and Restart**；或 **Settings** → **Advanced Settings** → 勾 **Use the ReSharper C++ language engine (CLion Nova)**。要重启 + 重新解析项目，所以热身赛就切好；设置每台机独立，换机器要重切。CLion 2025.3 起 Nova 已是默认，新版本不用管这节。

1.7 Windows：禁用 Shift 切换中英文输入

Windows 赛场机上敲代码，误碰 Shift（打大写、打符号松手稍慢）就会被输入法切到中文模式，接着敲出一串全角字符——开赛前先把这个切换键关掉。

1.7.1 微软拼音（系统自带，Win10 逐步截图）

1. 开始菜单 → 设置：

2. 进入时间和语言：


```

        res.r = b.r;
        res.sum = a.sum+b.sum;
        res.sum%=mod;
        return res;
    }
    void apply(const Tag &t) {
        if (!t.need_apply) return;
        sum*=t.mul;
        sum%=mod;
        sum+=t.add*(r-l+1);
        sum%=mod;
    }
};

template<class I=Info,class T=Tag>
struct SEG {
    ll N;
    vector<I> infos;
    vector<T> tags;

    static bool isIntersect(ll l1,ll r1,ll
    ↪ l2,ll r2) {
        if (!(l1>r2 || r1<l2)) return true;
        return false;
    }

    SEG(ll N, const vector<I> &a) {
        infos.resize(4 * N + 5);
        tags.resize(4*N+5);
        this->N = N;
        build(a, 1, 1, N);
    }
    void push(ll u) {
        if (infos[u].l==infos[u].r) {
            return;
        }
        if (!tags[u].need_apply) {
            return;
        }
        infos[u<<1].apply(tags[u]);
        infos[u<<1|1].apply(tags[u]);
        tags[u<<1].apply(tags[u]);
        tags[u<<1|1].apply(tags[u]);
        tags[u]={};
    }
    void pull(ll u) {
        infos[u] = I::merge(infos[u << 1],
        ↪ infos[u << 1 | 1]);
    }

    void build(const vector<I> &a, ll u, ll l,
    ↪ ll r) {
        if (l == r) {
            infos[u] = a[l];
            infos[u].l = l;
            infos[u].r = r;
            return;
        }
        ll mid = l + r >> 1;
        build(a, u << 1, l, mid);
        build(a, u << 1 | 1, mid + 1, r);
        pull(u);
    }
};

```

```

I query(ll u, ll l, ll r) {
    if (infos[u].l >= l && infos[u].r <= r)
    ↪ {
        return infos[u];
    }
    push(u);
    I res{};
    // 为什么要设置 isget 这个 bool 呢?
    ↪ 其实是因为防止使用初值进行 merge
    // 我们并不保证使用初值进行 merge
    ↪ 一定是对的。
    bool isget=false;
    if (!(infos[u << 1].r < l || infos[u <<
    ↪ 1].l > r)) {
        res = query(u << 1, l, r);
        isget=true;
    }
    if (!(infos[u << 1 | 1].r < l ||
    ↪ infos[u << 1 | 1].l > r)) {
        if (isget) {
            res = I::merge(res, query(u <<
            ↪ 1 | 1, l, r));
        }else {
            res=query(u << 1 | 1, l, r);
        }
    }
    return res;
}

void assign(ll u, ll pos, const I &val) {
    if (infos[u].l == infos[u].r &&
    ↪ infos[u].l == pos) {
        infos[u] = val;
        infos[u].l = pos;
        infos[u].r = pos;
        return;
    }
    push(u);
    if (pos >= infos[u << 1].l && pos <=
    ↪ infos[u << 1].r) {
        assign(u << 1, pos, val);
    }
    if (pos >= infos[u << 1 | 1].l && pos
    ↪ <= infos[u << 1 | 1].r) {
        assign(u << 1 | 1, pos, val);
    }
    pull(u);
}

void modify(ll u,ll l,ll r,const T&t) {
    if (infos[u].l >= l && infos[u].r <= r)
    ↪ {
        infos[u].apply(t);
        tags[u].apply(t);
        return;
    }
    // 在判断完全覆盖之后再push, 不影响结果,
    ↪ 会省一点操作
    push(u);
    if (isIntersect(infos[u<<1].l,infos[u<<
    ↪ 1].r,l,r)) {
        modify(u<<1,l,r,t);
    }
}

```

```

        if (isIntersect(infos[u<<1|1].l,infos[u]
        ↪ <<1|1].r,l,r)) {
            modify(u<<1|1,l,r,t);
        }
        pull(u);
    }
    // 返回第k个有值的下标, 当然,
    ↪ 要求线段树叶节点值是0或1
    // ll findkth(ll u,ll cur_k) {
    //     push(u);
    //     if (infos[u].l==infos[u].r) {
    //         return infos[u].l;
    //     }
    //     if (cur_k<=infos[u<<1].sum) {
    //         return findkth(u<<1,cur_k);
    //     }
    //     return
    ↪ findkth(u<<1|1,cur_k-infos[u<<1].sum);
    // }

    //
    // void opposite(ll u, ll pos) {
    //     if (infos[u].l == infos[u].r &&
    ↪ infos[u].l == pos) {
    //         infos[u].sum ^= 1;
    //         return;
    //     }
    //     if (pos >= infos[u << 1].l && pos <=
    ↪ infos[u << 1].r) {
    //         opposite(u << 1, pos);
    //     }
    //     if (pos >= infos[u << 1 | 1].l && pos
    ↪ <= infos[u << 1 | 1].r) {
    //         opposite(u << 1 | 1, pos);
    //     }
    //     pull(u);
    // }

    template<class F>
    ll findFirst(ll u,ll l,ll r,F && fun) {
        push(u);
        if (infos[u].l==infos[u].r) {
            if (fun(infos[u])) return
            ↪ infos[u].l;
            return -1;
        }
        ll res=-1;
        if (isIntersect(infos[u<<1].l,
        ↪ infos[u<<1].r, l, r) &&
        ↪ fun(infos[u<<1])) {
            res=findFirst(u<<1,l,r,fun);
        }
        if (res!=-1) return res;
        if (isIntersect(infos[u<<1|1].l,
        ↪ infos[u<<1|1].r, l, r) &&
        ↪ fun(infos[u<<1|1])) {
            res=findFirst(u<<1|1,l,r,fun);
        }
        return res;
    }
};

```

8.14 segMxSum (小白逛公园) (最大子段和)

```

struct Info {
    // 注意啊, 这里除了 L 和 R
    ↪ 以外的所有你所添加的东西,
    ↪ 你都要好好想一想其初值是什么。
    // 不过我们其实改进了这个 query 函数,
    ↪ 所以说如果说你给的这个数组当中所有的值都
    ↪ 是所有的值都是有的话, 其实不用考虑初值。
    // 如果一定要赋初值的话, 注意就是 sum
    ↪ 类的都不要赋 infinite 然后只有 max 和 min
    ↪ 的你要好好想想。
    ll l = 0, r = 0, pre_sum = 0, suf_sum = 0,
    ↪ mx_sum = 0, sum = 0;

    friend ostream &operator <<(ostream &os,
    ↪ const Info &a) {
        os << "[" << a.l << "; " << a.r << "]: "
        ↪ << "mx_sum:" << a.mx_sum;
        return os;
    }

    string to_string() const {
        stringstream ss;
        ss << *this;
        return ss.str();
    }

    static Info merge(const Info &a, const Info
    ↪ &b) {
        Info res;
        res.l = a.l;
        res.r = b.r;
        res.sum = a.sum + b.sum;
        res.pre_sum = max(a.pre_sum, a.sum +
        ↪ b.pre_sum);
        res.suf_sum = max(b.suf_sum, a.suf_sum
        ↪ + b.sum);
        res.mx_sum = max({a.mx_sum, b.mx_sum,
        ↪ a.suf_sum + b.pre_sum});
        return res;
    }
};

template<class T>
struct SEG {
    ll N;
    vector<T> infos;

    SEG(ll N, const vector<T> &a) {
        infos.resize(4 * N + 5);
        this->N = N;
        build(a, 1, 1, N);
    }

    void pull(ll u) {
        infos[u] = T::merge(infos[u << 1],
        ↪ infos[u << 1 | 1]);
    }

    void build(const vector<T> &a, ll u, ll l,
    ↪ ll r) {

```

```

    if (l == r) {
        infos[u] = a[l];
        infos[u].l = l;
        infos[u].r = r;
        return;
    }
    ll mid = l + r >> 1;
    build(a, u << 1, l, mid);
    build(a, u << 1 | 1, mid + 1, r);
    pull(u);
}

T query(ll u, ll l, ll r) {
    if (infos[u].l >= l && infos[u].r <= r)
        ↪ {
            return infos[u];
        }
    T res{};
    bool isget=false;
    if (!(infos[u << 1].r < l || infos[u << 1 | 1].l > r)) {
        ↪ res = query(u << 1, l, r);
        isget=true;
    }
    if (!(infos[u << 1 | 1].r < l || infos[u << 1 | 1 | 1].l > r)) {
        ↪ if (isget) {
            res = T::merge(res, query(u << 1 | 1, l, r));
        }
        ↪ else {
            res=query(u << 1 | 1, l, r);
        }
    }
    return res;
}

void assign(ll u, ll pos, const T &val) {
    if (infos[u].l == infos[u].r &&
        ↪ infos[u].l == pos) {
        infos[u] = val;
        infos[u].l = pos;
        infos[u].r = pos;
        return;
    }
    if (pos >= infos[u << 1].l && pos <=
        ↪ infos[u << 1].r) {
        assign(u << 1, pos, val);
    }
    if (pos >= infos[u << 1 | 1].l && pos
        ↪ <= infos[u << 1 | 1].r) {
        assign(u << 1 | 1, pos, val);
    }
    pull(u);
}
};

```

8.15 动态开点线段树（区间加，区间求和）

AC <https://www.luogu.com.cn/record/266006147>

```
/*
```

```

* 2026-03-08 P13825 【模板】线段树 1.5
↪ 动态开点线段树
* AC https://www.luogu.com.cn/record/266006147
*
*/
struct Tag {
    // 在母串中的这个位置
    ll add=0;
    bool need_apply=false;
    void apply(const Tag &t) {
        add+=t.add;
        need_apply=true;
    }
};

struct Info {
    ll l=0,r=0;
    i128 sum=0;
    static Info merge(const Info &a,const Info
        ↪ &b) {
        Info res=a;
        res.r=b.r;
        res.sum+=b.sum;
        return res;
    }
    void apply(const Tag&t) {
        if (!t.need_apply) {
            return;
        }
        ll len=r-l+1;
        sum+=len*t.add;
    }
};

// 我认为还是用 C++14 的写法,
↪ 把你要写的东西传进去比较好, 不容易写错
template<class I=Info,class T=Tag>
struct SEG {
    struct Node {
        I info;
        T tag;
        Node* rs=nullptr;
        Node* ls=nullptr;
    };

    void init_node(Node *u,ll l,ll r) {
        u->info.l=l;
        u->info.r=r;
        u->info.sum=i128(u->info.l+u->info.r)*(
            ↪ u->info.r-u->info.l+1)/2;
    }

    Node* root=nullptr;
    ll N;

    static bool isIntersect(ll l1, ll r1, ll
        ↪ l2, ll r2) {
        if (!(l1 > r2 || r1 < l2)) return true;
        return false;
    }

    SEG(ll N):N(N) {
        root=new Node();
    }

```

```

    root->info.l=1;
    root->info.r=N;
    init_node(root,1,N);
}

void extend(Node *u) {
    if (u->ls==nullptr) {
        u->ls=new Node();
        init_node(u->ls,u->info.l,(u->info.l
        ↪ l+u->info.r)>>1);
    }
    if (u->rs==nullptr) {
        u->rs=new Node();
        init_node(u->rs,((u->info.l+u->info.l
        ↪ .r)>>1)+1,u->info.r);
    }
}

void push(Node *u) {
    if (u->info.l==u->info.r) {
        return;
    }
    extend(u);
    if (u->tag.need_apply) {
        u->ls->info.apply(u->tag);
        u->rs->info.apply(u->tag);
        u->ls->tag.apply(u->tag);
        u->rs->tag.apply(u->tag);
        u->tag={};
    }
}

void pull(Node *u) {
    if (u->info.l==u->info.r) {
        return;
    }
    u->info=I::merge(u->ls->info,u->rs->info
    ↪ o);
}

void modify(Node*u,ll l,ll r,const T& t) {
    if (u->info.l>=l && u->info.r<=r) {
        u->info.apply(t);
        u->tag.apply(t);
        return;
    }
    push(u);
    if (isIntersect(u->ls->info.l,u->ls->info.r
    ↪ fo.r,l,r)) {
        modify(u->ls,l,r,t);
    }
    if (isIntersect(u->rs->info.l,u->rs->info.r
    ↪ fo.r,l,r)) {
        modify(u->rs,l,r,t);
    }
    pull(u);
}

Info query(Node *u,ll l,ll r) {
    if (u->info.l>=l && u->info.r<=r) {
        return u->info;
    }
    Info res;
    bool is_get=false;
    push(u);

```

```

    if (isIntersect(u->ls->info.l,u->ls->info.r
    ↪ fo.r,l,r)) {
        res=query(u->ls,l,r);
        is_get=true;
    }
    if (isIntersect(u->rs->info.l,u->rs->info.r
    ↪ fo.r,l,r)) {
        if (!is_get) {
            res=query(u->rs,l,r);
        }else {
            res=I::merge(res,query(u->rs,l,
            ↪ r));
        }
    }
    return res;
}
};

```

8.16 线段树上二分

```

// AC https://qoj.ac/submission/1263556
// 找到最靠近 r 的符合位置
template<class F> int findl(int r,F&&fun){
    return findl(1,r,0,sz-1,fun);
}

// 找到最靠近 r 的符合位置
template<class F> int findl(int o,int
    ↪ qr,int l,int r,F&&fun){
    if(l==r){
        return l;
    }
    pushdown(o);
    ll ls=o<<1,rs=ls|1;
    ll mid=l+r>>1;
    ll pos=-1;
    if(fun(tr[rs]) && mid+1<=qr)
        ↪ pos=findl(rs,qr,mid+1,r,fun);
    if(pos!=-1) return pos;
    if(fun(tr[ls]))
        ↪ pos=findl(ls,qr,l,mid,fun);
    return pos;
}

// 找最靠近 l 的符合位置
template<class F> int findr(int l,F&&fun){
    return findr(1,l,0,sz-1,fun);
}

// 找最靠近 l 的符合位置
template<class F> int findr(int o,int
    ↪ ql,int l,int r,F&&fun){
    if(l==r){
        return l;
    }
    pushdown(o);
    ll ls=o<<1,rs=ls|1;
    ll mid=l+r>>1;
    ll pos=-1;
    if(fun(tr[ls]) && mid>=ql)
        ↪ pos=findr(ls,ql,l,mid,fun);
    if(pos!=-1) return pos;
    if(fun(tr[rs]))
        ↪ pos=findr(rs,ql,mid+1,r,fun);
    return pos;
}

```

```
}

```

维护的线段树上二分。

```
// 找最靠近 l 的不符合位置
template<class F> int findr(int l, F&&fun){
    return findr(l, l, 0, sz-1, 0, fun);
}
// 找最靠近 l 的不符合位置
template<class F> int findr(int o, int
    ql, int l, int r, ll sum, F&&fun){
    if(l==r){
        return l;
    }
    pushdown(o);
    ll ls=o<<1, rs=ls|1;
    ll mid=l+r>>1;
    ll pos=-1;
    if(fun(tr[ls]*bei+sum) && mid>=ql)
        pos=findr(ls, ql, l, mid, sum, fun);
    if(pos!=-1) return pos;
    if(fun(tr[rs]*bei+sum+tr[ls]*bei))
        pos=findr(rs, ql, mid+1, r, sum+tr[ls]*
        bei, fun);
    return pos;
}
```

8.16.1 前缀含负数时找区间第一个前缀和 $\geq x$ 的位置

前缀单调不减时 (区间值非负), 节点维护 sum 就够: 左子 $sum \geq x$ 进左, 否则减掉左子和进右。但前缀含负数时 sum 不再代表「这段能达到的最大前缀」, 必须额外维护 pre (节点覆盖区间内、从该区间最左端起算的最大前缀和), $merge$ 时 $pre = \max(a.pre, a.sum + b.pre)$ 。线段树二分按「当前累积前缀 cur 加上左子的 pre 能否 $\geq x$ 」决定走左还是走右, 走右前把 cur 累加上左子的 sum (不是 pre)。

```
struct Info
{
    ll sum;
    ll pre;
};

Info operator+(const Info &a, const Info &b)
{
    return {a.sum + b.sum, max(a.pre, a.sum +
        b.pre)};
}

int findPrefix(int u, int l, int r, ll &cur, ll
    x)
{
    if (l == r)
        return l;

    int mid = (l + r) >> 1;

    if (cur + seg[u << 1].pre >= x)
        return findPrefix(u << 1, l, mid, cur,
            x);
}
```

```
cur += seg[u << 1].sum;
return findPrefix(u << 1 | 1, mid + 1, r,
    cur, x);
}
```

8.17 segValueModAdd 求全局最小众数的权值线段树

```
/*
 * 2026-03-25: 求全局最小众数的权值线段树
 * AC https://codeforces.com/gym/105941/submission/368114371
 */
struct Tag {
    ll add = 0;
    bool need_apply = false;

    void apply(const Tag &t) {
        add += t.add;
        need_apply = true;
    }

    friend ostream &operator<<(ostream &os,
        const Tag &t) {
        os << "add:" << t.add;
        return os;
    }
};

string to_string(const {
    stringstream ss;
    ss << *this;
    return ss.str();
}

};

struct Info {
    // 注意啊, 这里除了 L 和 R
    // 以外的所有你所添加的东西,
    // 你都要好好想一想其初值是什么。
    // 不过我们其实改进了这个 query 函数,
    // 所以说如果说你给的这个数组当中所有的值都
    // 是所有的值都是有的话, 其实不用考虑初值。
    // 如果一定要赋初值的话, 注意就是 sum
    // 类的都不要赋 infinite 然后只有 max 和 min
    // 的你要好好想想。
    struct Cnt_val {
        ll cnt, val = INF;

        bool operator<(const Cnt_val &o) const {
            if (o.cnt != cnt) return cnt <
                o.cnt;
            return val > o.val;
        }
    };

    ll l = 0, r = 0;
    Cnt_val mx{};

    static Info merge(const Info &a, const Info
        &b) {

```



```

    Info res;
    res.l = a.l;
    res.r = b.r;
    res.mx = max(a.mx, b.mx);
    return res;
}

void apply(const Tag &t) {
    if (!t.need_apply) return;
    mx.cnt += t.add;
}

friend ostream &operator <<(ostream &os,
    ↪ const Info &a) {
    os << "[" << a.l << ";" << a.r << "]:"
    ↪ << "mx_cnt:" << a.mx.cnt << "
    ↪ mx_val:" << a.mx.val;
    return os;
}

string to_string() const {
    stringstream ss;
    ss << *this;
    return ss.str();
}
};

using I = Info;
using T = Tag;

struct SEG {
    ll N;
    vector<I> infos;
    vector<T> tags;

    static bool isIntersect(ll l1, ll r1, ll
    ↪ l2, ll r2) {
        if (!(l1 > r2 || r1 < l2)) return true;
        return false;
    }

    SEG(ll N, const vector<I> &a) {
        infos.resize(4 * N + 5);
        tags.resize(4 * N + 5);
        this->N = N;
        build(a, 1, 1, N);
    }

    void push(ll u) {
        if (infos[u].l == infos[u].r) {
            return;
        }
        if (!tags[u].need_apply) {
            return;
        }
        infos[u << 1].apply(tags[u]);
        infos[u << 1 | 1].apply(tags[u]);
        tags[u << 1].apply(tags[u]);
        tags[u << 1 | 1].apply(tags[u]);
        tags[u] = {};
    }

    void pull(ll u) {

```

```

        infos[u] = I::merge(infos[u << 1],
        ↪ infos[u << 1 | 1]);
    }

    void build(const vector<I> &a, ll u, ll l,
    ↪ ll r) {
        if (l == r) {
            infos[u] = a[l];
            infos[u].l = l;
            infos[u].r = r;
            return;
        }
        ll mid = l + r >> 1;
        build(a, u << 1, l, mid);
        build(a, u << 1 | 1, mid + 1, r);
        pull(u);
    }

    I query(ll u, ll l, ll r) {
        if (infos[u].l >= l && infos[u].r <= r)
            ↪ {
                return infos[u];
            }
        push(u);
        I res{};
        // 为什么要设置 isget 这个 bool 呢?
        ↪ 其实是因为防止使用初值进行 merge
        // 我们并不保证使用初值进行 merge
        ↪ 一定是对的。
        bool isget = false;
        if (!(infos[u << 1].r < l || infos[u <<
        ↪ 1].l > r)) {
            res = query(u << 1, l, r);
            isget = true;
        }
        if (!(infos[u << 1 | 1].r < l ||
        ↪ infos[u << 1 | 1].l > r)) {
            if (isget) {
                res = I::merge(res, query(u <<
                ↪ 1 | 1, l, r));
            } else {
                res = query(u << 1 | 1, l, r);
            }
        }
        return res;
    }

    void assign(ll u, ll pos, const I &val) {
        if (infos[u].l == infos[u].r &&
        ↪ infos[u].l == pos) {
            infos[u] = val;
            infos[u].l = pos;
            infos[u].r = pos;
            return;
        }
        push(u);
        if (pos >= infos[u << 1].l && pos <=
        ↪ infos[u << 1].r) {
            assign(u << 1, pos, val);
        }
        if (pos >= infos[u << 1 | 1].l && pos
        ↪ <= infos[u << 1 | 1].r) {
            assign(u << 1 | 1, pos, val);
        }
    }

```

```

    pull(u);
}

void modify(ll u, ll l, ll r, const T &t) {
    if (infos[u].l >= l && infos[u].r <= r)
        ↪ {
            infos[u].apply(t);
            tags[u].apply(t);
            return;
        }
    // 在判断完全覆盖之后再去push, 不影响结果,
    ↪ 会省一点操作
    push(u);
    if (isIntersect(infos[u << 1].l,
        ↪ infos[u << 1].r, l, r)) {
        modify(u << 1, l, r, t);
    }
    if (isIntersect(infos[u << 1 | 1].l,
        ↪ infos[u << 1 | 1].r, l, r)) {
        modify(u << 1 | 1, l, r, t);
    }
    pull(u);
}

// 返回第k个有值的下标, 当然,
↪ 要求线段树叶节点值是0或1
// ll findkth(ll u, ll cur_k) {
//     push(u);
//     if (infos[u].l == infos[u].r) {
//         return infos[u].l;
//     }
//     if (cur_k <= infos[u << 1].sum) {
//         return findkth(u << 1, cur_k);
//     }
//     return
    ↪ findkth(u << 1 | 1, cur_k - infos[u << 1].sum);
// }

//
// void opposite(ll u, ll pos) {
//     if (infos[u].l == infos[u].r &&
    ↪ infos[u].l == pos) {
//         infos[u].sum ^= 1;
//         return;
//     }
//     if (pos >= infos[u << 1].l && pos <=
    ↪ infos[u << 1].r) {
//         opposite(u << 1, pos);
//     }
//     if (pos >= infos[u << 1 | 1].l && pos
    ↪ <= infos[u << 1 | 1].r) {
//         opposite(u << 1 | 1, pos);
//     }
//     pull(u);
// }

template<class F>
ll findFirst(ll u, ll l, ll r, F &&fun) {
    push(u);
    if (infos[u].l == infos[u].r) {
        if (fun(infos[u])) return
            ↪ infos[u].l;
        return -1;
    }
}

```

```

ll res = -1;
if (isIntersect(infos[u << 1].l,
    ↪ infos[u << 1].r, l, r) &&
    ↪ fun(infos[u << 1])) {
    res = findFirst(u << 1, l, r, fun);
}
if (res != -1) return res;
if (isIntersect(infos[u << 1 | 1].l,
    ↪ infos[u << 1 | 1].r, l, r) &&
    ↪ fun(infos[u << 1 | 1])) {
    res = findFirst(u << 1 | 1, l, r,
        ↪ fun);
}
return res;
}
};

```

8.18 线段树可视化

```

/*
 *AC https://codeforces.com/gym/106169/submission/363483057
 ↪ ission/363483057
 * 线段树可视化 AC 记录
 */
#ifdef LOCAL
string node_str(ll u) const {
    string s = infos[u].to_string();
    if (tags[u].need_apply)
        s += " {" + tags[u].to_string() +
            ↪ "}";
    return s;
}

vector<ll> vis_w;
ll vis_depth;

void visualize() {
    vis_w.assign(4 * N + 2, 0);
    vis_depth = 0;
    cal_w(1, 0);
    vector<string> rows(vis_depth + 1);
    render(1, 0, 0, rows);
    for (auto &s: rows) {
        while (!s.empty() && s.back() == '
            ↪ ') s.pop_back();
        cerr << s << "\n";
    }
}

void cal_w(ll u, ll d) {
    vis_depth = max(vis_depth, d);
    if (infos[u].l == infos[u].r) {
        vis_w[u] = node_str(u).size();
        return;
    }
    cal_w(u << 1, d + 1);
    cal_w(u << 1 | 1, d + 1);
    vis_w[u] = vis_w[u << 1] + 3 + vis_w[u
        ↪ << 1 | 1];
}

```

```

static void safe_put(string &row, ll col,
    ↪ char c) {
    if (col < 0) return;
    if (col >= (ll) row.size())
    ↪ row.resize(col + 1, ' ');
    row[col] = c;
}

void render(ll u, ll sc, ll d,
    ↪ vector<string> &rows) {
    string label = node_str(u);
    if (infos[u].l == infos[u].r) {
        for (ll i = 0; i < (ll)
            ↪ label.size(); i++)
            safe_put(rows[d], sc + i,
                ↪ label[i]);
        return;
    }
    ll pipe = sc + vis_w[u << 1] + 1;
    ll labStart = max(pipe - (ll)
    ↪ label.size() / 2, (ll) 0);
    for (ll i = 0; i < (ll) label.size();
    ↪ i++)
        safe_put(rows[d], labStart + i,
            ↪ label[i]);
    for (ll i = d + 1; i <= vis_depth; i++)
        safe_put(rows[i], pipe, '|');
    render(u << 1, sc, d + 1, rows);
    render(u << 1 | 1, sc + vis_w[u << 1] +
    ↪ 3, d + 1, rows);
}
#endif

```

8.19 pb_ds 红黑树 (tree)

`__gnu_pbds::tree` 是 GCC 自带的策略式 BST (默认红黑树), 加上 `tree_order_statistics_node_update` 之后可在 $O(\log n)$ 内做第 k 小 (`find_by_order`) 和排名 (`order_of_key`), 等于 `std::set` + 内置 `order_statistics`。

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
using ordered_set = tree<int, null_type, less<>,
    rb_tree_tag,
    tree_order_statistics_node_update>;

```

`find_by_order(k)` (迭代器, 0-based):

- `find_by_order(0)`: 最小元素。
- `find_by_order(n - 1)`: 最大元素 (共 n 个时)。
- 越界 ($k \geq n$) 返回 `s.end()`。

`order_of_key(x)`: 返回严格小于 x 的元素个数, 即 x 插入后排序序列中的 0-based 索引。例: 集合 $\{2, 5, 8, 10, 15\}$, `order_of_key(8) = 2` (小于 8 的有 2 个), `order_of_key(12) = 4`, `order_of_key(20) = 5`。

自带去重 → 想要可重时改 `pair`: `tree` 同 `set` 一样去重。要存可重值时把 `key` 包成 `pair<value, id>`, `id` 取插入序号即可:

```

using ordered_multiset =
    tree<pair<int, int>, null_type,
        less<pair<int, int>>,
        rb_tree_tag,
        tree_order_statistics_node_update>;

```

查询值落在区间 $[l, r]$ 内有几个 (多重集):

```

// 用 (l - 1, INF) 排除 = l - 1 的等价类
// 用 (r + 1, 0) 排除 > r 的全部
ll cnt = s.order_of_key({r + 1, 0})
    - s.order_of_key({l - 1, INF});

```

应用 1: 树上 dfs + 启发式合并 + 区间第 k 小 (LeetCode `kthSmallest`): 每个节点维护一个 `ordered_set`, 子树合并时把小集合并进大集合, 在 dfs 离开节点前用 `find_by_order(k - 1)` 取答案。

```

#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
using ordered_set = tree<int, null_type, less<>,
    rb_tree_tag,
    tree_order_statistics_node_update>;

class Solution {
public:
    vector<int> kthSmallest(vector<int>& par,
        vector<int>& vals,
        vector<vector<int>>& queries) {
        vector<vector<int>> g(par.size() + 5);
        vector<vector<array<int, 2>>>
            que(par.size() + 5);
        for (int i = 1; i < par.size(); ++i) {
            g[par[i]].push_back(i);
        }
        for (int i = 0; i < queries.size();
            ↪ ++i) {
            que[queries[i][0]].push_back(
                {queries[i][1], i});
        }
        vector<int> ans(queries.size());
        auto dfs = [&](auto&& self, int u, int
            ↪ val)
            -> ordered_set* {
            val ^= vals[u];
            ordered_set* st = new ordered_set();
            st->insert(val);
            for (auto v : g[u]) {
                ordered_set* t = self(self, v,
                    ↪ val);
                // 启发式合并: 小集合并入大集合
                if (t->size() > st->size())
                    swap(st, t);
                for (int k : *t) st->insert(k);
                delete t;
            }
            for (auto& t : que[u]) {
                int k = t[0], idx = t[1];
                if (k > st->size()) {

```

```

        ans[idx] = -1; continue;
    }
    ans[idx] = *st->find_by_order(k
        ↪ - 1);
    }
    return st;
};

dfs(dfs, 0, 0);
return ans;
}

};

```

```

11 ans
    = type_setOfTime[type].order_of_
    ↪ _key(
        {r + 1, 0})
    - type_setOfTime[type].order_of_
    ↪ _key(
        {l - 1, INF});
cout << ans << "\n";
}
}
}

```

应用 2: 按某个 key 维护时间序列、查 $[l, r]$ 计数(上师校赛 Problem F. 统计数据): 用 `priority_queue` 拿全局最早的事件, `map<ll, ordered_set<time_id>>` 按 type 分桶; 查询 `type=t` 在时间窗 $[l, r]$ 内的件数 = `order_of_key({r+1, 0}) - order_of_key({l-1, INF})`

```

struct type_time {
    ll type, time, id;
    bool operator<(const type_time& o) const {
        if (time != o.time) return time >
            ↪ o.time;
        return id > o.id;
    }
};

struct time_id {
    ll time, id;
    bool operator<(const time_id& o) const {
        if (time != o.time) return time <
            ↪ o.time;
        return id < o.id;
    }
};

#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
using ordered_set = tree<time_id, null_type,
    ↪ less<>,
    ↪ rb_tree_tag,
    ↪ tree_order_statistics_node_update>;

void Solve() {
    ll Q; cin >> Q;
    priority_queue<type_time> pq;
    map<ll, ordered_set> type_setOfTime;
    auto pop = [&]() {
        if (pq.empty()) return;
        auto [type, time, id] = pq.top();
        pq.pop();
        type_setOfTime[type].erase({time, id});
    };
    FOR(i, 1, Q) {
        ll opt; cin >> opt;
        if (opt == 1) {
            ll type, time; cin >> type >> time;
            pq.push({type, time, i});
            type_setOfTime[type].insert({time,
                ↪ i});
        } else if (opt == 2) {
            pop();
        } else {
            ll type, l, r;
            cin >> type >> l >> r;

```

8.20 gp_hash_table (哈希表, 比 unordered map 快)

`gp_hash_table` 是 `__gnu_pbds` 提供的开放寻址哈希表，比 `std::unordered_map` 在 CF / AtCoder 标程数据下普遍快 2-5 倍。但默认的 `hash<int>` 是恒等映射，容易被构造数据卡到 $O(n)$ 单次，**务必自定义 `chash`**。参考 [Codeforces blog/60737](https://codeforces.com/blog/entry/60737) “Anti-hash test”。

本机实测对比 `gp_hash_table` / `cc_hash_table` / `unordered_map` (含 `resize` 收益) 见 [gist: gp_cc_hash](#)
速度测试: $N=5e6$ 下 `gp + resize` 比 `unordered_map` 快约 $4.85\times$; `gp` 的 `resize` 收益巨大 (插入 $3.8\times$ 提速), `cc` 的 `resize` 几乎无差。

简化版 (仅做 XOR 扰动):

```
const int RANDOM = chrono::high_resolution_clock
    ::now().time_since_epoch().count();
struct chash {
    int operator()(int x) const {
        return x ^ RANDOM;
    }
};
gp_hash_table<int, int, chash> table;
```

加强版 (splitmix64 风格哈希 + 堆地址扰动):

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

struct chash {
    const int RANDOM
        = (long long)(make_unique<char>().get())
        ^ chrono::high_resolution_clock::now()
        .time_since_epoch().count();

    static unsigned long long
    hash_f(unsigned long long x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) *
            ↪ 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) *
            ↪ 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    int operator()(int x) const {
        return hash_f(x) ^ RANDOM;
    }
};
```

```
// 可 resize 版别名 (默认 resize_policy 锁死
→ resize,
// 这里把第 7 个模板参数换成
→ external_size_access=true)
template<class K, class V, class H = chash>
using HashMap = gp_hash_table<K, V, H,
    equal_to<K>,
    direct_mask_range_hashing<>,
    linear_probe_fn<>,
    hash_standard_resize_policy<
        hash_exponential_size_policy<>,
        hash_load_check_resize_trigger<true>,
        true>>;
```

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

// 上面定义好的 chash
gp_hash_table<int, int, chash> mp;

mp[42] = 7; // 直接下标插入 / 修改
mp[42]++; // 自增
if (mp.find(42) != mp.end()) { /* ... */ }
mp.erase(42); // 按 key 删
ll cnt = mp.size(); // 元素个数
for (auto& [k, v] : mp) { // 遍历, 无序
    cout << k << " -> " << v << "\n";
}
```

pair 当 key (哈希两次再合并):

```
struct pair_chash {
    size_t operator()(const pll& p) const {
        return chash{}(p.first) * 1000003ull
            + chash{}(p.second);
    }
};
gp_hash_table<pll, ll, pair_chash> mp;
mp[{x, y}] += 1;
```

预分配桶数 (*resize*): 用上面定义好的 HashMap 别名即可, 免写 7 层模板参数。

```
HashMap<int, int> mp;
mp.resize(1 << 20); // 预分配 1M 桶
```

8.21 rope ($O(1)$ 拷贝的可持久化序列)

`__gnu_cxx::rope` 与 `pb_ds` 同属 GCC libstdc++ 扩展 (MSVC 无), 底层是 **平衡 concat 树**, 节点 immutable + 引用计数。复制赋值仅共享根指针, 是 $O(1)$; 修改走 **path copying**: 只复制根到目标位置的 $O(\log n)$ 个节点, 其他子树原样共享。保留历史版本不需要额外 API——把 rope 对象本身存起来即可, 每个新版本均摊 $O(\log n)$ 空间, q 次操作总空间 $O(q \log n)$, 与可持久化线段树同档。

基本用法 (位置寻址, 下标 0-based):

```
#include <ext/rope>
using namespace __gnu_cxx;
```

```
rope<int> R;
R.push_back(7); // 末尾追加
R.insert(2, 99); // 在下标 2 插入,
→  $O(\log n)$ 
R.erase(1, 3); // 从下标 1 起删 3 个,
→  $O(\log n)$ 
int x = R[0]; // 单点访问  $O(\log n)$ ,
→ 不是  $O(1)$ 
auto S = R.substr(2, 5); // 取  $[2, 5)$ ,  $O(\log n)$ , 共享子树
→  $n$ , 共享子树
size_t n = R.size();
```

持久化 = 普通 copy / 赋值。把每个版本装进数组即拿到版本树:

```
vector<rope<int>> ver;
ver.push_back(rope<int>()); // 版本
→ 0: 空
for (int i = 1; i <= q; i++) {
    ver.push_back(ver[parent[i]]); //
    → 从父版本 fork,  $O(1)$ 
    ver.back().insert(pos[i], val[i]); //
    → 在新版本上改
}
int ans = ver[k][i]; //
→ 查询版本 k 的下标 i
```

8.22 FHQ-treap (无旋 Treap)

无旋 Treap 用 split / merge 两个 $O(\log n)$ 操作做底, 实现 insert / del / getrank / kth / prex / sufx 全套平衡树标准接口。下方实现用 namespace 而非 struct: 既起到打包作用, 又避免实例化时的默认值开销。

```
// 使用时简化可以这样: namespace F = FHQ;
// AC https://loj.ac/s/2331639
// AC https://www.luogu.com.cn/record/219062740
namespace FHQ {
    const int maxn
        = static_cast<ll>(1.1e6) + 10;
    using KEY = int;
    KEY key[maxn];
    int pri[maxn], ls[maxn], rs[maxn],
    → size[maxn];
    int root = 0, cnt = 0;
    // pri 超 int 范围问题不大, 正负无所谓
    mt19937 rng(chrono::steady_clock::now()
        .time_since_epoch().count())
    → );

    inline void makeN(KEY val) {
        ++cnt; pri[cnt] = rng(); key[cnt] = val;
        ls[cnt] = 0; rs[cnt] = 0; size[cnt] = 1;
    }

    inline void pushup(int node) {
        size[node] = size[ls[node]]
            + size[rs[node]] + 1;
    }

    // 把树 node 按值 val 分裂为两棵:  $x (\leq val) +$ 
    →  $y (> val)$ 
```

```

void split(int node, KEY val, int& x, int&
→ y) {
    if (!node) { x = 0; y = 0; return; }
    if (key[node] <= val) {
        x = node;
        split(rs[node], val, rs[node], y);
    } else {
        y = node;
        split(ls[node], val, x, ls[node]);
    }
    pushup(node);
}
// 按 pri 合并, pri 小的在上面 (小根堆性质)
int merge(int x, int y) {
    if (!x || !y) return x + y;
    if (pri[x] < pri[y]) {
        rs[x] = merge(rs[x], y);
        pushup(x); return x;
    } else {
        ls[y] = merge(x, ls[y]);
        pushup(y); return y;
    }
}
inline void insert(KEY val) {
    int x, y;
    split(root, val, x, y);
    makeN(val);
    root = merge(x, cnt);
    root = merge(root, y);
}
inline void del(KEY val) {
    int x, y, z;
    split(root, val, x, y);
    split(x, val - 1, x, z);
    z = merge(ls[z], rs[z]);
    root = merge(x, z);
    root = merge(root, y);
}
inline int getrank(KEY val) {
    int x, y;
    split(root, val - 1, x, y);
    int ret = size[x] + 1;
    root = merge(x, y);
    return ret;
}
KEY _kth(int node, int rank) {
    if (size[ls[node]] >= rank)
        return _kth(ls[node], rank);
    if (size[ls[node]] + 1 >= rank)
        return key[node];
    return _kth(rs[node],
        rank - size[ls[node]] - 1);
}
inline KEY kth(int rank) {
    return _kth(root, rank);
}
// 前驱: < val 的最大值; 后继: > val 的最小值
inline KEY prex(KEY val) {
    return kth(getrank(val) - 1);
}
inline KEY sufx(KEY val) {
    return kth(getrank(val) + 1);
}
}

```

```

}

```

8.23 替罪羊树 (Scapegoat Tree)

替罪羊树通过部分重建保持 $\alpha = 0.7$ 平衡, 每次 insert 后若发现某子树的「最大孩子规模 / 子树规模」超过 α , 就把整棵子树拍平 → 中序收集存活节点 → 对半重建。下方实现支持惰性删除 (isdel), 鲁棒性比较强: 不要求 del / prex / sufx 操作合法, getrank 也不要求 x 曾经出现过。

```

// 代码具有一定的鲁棒性, 不要求 del 操作合法
// prex / sufx 也不要求合法
// getrank 操作不要求 x 曾经出现过
class ScapegoatTree {
private:
    typedef int KEY; // key 较大时改为 ll
    // 空间约 30MB, maxn=1.1e6+7
    static const int
        maxn = static_cast<int>(1.1e6) + 7,
        inf = static_cast<KEY>(2e9) + 7;
    static constexpr DB alpha = 0.7;
    // diff 为节点总数 (用于调整)
    // size 为数字总数 (用于询问)
    int ls[maxn], rs[maxn], count[maxn],
        diff[maxn], size[maxn];
    bool isdel[maxn];
    KEY key[maxn];
    int collect[maxn], ci = 0;
    // 最上方不平衡节点 top, 其父 father
    // side: 1=left, 0=right
    int top = 0, father = 0;
    bool side = false;
    // 0 是超级源点
    // 1 是辅助节点 (key=-inf), 从不进入 0
    int head = 1, cnt = 1;
    bool neebal = false;

    inline void makeN(int node, KEY val) {
        ls[node] = 0; rs[node] = 0;
        count[node] = 1; diff[node] = 1;
        size[node] = 1; key[node] = val;
    }

    inline void initN(int node) {
        if (node != 1 && node != 0) {
            ls[node] = 0; rs[node] = 0;
            size[node] = count[node];
            diff[node] = 1;
        } else {
            ls[node] = 0; rs[node] = 0;
            size[node] = count[node];
            diff[node] = 0;
        }
    }

    inline bool isbal(int node) {
        if (node == 1 || node == 0) return true;
        int mxDiff = max(diff[ls[node]],
            diff[rs[node]]);
        if (mxDiff > diff[node] * alpha)
            return false;
        return true;
    }
}

```



```

bool insert(int node, KEY val,
            int fa, bool isLeft) {
    if (key[node] == val) {
        ++count[node]; ++size[node];
        isdel[node] = false;
        return false;
    } else if (val > key[node]) {
        ++size[node];
        if (rs[node] == 0) {
            rs[node] = ++cnt;
            makeN(cnt, val);
            return true;
        } else {
            bool ret = insert(rs[node], val,
                              node, false);
            if (ret) {
                ++diff[node];
                if (!isbal(node)) {
                    top = node; father = fa;
                    side = isLeft;
                    neebal = true;
                }
            }
            return ret;
        }
    } else {
        ++size[node];
        if (ls[node] == 0) {
            ls[node] = ++cnt;
            makeN(cnt, val);
            return true;
        } else {
            bool ret = insert(ls[node], val,
                              node, true);
            if (ret) {
                ++diff[node];
                if (!isbal(node)) {
                    top = node; father = fa;
                    side = isLeft;
                    neebal = true;
                }
            }
            return ret;
        }
    }
}

void inorder(int node) {
    if (ls[node] != 0) inorder(ls[node]);
    if (!isdel[node]) collect[++ci] = node;
    if (rs[node] != 0) inorder(rs[node]);
}

void rebuild(int l, int r) {
    int mid = l + r >> 1;
    int node = collect[mid];
    initN(node);
    if (l >= r) return;
    if (l <= mid - 1) {
        rebuild(l, mid - 1);
        int n1 = collect[(l + mid - 1) >>
                        1];
        ls[node] = n1;
        size[node] += size[n1];
        diff[node] += diff[n1];
    }
}

```

```

    if (r >= mid + 1) {
        rebuild(mid + 1, r);
        int n2 = collect[(r + mid + 1) >>
                        1];
        rs[node] = n2;
        size[node] += size[n2];
        diff[node] += diff[n2];
    }
}

inline void balan() {
    ci = 0;
    inorder(top);
    if (ci > 0) {
        int lhead = collect[(1 + ci) / 2];
        if (top == head) head = lhead;
        if (side) ls[father] = lhead;
        else rs[father] = lhead;
        rebuild(1, ci);
    }
    ci = 0; top = 0; neebal = false;
    father = 0; side = false;
}

// 1: count>1 减完即可; 2: 彻底删除; 0: 没有
int del(int node, KEY val) {
    if (key[node] == val) {
        if (count[node] > 1) {
            --count[node]; --size[node];
            return 1;
        } else {
            count[node] = 0;
            --diff[node]; --size[node];
            isdel[node] = true;
            return 2;
        }
    } else if (val > key[node]) {
        int ret = 0;
        if (rs[node] != 0)
            ret = del(rs[node], val);
        if (ret != 0) --size[node];
        if (ret == 2) --diff[node];
        return ret;
    } else {
        int ret = 0;
        if (ls[node] != 0)
            ret = del(ls[node], val);
        if (ret != 0) --size[node];
        if (ret == 2) --diff[node];
        return ret;
    }
}

int getrank(int node, KEY val, int k) {
    if (key[node] == val) {
        k += size[ls[node]];
        k += 1;
        return k;
    } else if (val > key[node]) {
        k += size[ls[node]];
        k += count[node];
        if (rs[node] != 0)
            return getrank(rs[node], val,
                            k);
        return k + 1;
    } else {

```

```

        if (ls[node] != 0)
            return getrank(ls[node], val,
                ↪ k);
        return k + 1;
    }
}
KEY kth(int node, int k) {
    if (size[ls[node]] >= k) {
        if (ls[node] != 0)
            return kth(ls[node], k);
        return -inf;
    } else {
        if (size[ls[node]]
            + count[node] >= k)
            return key[node];
        if (rs[node] != 0)
            return kth(rs[node],
                k - (size[ls[node]]
                    + count[node]));
        return -inf;
    }
}
public:
    inline void insert(KEY val) {
        // 0 当 fa 防止 fa 指向自己造成的问题
        insert(head, val, 0, false);
        if (neebal) balan();
    }
    inline void del(KEY val) {
        del(head, val);
    }
    inline int getrank(KEY val) {
        return getrank(head, val, 0);
    }
    inline KEY kth(int k) {
        return kth(head, k);
    }
    // 前驱: < val 且最大
    inline KEY prex(KEY val) {
        return kth(getrank(val) - 1);
    }
    // 后继: > val 且最小
    inline KEY sufx(KEY val) {
        return kth(getrank(val) + 1);
    }
}
ScapeGoatTree() {
    // 由于可能有 0, 设置 key[1]=-inf
    key[1] = -inf;
    // 异常时输出节点 0 的值即 -inf 显眼
    key[0] = -inf;
}
inline void clear() {
    memset(key, 0, sizeof(key));
    memset(count, 0, sizeof(count));
    memset(isdel, 0, sizeof(isdel));
    memset(ls, 0, sizeof(ls));
    memset(rs, 0, sizeof(rs));
    memset(size, 0, sizeof(size));
    memset(diff, 0, sizeof(diff));
    memset(collect, 0, sizeof(collect));
    ci = 0; cnt = 1; head = 1;
    key[1] = -inf; key[0] = -inf;
}

```

```
};
```

8.24 主席树 (可持久化权值线段树)

主席树即可持久化权值线段树, 每次单点更新只新建 $O(\log n)$ 个节点, 多保存一份「版本根」。处理静态区间第 k 小问题时, 第 i 个版本对应前 i 个数构成的权值集合, 区间 $[l, r]$ 的第 k 小按左子树差量 $tree[ls[v]].sum - tree[ls[u]].sum$ 决定向左还是向右走。

空间按 $25n$ 预留 ($\log_2 5 \times 10^5 \approx 19$, 留余量)。vector+reserve 实现, 可放局部作用域; 在线插入需保证新元素已在建树时出现在离散化表里。

```

// 值域可持久化线段树
class Segment {
    struct node { int sum = 0, leftChild = 0,
        ↪ rightChild = 0; };
    vector<node> tree;
    vector<int> root;
    vector<ll> discreted, origin;
    int idx = 0, N;

    void insert(int prev, int &cur, int l, int
        ↪ r, int p) {
        cur = ++idx;
        tree.push_back(node());
        tree[cur] = tree[prev];
        tree[cur].sum++;
        if (l == r) return;
        int mid = (l + r) >> 1;
        if (p <= mid)
            ↪ insert(tree[prev].leftChild,
                ↪ tree[cur].leftChild, l, mid, p);
        else
            ↪ insert(tree[prev].rightChild,
                ↪ tree[cur].rightChild, mid + 1, r,
                ↪ p);
    }
    ll queryKthNumber(int prev, int cur, int l,
        ↪ int r, int x) {
        if (l == r) return discreted[l - 1];
        int leftSum =
            ↪ tree[tree[cur].leftChild].sum -
            ↪ tree[tree[prev].leftChild].sum;
        int mid = (l + r) >> 1;
        if (leftSum < x)
            return queryKthNumber(tree[prev].ri
                ↪ ghtChild, tree[cur].rightChild,
                ↪ mid + 1, r, x - leftSum);
        else
            return queryKthNumber(tree[prev].le
                ↪ ftChild, tree[cur].leftChild,
                ↪ l, mid, x);
    }
    ll queryRank(int prev, int cur, int l, int
        ↪ r, int p) {
        if (p < l) return 0;
        if (r <= p) return tree[cur].sum -
            ↪ tree[prev].sum;
        int mid = (l + r) >> 1;
        return queryRank(tree[prev].leftChild,
            ↪ tree[cur].leftChild, l, mid, p)

```



```

        + queryRank(tree[prev].rightChild,
        ↪ tree[cur].rightChild, mid + 1,
        ↪ r, p);
    }
    int id(ll x) { return
    ↪ lower_bound(all(discreted), x) -
    ↪ discreted.begin() + 1; }

public:
    Segment(int n) : tree(1, node()), root(n +
    ↪ 1), origin(n + 1) {
        tree.reserve(2511 * n + 5);
        for (int i = 1; i <= n; i++) {
            cin >> origin[i];
            discreted.push_back(origin[i]);
        }
        sort(all(discreted));
        discreted.erase(unique(all(discreted)),
        ↪ discreted.end());
        N = discreted.size();
        for (int i = 1; i <= n; i++)
            insert(root[i - 1], root[i], 1, N,
            ↪ id(origin[i]));
    }

    // 区间第 k 小元素
    ll queryKthNumber(int l, int r, int k) {
        return queryKthNumber(root[l - 1],
        ↪ root[r], 1, N, k);
    }
    // 区间 <= x 的数字个数 (x 可未出现过, 故走
    ↪ upper_bound 不走 id())
    ll queryRank(int l, int r, ll x) {
        int p = upper_bound(all(discreted), x)
        ↪ - discreted.begin();
        if (p == 0) return 0;
        return queryRank(root[l - 1], root[r],
        ↪ 1, N, p);
    }
};

```

典型用法 (构造时读入 n 个数, 区间第 k 小 / 区间 $\leq x$ 计数):

```

inline void solve() {
    int n, m; cin >> n >> m;
    Segment tr(n); // 构造时一次性读入 n 个数
    FOR(_, 1, m) {
        int l, r, k; cin >> l >> r >> k;
        cout << tr.queryKthNumber(l, r, k) <<
        ↪ "\n";
    }
}

```

8.25 分块 (单点修改)

分块以 $\sqrt{N}$ 为块长, 把数组切成若干块, 块内保持有序。查询区间 $[a, b]$ 内 $\geq c$ (或 $\leq c$) 的元素个数: 散块 (左右两端不完整的块) 暴力扫; 整块用 `lower_bound` / `upper_bound` 二分。单点修改 $O(\sqrt{N})$, 单次查询 $O(\sqrt{N} \log \sqrt{N})$ 。

```

// AC https://www.luogu.com.cn/record/220963190
// 单点修改 + 查询区间 [a, b] 内 >=c 的数的个数
ll L[MAXN], R[MAXN];
vector<arr2> g[MAXN]; // 记得清空

inline void solve() {
    cin >> N;
    ll sqn = sqrt(N), idx = 1;
    L[idx] = 1; R[idx] = sqn;
    FOR(i, 1, N) {
        ll a; cin >> a;
        if (i <= R[idx]) {
            g[idx].pb({a, i});
        } else {
            ++idx;
            L[idx] = i; R[idx] = i + sqn - 1;
            g[idx].pb({a, i});
        }
    }
    FOR(i, 1, idx) sort(all(g[i]));
    L[idx + 1] = INF;
    cin >> Q;
    FOR(_, 1, Q) {
        ll op, a, b, c;
        cin >> op;
        if (op == 0) {
            cin >> a >> b >> c;
            ll ida = upper_bound(L + 1,
            ↪ L + 1 + idx, a) - L;
            ida -= 1;
            ll idb = upper_bound(L + 1,
            ↪ L + 1 + idx, b) - L;
            idb -= 1;
            ll ans = 0;
            // 左散块
            FOR(i, 0, SZ(g[ida]) - 1) {
                if (g[ida][i][1] >= a
                ↪ && g[ida][i][1] <= b
                ↪ && g[ida][i][0] >= c)
                    ++ans;
            }
            // 右散块 (若与左不同块)
            if (ida != idb) {
                FOR(i, 0, SZ(g[idb]) - 1) {
                    if (g[idb][i][1] >= a
                    ↪ && g[idb][i][1] <= b
                    ↪ && g[idb][i][0] >= c)
                        ++ans;
                }
            }
            // 整块二分
            FOR(i, ida + 1, idb - 1) {
                // <=c 版本: upper_bound(c, INF)
                // ll id =
                ↪ upper_bound(all(g[i]),
                ↪ (arr2){c, INF})
                ↪ - g[i].begin();
                // ans += id;
                ll id = lower_bound(all(g[i]),
                ↪ (arr2){c, 0})
                ↪ - g[i].begin();
                ans += SZ(g[i]) - id;
            }
        }
    }
}

```

```

        cout << ans << "\n";
    } else { // 单点修改 a → b
        cin >> a >> b;
        ll id = upper_bound(L + 1,
                             L + 1 + idx, a) - L;
        id -= 1;
        FOR(i, 0, SZ(g[id]) - 1) {
            if (g[id][i][1] == a) {
                g[id][i][0] = b;
                break;
            }
        }
        sort(all(g[id]));
    }
}
}

```

8.26 回滚莫队

普通莫队需要支持加 + 删两种 update。当题目里「删」很难写（如众数：删掉最大频次后要回到第二大频次，没有方便的 $O(1)$ 回退）时，用回滚莫队：每个询问的左端点对应到所属块，按 (块号, r) 排序后， r 单调递增；左端点每次从块右端开始向左扩展，扫描完后只 **minus 撤销**（不动 r ）。如此整个过程只用 add，加完即丢，不需要写 del 的逆操作。

l, r 同块的情况单独暴力处理（块内最多 $\sqrt{N}$ 个元素）。下方实现求区间众数 $\geq w$ (LeetCode threshold-majority-queries)：

```

struct Que { ll bid, l, r, w, i; };
bool cmp(const Que& a, const Que& b) {
    if (a.bid != b.bid) return a.bid < b.bid;
    return a.r < b.r;
}

class Solution {
public:
    vector<int> subarrayMajority(
        vector<int>& nums,
        vector<vector<int>>& queries) {
        ll Q = SZ(queries);
        ll n = max(1ll,
                   SZ(nums) / (1ll)sqrtl(Q));
        vector<int> ans(Q);
        vector<Que> que;
        FOR(i, 0, Q - 1) {
            ll l = queries[i][0],
                r = queries[i][1],
                w = queries[i][2];
            ll bid = l / n;
            ll rbid = r / n;
            if (bid == rbid) {
                // l, r 同块, 暴力
                unordered_map<ll, ll> cnt;
                ll lans = INF, mxc = 0;
                FOR(j, l, r) {
                    cnt[nums[j]]++;
                    if (cnt[nums[j]] == mxc)
                        lans = min<ll>(lans,
                                       nums[j]);
                }
            }
        }
    }
}

```

```

        else if (cnt[nums[j]] >
                 → mxc) {
            mxc = cnt[nums[j]];
            lans = nums[j];
        }
    }
    if (lans == INF || mxc < w)
        ans[i] = -1;
    else ans[i] = lans;
} else {
    que.EB(bid, l, r, w, i);
}
}
sort(all(que), cmp);
ll lans = INF, mxc = 0;
unordered_map<ll, ll> freq;
auto add = [&](ll num, ll cc) {
    freq[num] += cc;
    if (freq[num] == mxc)
        lans = min(lans, num);
    else if (freq[num] > mxc) {
        mxc = freq[num];
        lans = num;
    }
};
auto minus = [&](ll num, ll cc) {
    freq[num] -= cc;
};
ll up = -1, upr = -1;
ll tmplans = INF, tmpmxc = 0;
FOR(i, 0, SZ(que) - 1) {
    auto& [bid, l, r, w, id] = que[i];
    if (bid != up) {
        lans = INF; mxc = 0;
        freq.clear();
        up = bid;
        upr = (bid + 1) * n;
    }
    // r 单调右扩
    for (; upr <= r; ++upr)
        add(nums[upr], 1);
    // 回滚莫队的精髓：备份 + 临时左扩
    tmplans = lans; tmpmxc = mxc;
    FOR(j, l,
         min<ll>((bid + 1) * n - 1,
                 SZ(nums) - 1))
        add(nums[j], 1);
    if (lans == INF || mxc < w)
        ans[id] = -1;
    else ans[id] = lans;
    // 回滚
    lans = tmplans; mxc = tmpmxc;
    FOR(j, l,
         min<ll>((bid + 1) * n - 1,
                 SZ(nums) - 1))
        minus(nums[j], 1);
}
return ans;
}
}

```

9 图论

9.1 倍增法求 LCA

倍增法 LCA 用 $Anc[u][k]$ 存「u 的第 2^k 级祖先」。先把两点跳到同深度，再一起往上跳直到分叉，跳完返回 $Anc[a][0]$ 就是 LCA。模板题 P3398 仓鼠找 sugar：判两条树上路径是否相交，技巧是用 LCA 把“两条路径”翻译成“两个区间端点祖先关系”。

```
11 N,M,Q,K,T;
11 Deep[MAXN],Anc[MAXN][30];
bool vis[MAXN];
vector<ll> g[MAXN];

void dfs(ll x,ll fa){
    if(vis[x]) return;
    vis[x]=true;
    Deep[x]=Deep[fa]+1;
    Anc[x][0]=fa;
    FOR(i,0,SZ(g[x])-1){
        ll vnode=g[x][i];
        if(vis[vnode]) continue;
        dfs(vnode,x);
    }
}

inline ll LCA(ll a,ll b){    // 倍增法求 LCA
    if(Deep[a]<Deep[b]) swap(a,b);
    ROF(i,18,0){
        if(Deep[Anc[a][i]]>=Deep[b]){
            a=Anc[a][i];
        }
    }
    if(a==b) return a;
    ROF(i,18,0){
        if(Anc[a][i]!=Anc[b][i]){
            a=Anc[a][i],b=Anc[b][i];
        }
    }
    return Anc[a][0];
}

inline void solve(){
    cin>>N>>Q;
    FOR(i,1,N-1){
        ll u,v; cin>>u>>v;
        g[u].pb(v); g[v].pb(u);
    }
    dfs(1,0);
    FOR(j,1,18) FOR(i,1,N)
        ↪ Anc[i][j]=Anc[Anc[i][j-1]][j-1];

    FOR(i,1,Q){
        ll a,b,c,d; cin>>a>>b>>c>>d;
        if(Deep[a]<Deep[b]) swap(a,b);
        if(Deep[c]<Deep[d]) swap(c,d);
        ll lcaAB=LCA(a,b);
        ll lcaCD=LCA(c,d);
        // 保证共线: lcaCD 在 lcaAB 子树内 且
        ↪ lcaAB 在 c-d 路径上
        if(LCA(lcaAB,lcaCD)==lcaCD &&
            ↪ LCA(c,lcaAB)==lcaAB){ cout<<"Y\n";
            ↪ continue; }
    }
}
```

```
if(LCA(lcaAB,lcaCD)==lcaCD &&
    ↪ LCA(lcaAB,d)==lcaAB){ cout<<"Y\n";
    ↪ continue; }
// 看 lcaCD 在不在 a->lcaAB 上
if(LCA(lcaCD,lcaAB)==lcaAB &&
    ↪ LCA(lcaCD,a)==lcaCD){ cout<<"Y\n";
    ↪ continue; }
if(LCA(lcaCD,lcaAB)==lcaAB &&
    ↪ LCA(lcaCD,b)==lcaCD){ cout<<"Y\n";
    ↪ continue; }
cout<<"N\n";
}
}
// AC https://www.luogu.com.cn/record/217989209
```

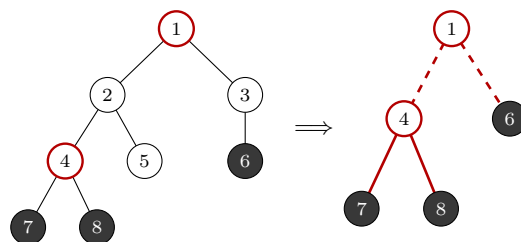
9.2 虚树 (Virtual Tree)

适用场景：多组询问，每次给出一批关键点，要在树上做只跟这些关键点有关的 DP / 贪心。保证 $\sum k \leq n$ ，但单次询问若还遍历整棵原树就是 $O(nq)$ 。虚树把每次的规模压到 $O(k)$ ：只保留关键点以及任意两关键点的 LCA，其余点全部缩掉，两点间的原树路径压成一条虚边。总复杂度 $O(\sum k \log k)$ (LCA 用 dfn 序 ST 表做到 $O(1)$ ，瓶颈在按 dfn 排序)。

关键事实：所有关键点两两的 LCA 集合，等于「按 dfn 排序后相邻两点的 LCA」集合——所以只需算 $k-1$ 次 LCA，不必两两枚举。

构造步骤 (本模板用的是“排序 + 相邻连边”版，不写单调栈，短且不易错)：

1. 关键点按 dfn 升序排序；
2. 对每对相邻关键点求 LCA，一并塞进点集；
3. 再按 dfn 排序并去重，得到虚树点集；
4. 对排好序的点集，把 $LCA(keys_i, keys_{i+1})$ 连向 $keys_{i+1}$ ，即得虚树的所有边。



关键点 (黑) {6, 7, 8}：相邻 LCA 补进 4 = $LCA(7, 8)$ 与 1 = $LCA(8, 6)$ (红圈)，点 2, 3, 5 被缩掉。虚线虚边表示它在原树里对应一条多边形路径 (如 $1 \rightarrow 4$ 实为 $1-2-4$)，做 DP 时边权取 dep 之差。

易错点：

- 清空虚树邻接表只能清本次用到的点 (for u in keys: $_g[u].clear()$)，整表清是 $O(n)$ ，多测直接退化。
- 虚树的根是 $keys.front()$ (dfn 最小者)，不一定是原树根 1；DFS 起点要用它。

- 关键点集合会被构造过程就地扩充，需要原始关键点判定时先拷一份（下面的 `origin_keys`）。
- 判「两个关键点在原树里是否直接相邻」要查原邻接表，不能看虚树边——虚边可能跨了很多原边。
- 黄色高亮的两处 `init()` 是惰性初始化（`isInit` 保证只跑一遍 `dfs` + `ST` 表预处理）：`lca_ab` 与 `build_virtual_tree` 各自开头都要调，才能不管调用顺序都拿到正确的 `dfn` / `Dep` / `Anc`。漏掉就是全 0 的静默 WA。

例题 CF 613D Kingdom and its Cities：每次给 k 个关键城市，问最少删几个非关键点使关键点两两不连通，两个关键点直接相邻则无解。虚树上贪心：`dfs` 返回子树里「还没被切断的关键点数」`cnt`——当前点是关键点就把子树里所有 `cnt` 都就地切掉（`ans += cnt`）并返回 1；否则 `cnt ≥ 2` 时删自己最划算（`ans++`，返回 0），`cnt ≤ 1` 时原样上传。

```
struct Tree {
    vector<vector<int>> g; // 原树邻接表
    vector<int> dfn; // dfs 序，虚树排序的依据
    vector<int> rev; // rev[dfn[u]] = u, dfn
    ↪ 反查节点
    vector<int> Fa; // 父亲（替代原来的 Anc
    ↪ 倍增表）
    vector<ll> Dep; // 深度，LCA 不再需要，但树形
    ↪ DP 常用，保留
    vector<vector<int>> st; // st[j][i]: dfn
    ↪ 区间 [i, i+2^j) 内 dfn 最小的父亲
    int n, dfIdx = 0;
    const int LOG;
    bool isInit = false;

    Tree(int _n) : n(_n), LOG(_lg(n) + 1) {
        g.resize(n + 2);
        dfn.assign(n + 2, 0);
        rev.assign(n + 2, 0);
        Fa.assign(n + 2, 0);
        Dep.assign(n + 2, 0);
        // 只有 LOG+1 次堆分配；原来的
        ↪ Anc.resize(n+2, vector<int>(LOG+1))
        ↪ 是 n+2 次
        st.assign(LOG + 1, vector<int>(n + 2,
        ↪ 0));
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs(int u, int fa, ll dep) {
        Fa[u] = fa;
        dfn[u] = ++dfIdx;
        rev[dfIdx] = u;
        Dep[u] = dep;
        for (auto to: g[u]) {
            if (to == fa || to == u) {
                continue;
            }
        }
    }
};
```

```
dfs(to, u, dep + 1);
}
}

// 按 dfn 取较小者 (ST 表的合并函数)
inline int mn(int x, int y) const {
    return dfn[x] < dfn[y] ? x : y;
}

void init() {
    // 幂等：多次调用只跑一遍
    if (isInit) {
        return;
    }
    isInit = true;
    dfs(1, 0, 0);
    // st[0][i] = dfn 为 i 的点的父亲。i=1
    ↪ 是根，父亲是 0 号虚点 (dfn=0 会污染
    ↪ min)，
    // 但查询区间左端 >= 2 永远取不到它，
    ↪ 这里仍填根自己保证安全
    for (int i = 2; i <= dfIdx; ++i) {
        st[0][i] = Fa[rev[i]];
    }
    st[0][1] = rev[1];
    for (int j = 1; j <= LOG; ++j) {
        for (int i = 1; i + (1 << j) - 1 <=
        ↪ dfIdx; ++i) {
            st[j][i] = mn(st[j - 1][i],
            ↪ st[j - 1][i + (1 << (j -
            ↪ 1))]);
        }
    }
}

int lca_ab(int a, int b) {
    init();
    if (a == b) {
        return a;
    }
    int x = dfn[a], y = dfn[b];
    if (x > y) {
        swap(x, y);
    }
    ++x; // 区间 (dfn[a], dfn[b]] ——
    ↪ 左开右闭，长度 >= 1
    int j = _lg(y - x + 1);
    return mn(st[j][x], st[j][y - (1 << j)
    ↪ + 1]);
}

// keys 就地扩充成「虚树点集」；_g
↪ 是虚树邻接表（有向：父 -> 子）
void build_virtual_tree(vector<vector<int>>
    ↪ &_g, vector<int> &keys) {
    init();
    auto byDfn = [&](const int &a, const
    ↪ int &b) {
        return dfn[a] < dfn[b];
    };
    sort(all(keys), byDfn);
    ll sz_key = SZ(keys);
    keys.reserve(sz_key * 2); //
    ↪ 避免扩充过程中反复 realloc
```

```

    for (int i = 0; i < sz_key - 1; ++i) {
        // 只需相邻两点的 LCA
        keys.push_back(lca_ab(keys[i],
            ↪ keys[i + 1]));
    }
    sort(all(keys), byDfn);
    keys.resize(unique(all(keys)) -
        ↪ keys.begin());
    for (auto u: keys) {
        _g[u].clear(); // 只清本次用到的点
    }
    for (int i = 0; i < SZ(keys) - 1; ++i) {
        _g[lca_ab(keys[i], keys[i +
            ↪ 1])].push_back(keys[i + 1]);
    }
}
};

// ---- 例题 CF613D 的虚树 DP ----
int dfs(const vector<vector<int>> &g, int u,
    ↪ int fa, ll &ans,
        const vector<int> &origin_keys) {
    int cnt = 0;
    bool isOri =
        ↪ binary_search(all(origin_keys), u);
    for (auto to : g[u]) {
        if (to == fa) continue;
        cnt += dfs(g, to, u, ans, origin_keys);
    }
    if (isOri) { ans += cnt; return 1; } //
    ↪ 关键点: 切断子树中所有未切断的
    if (cnt >= 2) { ++ans; return 0; } //
    ↪ 非关键点且是汇合处: 删自己
    return cnt;
}

void Solve() {
    cin >> N;
    Tree g(N);
    for (int i = 1; i <= N - 1; ++i) { ll u, v;
        ↪ cin >> u >> v; g.add_edge(u, v); }
    for (int i = 1; i <= N; ++i)
        ↪ sort(all(g.g[i])); // 为了
        ↪ binary_search 判相邻
    ll Q; cin >> Q;
    vector<vector<int>> _g(N + 2); //
    ↪ 虚树邻接表复用, 不重开
    while (Q--) {
        ll num; cin >> num;
        vector<int> origin_keys(num);
        for (auto &x : origin_keys) cin >> x;
        vector<int> keys = origin_keys; //
        ↪ 拷一份, keys 会被就地扩充
        sort(all(origin_keys));
        g.build_virtual_tree(_g, keys);
        ll ans = 0; bool havAns = true;
        for (auto u : origin_keys) { //
            ↪ 两关键点在原树直接相邻 -> 无解
            for (auto v : _g[u]) {
                if (binary_search(all(origin_ke
                    ↪ ys), v) &&
                    binary_search(all(g.g[u]),
                        ↪ v)) {

```

```

                havAns = false; cout << -1
                    ↪ << "\n"; break;
            }
        }
        if (!havAns) break;
    }
    // 根是 dfn 最小的点 keys.front(),
    ↪ 不一定是 1
    if (havAns) { dfs(_g, keys.front(), -1,
        ↪ ans, origin_keys); cout << ans <<
        ↪ "\n"; }
}
// AC https://codeforces.com/contest/613/submis
    ↪ sion/384497446

```

9.3 并查集 (DSU) 与 DSU on next 技巧

朴素并查集 + 路径压缩, 外加 `tagl(r, l, r)` 把 $[l, r]$ 这段区间整段标记为「已处理」——具体做法是让区间里的每个点都不指向自己 (指向 $r+1$)。find(x) 因此天然返回 x 之后第一个未被标记的位置, 扫描时常用来跳过整段已处理区间。

```

/** 并查集 (DSU) + DSU on next
 * find(x) 表示 x 之后第一个没有被标记的数字。
 * 构造函数传入的 n 必须不多不少, 否则 countFa
    ↪ 会出错。
 * base = 0 / 1 切换 0-based 与 1-based,
    ↪ 合法下标范围分别是 [0, n) 与 [1, n]。
 */
struct DSU {
    vector<int> fa;
    ll n;
    int base; // 0 或
    ↪ 1, 决定合法下标的起点

    DSU() {}
    DSU(ll n, int base = 0) : n(n), base(base)
        ↪ { init(n); }

    void init(ll n) {
        fa.resize(n + base + 5);
        for (int i = 0; i < SZ(fa); ++i) fa[i] =
            ↪ i;
    }

    ll find(ll x) {
        ll root = x;
        while (root != fa[root]) {
            root = fa[root];
        }
        while (x != root) {
            tie(x, fa[x]) = make_tuple(fa[x],
                ↪ root);
        }
        return x;
    }

    bool same(ll x, ll y) { return find(x) ==
        ↪ find(y); }

```

```

bool merge(ll x, ll y) { //
    ↪ 注意方向: fa[x] = y, 配合 dsu on next
    x = find(x); y = find(y);
    if (x == y) return false;
    fa[x] = y;
    return true;
}

// 把 [l, r] 这段区间整段标记,
    ↪ 标记后这段区间内任意点 find 都会越过去
void taglr(ll l, ll r) {
    l = max((ll)base, l);
    r = min((ll)SZ(fa) - 2, r);
    l = find(l);
    while(l <= r) {
        merge(l, l + 1);
        l = find(l + 1);
    }
}

ll countFa() { //
    ↪ 当前还有多少个独立的根
    ll res = 0;
    for(ll i = base; i < base + n; ++i) if
        ↪ (fa[i] == i) ++res;
    return res;
}
};

```

有时候这套思路别僵在数组上，用 map 当并查集容器也行，没出现过的 key 默认就是它自己当根。参考 CF Round 1077E 的提交：

```

map<ll,ll> fa;
auto find = [&](ll x) {
    ll root = x;
    while (fa.contains(root)) root = fa[root];
    ↪ // 没 contain 默认就是 root
    while (fa.contains(x)) {
        ↪ // 路径压缩
        ll p = fa[x];
        fa[x] = root;
        x = p;
    }
    return root;
};
auto merge = [&](ll x, ll y) {
    ll fax = find(x), fay = find(y);
    fa[fax] = fay;
};

```

维护到祖宗节点距离的并查集（带权并查集）：d[x] 存 x 到 p[x] 的距离，find 的同时累加距离。

```

int p[N], d[N];
// p[] 存每个点的祖宗节点, d[x] 存 x 到 p[x]
    ↪ 的距离

int find(int x) {
    if (p[x] != x) {
        int u = find(p[x]);
        d[x] += d[p[x]];
    }
}

```

```

    p[x] = u;
}
return p[x];
}

// 初始化 (节点 1..n)
for (int i = 1; i <= n; ++i) { p[i] = i; d[i] =
    ↪ 0; }

// 合并 a 与 b 所在的两个集合:
p[find(a)] = find(b);
d[find(a)] = distance; // 根据具体问题, 初始化
    ↪ find(a) 的偏移量

```

9.3.1 带权并查集（class 封装，维护到根距离与连通块大小）

把「路径压缩 + 维护 dis[x] (value[x] - value[fa[x]])」与按大小合并打包成完整类。find 递归回程时把 fa[x] 直接压到 root、dis[x] 累成 value[x] - value[root]; merge(x, y, w) 加入约束 value[y] - value[x] = w, 若和已有关系矛盾返回 false; query(x, y) 返回同一连通块下的 value[y] - value[x]。

```

// 带权 DSU 维护点到父节点的距离
class WeightedDSU
{
private:
    vector<int> fa, siz;
    vector<ll> dis; // dis[x] = value[x] -
        ↪ value[fa[x]], find 后变成 value[x] -
        ↪ value[root]

public:
    WeightedDSU(int n)
    {
        fa.resize(n + 1);
        siz.assign(n + 1, 1);
        dis.assign(n + 1, 0);
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int x)
    {
        if (fa[x] == x)
            return x;
        int oldFa = fa[x];
        int root = find(oldFa);
        dis[x] += dis[oldFa]; // x 到旧父亲 +
            ↪ 旧父亲到根 = x 到根
        fa[x] = root;
        return root;
    }

    bool same(int x, int y)
    {
        return find(x) == find(y);
    }

    // 加入约束: value[y] - value[x] = w
    // 返回 false 表示和已有关系矛盾
    bool merge(int x, int y, ll w)
    {

```



```

int rootX = find(x);
int rootY = find(y);

if (rootX == rootY)
    return dis[y] - dis[x] == w;

// 按大小合并
if (siz[rootX] > siz[rootY])
{
    swap(x, y);
    swap(rootX, rootY);
    w = -w;
}
// 让 rootX 挂到 rootY 下面
fa[rootX] = rootY;
siz[rootY] += siz[rootX];
// 推导:
// value[y] - value[x] = w
// value[x] = dis[x] + value[rootX]
// value[y] = dis[y] + value[rootY]
//
// 所以:
// dis[y] + value[rootY] - dis[x] -
//   value[rootX] = w
//
// 得:
// value[rootX] - value[rootY] = dis[y]
//   - dis[x] - w
dis[rootX] = dis[y] - dis[x] - w;

return true;
}
// 查询 value[x] - value[root]
ll distToRoot(int x)
{
    find(x);
    return dis[x];
}
// 查询 value[y] - value[x]
// 调用前最好保证 same(x, y)
ll query(int x, int y)
{
    find(x);
    find(y);
    return dis[y] - dis[x];
}
};

```

9.4 Borůvka 最小生成树

每轮让每个连通块各挑一条最小出边一次性合并，块数至少减半，故至多 $O(\log V)$ 轮、每轮 $O(E)$ 扫边，总 $O(E \log V)$ 。边以 (w, u, v) 数组给入，节点 0-based，函数自带精简 DSU（与上一节的 DSU 独立）。

```

struct DSU {
    vector<int> f;
    DSU(int n) : f(n) { iota(all(f), 0); }
    int find(int x) { return f[x] == x ? x : f[x] = find(f[x]); } // 真递归路径压缩
    bool unite(int a, int b) { a = find(a); b = find(b); if (a == b) return false; f[a] = b; return true; }
};

```

```

};

// 返回 MST 总权; edges: (w, u, v)
ll boruvka(int n, vector<array<ll, 3>> &edges) {
    DSU d(n);
    int comps = n;
    ll total = 0;
    while (comps > 1) {
        vector<int> cheap(n, -1);
        // cheap[块根] = 最优出边下标
        for (int i = 0; i < (int)edges.size(); ++i) {
            auto [w, u, v] = edges[i];
            int ru = d.find(u), rv = d.find(v);
            if (ru == rv) continue;
            // 同块, 不是出边
            if (cheap[ru] == -1 || w < edges[cheap[ru]][0]) cheap[ru] = i;
            if (cheap[rv] == -1 || w < edges[cheap[rv]][0]) cheap[rv] = i;
        }
        for (int r = 0; r < n; ++r) {
            if (cheap[r] == -1) continue;
            auto [w, u, v] = edges[cheap[r]];
            if (d.unite(u, v)) { total += w; --comps; } // 兜底跳过本轮已连通者
        }
    }
    return total;
}

```

9.5 树上背包

经典做法：每个点合并儿子时，外层倒序枚举背包总容量，内层正序枚举划分给当前儿子的容量。这样总复杂度可以做到 $O(n \cdot \text{budget})$ 。下面这份带“父亲买了 / 父亲没买”两种状态（来自 LeetCode-style 题 maxProfit）， $\text{dp}[1][u][i]$ = 父没买时 u 子树用预算 i 的最大收益， $\text{dp}[2][u][i]$ = 父买了时（当前点享半价）。

```

class Solution {
public:
    int maxProfit(int n, vector<int>& present,
        vector<int>& future,
        vector<vector<int>>& hierarchy, int budget) {
        int dp[3][170][170];
        for(int i = 1; i <= 2; ++i)
            for(int j = 0; j <= n + 5; ++j)
                for(int k = 0; k <= budget + 5; ++k)
                    dp[i][j][k] = 0;
        vector<vector<int>> g(n + 5);
        for(auto &t : hierarchy)
            g[t[0]].push_back(t[1]);

        // ism == 1: 父亲没买; ism == 2:
        //   父亲买了 (享半价)
    }
};

```

```

auto dfs = [&](this auto &&self, int u,
    ↪ int p) -> void {
    for(auto v : g[u]) { if(v == p)
        ↪ continue; self(v, u); }

    // memo: 暂时不考虑当前节点 u
    ↪ 的儿子合并结果
    vector<array<int,3>> memo(budget +
        ↪ 5, {0,0,0});
    for(auto &v : g[u]) {
        for(int i = budget; i >= 0;
            ↪ --i) {
            for(int j = 0; j <= i; ++j)
                ↪ { // 内层必须正序
                memo[i][1] =
                    ↪ max(memo[i][1],
                    ↪ memo[i-j][1] +
                    ↪ dp[1][v][j]);
                memo[i][2] =
                    ↪ max(memo[i][2],
                    ↪ memo[i-j][2] +
                    ↪ dp[2][v][j]);
                }
            }
        }
    }
    for(int i = budget; i >= 0; --i) {
        for(int j = 1; j <= 2; ++j) {
            int cost = present[u-1] / j;
            dp[j][u][i] =
                ↪ max(dp[j][u][i],
                ↪ memo[i][1]);
            if(i >= cost) {
                dp[j][u][i] =
                    ↪ max(dp[j][u][i],
                    ↪ future[u-1] - cost +
                    ↪ memo[i-cost][2]);
            }
        }
    }
};
dfs(1, 1);
int ans = 0;
for(int i = budget; i >= 0; --i) ans =
    ↪ max({ans, dp[1][1][i]});
return ans;
}
};

```

9.6 Floyd 全源最短路

三层循环，外层 i 是「中转点」，必须放最外面。复杂度 $O(n^3)$ ，写起来最简单。

```

const int N = 110;
int n, m, D[N][N];

inline void Floyd() {
    for(int i = 1; i <= n; ++i) //
        ↪ 中转点 i 必须在最外层
        for(int j = 1; j <= n; ++j) {
            if(j == i) continue;
            for(int k = 1; k <= n; ++k) {
                if(k == i) continue;

```

```

                D[j][k] = min(D[j][k], D[j][i]
                    ↪ + D[i][k]);
            }
        }
    }

int main() {
    ios::sync_with_stdio(false); cin.tie(0);
    ↪ cout.tie(0);
    cin >> n >> m;
    memset(D, 0x3f, sizeof(D));
    for(int i = 1; i <= n; ++i) D[i][i] = 0;
    for(int i = 1; i <= m; ++i) {
        int u, v, w; cin >> u >> v >> w;
        D[u][v] = min(D[u][v], w); //
        ↪ 处理重边、自环
        D[v][u] = min(D[v][u], w);
    }
    Floyd();
}
// AC https://www.luogu.com.cn/record/186274091

```

9.7 Dijkstra (堆优化)

核心：小根堆存 {点, 距离}，每次取距离最小的点。同一点可能以不同距离多份入堆，第一次出堆即为最短路、定为已固定点，靠 vis 懒删除跳过其余过期记录。松弛成功（能让邻居更短）才把邻居压回堆；堆开在函数里，多测天然清空。

```

struct Node {
    int node, dis;
    bool operator<(const Node &o) const {
        return dis > o.dis;
        ↪ // dis 大的优先级低 -> 小根堆
    }
};

void Solve() {
    int N, M, s;
    cin >> N >> M >> s;
    vector<vector<pair<int,int>>> g(N + 3);
    ↪ // g[u] = {邻点 v, 边权 w}
    for (int i = 1; i <= M; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        g[u].emplace_back(v, w);
    }
    vector<int> dist(N + 3, INF);
    vector<char> vis(N + 3);
    priority_queue<Node> pq;
    dist[s] = 0;
    pq.push({s, 0});
    while (!pq.empty()) {
        auto [u, dis] = pq.top();
        pq.pop();
        if (vis[u]) continue;
        ↪ // 过期记录, 懒删除
        vis[u] = true;
        ↪ // 第一次出堆即定点
        for (auto [to, w] : g[u]) {

```



```

        if (dis + w < dist[to]) {
            ↪ // 松弛成功才入堆
            dist[to] = dis + w;
            pq.push({to, dis + w});
        }
    }
}
// AC https://acm.hdu.edu.cn/contest/view-code?
↪ cid=1203&rid=10964

```

9.8 二分图判定 (染色法)

DFS 给每个点染 1 或 2, 相邻点必须异色; 用 $3 - c$ 在 1 和 2 之间翻转就够。注意图可能不连通, 外层要对每个未染色的点都启动一次。

```

const ll N = 1e5 + 10;
ll n, m;
short int color[N];
vector<ll> g[N];
bool isErfen = true;

void dfs(int x, int c) {
    if(!isErfen) return;
    if(color[x] == 0) color[x] = c;
    else {
        if(color[x] != c) { isErfen = false;
            ↪ return; }
        return;
    }
    for(int i = 0; i < (int)g[x].size(); ++i) {
        dfs(g[x][i], 3 - c); // 1
        ↪ 和 2 之间翻转的小 trick
    }
}

int main() {
    cin >> n >> m;
    for(int i = 1; i <= m; ++i) {
        int u, v; cin >> u >> v;
        g[u].push_back(v); g[v].push_back(u);
    }
    for(int i = 1; i <= n; ++i) { //
        ↪ 处理不连通
        if(color[i] == 0) dfs(i, 1);
        if(!isErfen) break;
    }
    cout << (isErfen ? "Yes" : "No") << endl;
}

```

9.9 匈牙利算法 (二分图最大匹配)

把 `match[]` 哨兵值设为 -1 (不是 0), 可以避免 0-based 编号造成歧义。每次给左侧某点 u 找匹配时清空 `vis`, DFS 沿增广路尝试腾位。

```

/* 二分图最大匹配, 匈牙利算法
 * 2026-03-31 AC https://acm.hdu.edu.cn/contest
 * ↪ /view-code?cid=1198&rid=18859
 */
struct Bi_graph_matcher {

```

```

    vector<vector<ll>> g;
    vector<char> vis;
    vector<ll> match;
    ll left_cnt;

    Bi_graph_matcher(vector<vector<ll>> _g, ll
    ↪ _left_cnt)
        : g(std::move(_g)), vis(SZ(g) + 2),
        ↪ match(SZ(g) + 2, -1),
        ↪ left_cnt(_left_cnt) {}

    bool find(ll u) {
        for (auto to : g[u]) {
            if (vis[to]) continue;
            vis[to] = true;
            // v 还没匹配 / v
            ↪ 原本匹配的人能腾出位置去找别人
            if (match[to] == -1 ||
            ↪ find(match[to])) {
                match[to] = u;
                return true;
            }
        }
        return false;
    }

    ll mx_match() {
        ll ans = 0;
        for (int i = 0; i <= left_cnt; ++i) {
            vis.assign(SZ(g) + 2, 0);
            if (find(i)) ++ans;
        }
        return ans;
    }
};

```

9.9.1 Hall 婚配定理 (完美匹配判定)

给定二分图 $G = (L \cup R, E)$ 。 L 存在完美匹配 (即所有 L 中的点都被匹配) 当且仅当: 对 L 的任意子集 $S \subseteq L$, 都有

$$|N(S)| \geq |S|$$

其中 $N(S)$ 是 S 在 R 中所有邻居构成的集合。

推论 (亏量定理 / Deficiency form): L 的最大匹配大小为

$$|L| - \max_{S \subseteq L} (|S| - |N(S)|)$$

右边的 $\max$ 项称为 Hall 亏量, 刻画 L 中“必然落单的点数下界”。

9.10 差分约束

差分约束就是把一组形如 $x \leq y + a$ 的约束转化成最短路问题: 把约束写成三角不等式 $\text{dist}(v) \leq \text{dist}(u) + w(u, v)$, 每条约束就是一条有向边 $u \rightarrow v$ 边权 a , 跑最短路得到的 `dist[]` 就是一组合法解 (超级源点的 `dist` 值不重要, 差值才重要)。

例题 ABC 395F Smooth Occlusion: 相邻牙齿高度差 $\leq X$, 转成 $\text{dist}_v \leq \text{dist}_u + X$ 跑 Dijkstra。

```

ll N, X;
arr ud[MAXN];

inline void solve(){
    cin >> N >> X;
    ll sumUd = 0;
    vector<ll> dist(N + 5, INF);
    vector<vector<ll>> g(N + 5);
    vector<bool> vis(N + 5, false);
    priority_queue<arr, vector<arr>, cmp> q;
    FOR(i, 1, N) {
        cin >> ud[i][0] >> ud[i][1];
        sumUd += (ud[i][0] + ud[i][1]);
        if(i != N) g[i+1].pb(i), g[i].pb(i+1);
        dist[i] = ud[i][0];
        q.push({i, dist[i]});
    }
    // 在差分约束图上跑 Dijkstra
    while(!q.empty()) {
        ll u = q.top()[0], dis = q.top()[1];
        q.pop();
        if(vis[u]) continue;
        vis[u] = true; dist[u] = dis;
        FOR(i, 0, SZ(g[u]) - 1) {
            ll v = g[u][i];
            if(!vis[v]) q.push({v, dis + X});
        }
    }
    ll H = INF;
    FOR(i, 1, N) H = min(H, dist[i] + ud[i][1]);
    cout << sumUd - N * H << "\n";
}
// AC https://atcoder.jp/contests/abc395/submissions/64194370

```

9.11 Tarjan 缩点为边双连通分量 (EBCC)

把求桥和缩点合在一起：先一遍 Tarjan 标出所有桥，再绕开桥跑一遍连通块染色，每个连通块就是一个 EBCC。输入是带边编号 (eid) 的无向图，重边用不同 eid 区分。

```

// Tarjan_EBCC: 在原图上 (带边编号) 求割边 (桥) 并缩点为边双连通分量 (edge-biconnected components)
// 下标约定: EBCC 数组下标采用 0-base, 但是点所在的 EBCC_ID(comp[u]) 采用 1-base
class Tarjan_EBCC {
    int n;
    vector<int> dfn, low;
    int timer;
    vector<int> bridges;
    vector<int> comp;
    int comp_cnt;

public:
    Tarjan_EBCC(int n = 0) : n(n), g(n+1),
        dfn(n+1, 0), low(n+1, 0), timer(0),
        comp(n+1, 0), comp_cnt(0) {}

    void add_edge(int u, int v, int id) {
        g[u].push_back({v, id});
    }

```

```

        g[v].push_back({u, id});
    }

    void run() {
        for(int i=1; i<=n; i++){
            if(!dfn[i]) dfs_bridge(i, -1);
        }
        unordered_set<int> bridgeSet;
        bridgeSet.reserve(bridges.size()*2+10);
        for(int eid: bridges)
            bridgeSet.insert(eid);
        for(int i=1; i<=n; i++){
            if(!comp[i]){
                ++comp_cnt;
                dfs_comp(i, comp_cnt, bridgeSet);
            }
        }
    }

    vector<int> get_bridges() const { return bridges; }
    vector<int> get_components() const { return comp; }
    int get_comp_cnt() const { return comp_cnt; }

private:
    void dfs_bridge(int u, int peid) {
        dfn[u] = low[u] = ++timer;
        for(auto [v, eid] : g[u]){
            if(!dfn[v]){
                dfs_bridge(v, eid);
                low[u] = min(low[u], low[v]);
                if(low[v] > dfn[u])
                    bridges.push_back(eid);
            } else if(eid != peid){
                low[u] = min(low[u], dfn[v]);
            }
        }
    }

    void dfs_comp(int u, int cid, const unordered_set<int> &bridgeSet) {
        comp[u] = cid;
        for(auto [v, eid] : g[u]){
            if(bridgeSet.count(eid)) continue;
            if(!comp[v]) dfs_comp(v, cid, bridgeSet);
        }
    }
};

```

9.12 Tarjan 求强连通分量 (SCC)

单次 DFS 维护栈：入栈时打 tin / low，遇到栈中点用 low[v] 而非 tin[v] 更新（书上常见两种写法都对）。退栈条件 tin[u] == low[u] 时把栈顶到 u 全部弹出，组成一个 SCC。

```

vector<ll> tin(N + 5), low(N + 5);
vector<ll> stk;

```

```

vector<char> instk(N + 5);
vector<ll> scc(N + 5);
vector<ll> scca(N + 5); //
↳ 每个 scc 的点权之和 (按需)
ll idx = 0, sccidx = 0;

auto dfs1 = [&](auto self, ll u) -> void {
    tin[u] = low[u] = ++idx;
    stk.PB(u);
    instk[u] = 1;
    for (auto v : g[u]) {
        if (!tin[v]) {
            self(self, v);
            low[u] = min(low[u], low[v]);
        } else if (instk[v]) {
            low[u] = min(low[u], low[v]);
        }
    }
    if (tin[u] == low[u]) { // u
        ↳ 是当前 scc 的根
        ++sccidx;
        ll y = 0;
        do {
            y = stk.back(); stk.pop_back();
            instk[y] = 0;
            scc[y] = sccidx;
            scca[sccidx] += A[y];
        } while (y != u);
    }
};

FOR(i, 1, N) if(!tin[i]) dfs1(dfs1, i);

```

9.13 点分治

找重心 → 统计经过重心的路径 → 删掉重心递归。把当前重心记为 c ，删掉 c 之后整棵树被分成若干棵子树，每棵子树的根是 c 的某个儿子 v 。任意一条跨子树路径（比如从 v_1 子树某点 x 走到 v_2 子树某点 y ）都一定经过 c 。

标准套路：

1. 对每个儿子 v ，跑一次 DFS：从 c 出发把“以 c 为起点到这棵子树任意点的路径信息”（长度 / 最值 / 计数等）收进 bucket_v 。
2. 一棵子树一棵子树处理：当前子树 = bucket_v ，之前已处理过的所有子树用一个全局桶维护（map / vector / Fenwick 都可）。
3. 处理某棵子树时，先用它的每条路径信息和“之前所有子树”配对，统计跨子树路径；配对完再把这棵子树的信息塞进全局桶，留给后面的子树用。
4. 重心 c 自己若也能作为路径端点，最后单独处理一下（如 $\text{ans} += \text{cnt}[A[\text{root}]]$ ）。

为什么不重不漏：每条路径的「最高层重心」（点分治那棵重心树最上面的那个）唯一，所以每条路径只在它的最高层重心处算一次；不经过 c 的路径，等递归下去到那棵子树自己的重心时再算。

模板（来自 [Nowcoder 提交](#)）：把“DFS 收集”和“在重心处合并”两个钩子分别叫 `collector` / `merger`，外面的题目逻辑通过 `lambda` 传进去。

```

const ll INF = (ll)1e17 + 9;

// 题意相关：每条“从当前重心某儿子出发”的路径上，
// 我们关心的只是“成为严格新最小值”的点的权值 A[x]
struct Info { ll val; };

// 点分治结构体：负责树、找重心、递归分治。题意通过
// ↳ collector / merger 传入。
struct CentroidDecomp {
    int n;
    vector<vector<int>> g;
    vector<int> sz;
    vector<char> vis;
    int centroid, best;

    CentroidDecomp(int n_) { init(n_); }
    void init(int n_) {
        n = n_;
        g.assign(n + 1, {}); sz.assign(n + 1, 0);
        vis.assign(n + 1, 0);
    }
    void add_edge(int u, int v) {
        g[u].push_back(v); g[v].push_back(u);
    }

    template <class InfoT, class Collector,
        ↳ class Merger>
    void run(int rt, Collector collector,
        ↳ Merger merger) {
        int tot = get_size(rt, 0);
        best = tot; centroid = rt;
        get_centroid(rt, 0, tot);
        decompose<InfoT>(centroid, collector,
            ↳ merger);
    }

private:
    int get_size(int u, int p) {
        sz[u] = 1;
        for (int v : g[u]) if (v != p &&
            ↳ !vis[v]) sz[u] += get_size(v, u);
        return sz[u];
    }
    void get_centroid(int u, int p, int tot) {
        int mx = tot - sz[u];
        for (int v : g[u]) {
            if (v == p || vis[v]) continue;
            get_centroid(v, u, tot);
            mx = max(mx, sz[v]);
        }
        if (mx < best) { best = mx; centroid =
            ↳ u; }
    }

    template <class InfoT, class Collector,
        ↳ class Merger>
    void decompose(int c, Collector &collector,
        ↳ Merger &merger) {
        vis[c] = 1;
    }

```

```

// 1) 对当前重心 c 的每棵未删子树 dfs
→ 收集信息
vector<vector<InfoT>> buckets;
buckets.reserve(g[c].size());
for (int v : g[c]) {
    if (vis[v]) continue;
    buckets.push_back(collector(v, c,
        → c));
}
// 2) 在重心 c
→ 这里把这些子树的信息合并进答案
merger(c, buckets);
// 3) 对每棵子树递归
for (int v : g[c]) {
    if (vis[v]) continue;
    int tot = get_size(v, c);
    best = tot; centroid = v;
    get_centroid(v, c, tot);
    decompose<InfoT>(centroid,
        → collector, merger);
}
}
};

// 用法: 求树上「严格递减最小值序列」的有序对 (u,
→ v) 数 (举例)
// auto collector = [&](int u, int p, int root)
→ -> vector<Info> {
//     vector<Info> vec;
//     function<void(int,int,ll)> dfs = [&](int
→ x, int fa, ll mn) {
//         if (mn > A[x]) vec.push_back({A[x]});
→ // 严格新最小值才收
//         mn = min(mn, A[x]);
//         for (int y : cd.g[x]) if (y != fa &&
→ !cd.vis[y]) dfs(y, x, mn);
//     };
//     dfs(u, p, INF);
→ // 关键: 初始 mn = INF, 不是 A[root]
//     return vec;
// };
// auto merger = [&](int root,
→ vector<vector<Info>> &buckets) {
//     map<ll,ll> cnt;
//     for (auto &vec : buckets) {
//         for (auto &info : vec) {
//             ll a = info.val;
//             if (a >= A[root]) continue;
//             auto it = cnt.find(a);
//             if (it != cnt.end()) ans +=
→ it->second;
//         }
//         for (auto &info : vec)
→ cnt[info.val]++;
//     }
//     ans += cnt[A[root]];
// };
// cd.run<Info>(1, collector, merger);

```

9.14 重链剖分 (HLD + 线段树)

参考实现: [CodeChef Beautiful Sum \(XPS\)](#) 的提交。把树拆成若干条重链, 每条链在线段树上变成一段连续区

间; 路径修改/查询 $O(\log^2 n)$, 子树修改/查询 $O(\log n)$ (一段连续区间 $[dfn[u], dfn[u] + siz[u] - 1]$)。下面这套含奇偶深度分开的异或 Tag、按位拆分的 Info、ZKW 线段树、HLD 的路径 / 子树双接口, 以及对边权题”边挂儿子 + 子树边接口排除自身”的处理。

```

static constexpr int LOG = 21;

// --- 核心数据结构 ---
struct Tag {
    ll xor_val[2]; // [0]: 偶深, [1]: 奇深
    explicit Tag(ll e = 0, ll o = 0) {
        → xor_val[0] = e; xor_val[1] = o; }
    void apply(const Tag &t) { xor_val[0] ^=
        → t.xor_val[0]; xor_val[1] ^=
        → t.xor_val[1]; }
    bool empty() const { return xor_val[0] == 0
        → && xor_val[1] == 0; }
};

struct Info {
    int cnt[2]; // 偶深 / 奇深
    → 节点数
    int bit_num[2][LOG]; //
    → bit_num[p][i]: p 奇偶性中第 i 位为 1
    → 的个数

    Info() { memset(cnt, 0, sizeof(cnt));
        → memset(bit_num, 0, sizeof(bit_num)); }

    static Info make_structure(int depth) {
        Info res;
        res.cnt[depth & 1] = 1;
        return res;
    }

    static Info merge(const Info &l, const Info
    → &r) {
        Info res;
        FOR(p, 0, 1) {
            res.cnt[p] = l.cnt[p] + r.cnt[p];
            FOR(i, 0, LOG-1) res.bit_num[p][i]
            → = l.bit_num[p][i] +
            → r.bit_num[p][i];
        }
        return res;
    }

    void apply(const Tag &t) {
        FOR(p, 0, 1) {
            if (t.xor_val[p] == 0) continue;
            FOR(i, 0, LOG-1) {
                if ((t.xor_val[p] >> i) & 1)
                    bit_num[p][i] = cnt[p] -
                    → bit_num[p][i];
            }
        }
    }

    static Info zero() { return Info(); }
};

// --- ZKW 线段树 (懒标记) ---
struct SEG {
    int n, sz, dep;
    vector<Info> info;

```

```

vector<Tag> tag;

SEG(int n_) {
    n = n_ + 2;
    dep = __lg(n) + 1;
    sz = 1 << dep;
    info.assign(sz * 2 + 5, Info());
    tag.assign(sz * 2 + 5, Tag());
}

static void pull(Info &fa, const Info &lc,
    ↪ const Info &rc) { fa = Info::merge(lc,
    ↪ rc); }

void apply_one(int o, const Tag &t) {
    ↪ info[o].apply(t); tag[o].apply(t); }
void pushdown(int o) {
    if (tag[o].empty()) return;
    int l = o << 1, r = 1 | 1;
    apply_one(l, tag[o]); apply_one(r,
    ↪ tag[o]);
    tag[o] = Tag();
}

void assign_raw(int i, const Info &v) {
    ↪ info[i + sz] = v; }
void build() {
    for (int o = sz - 1; o >= 1; --o)
        pull(info[o], info[o << 1], info[o
    ↪ << 1 | 1]);
}

void push_path(int l, int r) {
    l = l - 1 + sz; r = r + 1 + sz;
    int i = dep, j = __lg(l ^ r);
    while (i > j) pushdown(l >> i--);
    while (i) { pushdown(l >> i);
    ↪ pushdown(r >> i); --i; }
}

Info query(int L, int R) {
    if (L > R) return Info::zero();
    push_path(L, R);
    int l = L - 1 + sz, r = R + 1 + sz;
    Info left = Info::zero(), right =
    ↪ Info::zero();
    for (; l ^ r ^ 1; l >>= 1, r >>= 1) {
        if (~l & 1) left =
        ↪ Info::merge(left, info[l ^ 1]);
        if (r & 1) right =
        ↪ Info::merge(info[r ^ 1], right);
    }
    return Info::merge(left, right);
}

void range_apply(int L, int R, const Tag
    ↪ &t) {
    if (L > R) return;
    push_path(L, R);
    int l = L - 1 + sz, r = R + 1 + sz;
    int L0 = l, R0 = r;
    for (; l ^ r ^ 1; l >>= 1, r >>= 1) {
        if (~l & 1) apply_one(l ^ 1, t);
        if (r & 1) apply_one(r ^ 1, t);
    }
    for (L0 >>= 1; L0; L0 >>= 1)
        ↪ pull(info[L0], info[L0 << 1],
        ↪ info[L0 << 1 | 1]);
}

```

```

    for (R0 >>= 1; R0; R0 >>= 1)
        ↪ pull(info[R0], info[R0 << 1],
        ↪ info[R0 << 1 | 1]);
    }
};

// --- HLD ---
struct HLD {
    int n, cur;
    vector<vector<int>> g;
    vector<int> fa, dep, heavy, head, dfn, siz;

    HLD(int n_) : n(n_), cur(0), g(n_ + 1),
        ↪ fa(n_ + 1), dep(n_ + 1),
        ↪ heavy(n_ + 1), head(n_ + 1),
        ↪ dfn(n_ + 1), siz(n_ + 1)
        ↪ {}

    void addEdge(int u, int v) {
        ↪ g[u].push_back(v); g[v].push_back(u); }
    int dfs1(int u, int f) {
        fa[u] = f; dep[u] = dep[f] + 1; siz[u]
        ↪ = 1;
        int maxsz = 0;
        for (int v : g[u]) if (v != f) {
            int sub = dfs1(v, u);
            siz[u] += sub;
            if (sub > maxsz) { maxsz = sub;
            ↪ heavy[u] = v; }
        }
        return siz[u];
    }

    void dfs2(int u, int h) {
        head[u] = h; dfn[u] = ++cur;
        if (heavy[u]) dfs2(heavy[u], h);
        for (int v : g[u]) if (v != fa[u] && v
        ↪ != heavy[u]) dfs2(v, v);
    }

    void build(int root = 1) { cur = 0;
    ↪ dfs1(root, 0); dfs2(root, root); }

    int lca(int u, int v) {
        while (head[u] != head[v]) {
            if (dep[head[u]] > dep[head[v]]) u
            ↪ = fa[head[u]];
            else v = fa[head[v]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    template<class F> void for_subtree(int u, F
    ↪ &&op) { op(dfn[u], dfn[u] + siz[u] -
    ↪ 1); }
    template<class F> void for_path(int u, int
    ↪ v, F &&op) {
        while (head[u] != head[v]) {
            if (dep[head[u]] < dep[head[v]])
            ↪ swap(u, v);
            op(dfn[head[u]], dfn[u]);
            u = fa[head[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        op(dfn[u], dfn[v]);
    }
};

```



```
// --- HLD + ZKW 组合：路径/子树修改、
↳ 路径/子树查询、边接口 ---
struct HLDZKW {
    HLD hld;
    SEG seg;
    HLDZKW(int n) : hld(n), seg(n) {}
    void addEdge(int u, int v) { hld.addEdge(u,
↳ v); }
    void build() {
        hld.build(1);
        FOR(i, 1, hld.n)
↳ seg.assign_raw(hld.dfn[i],
↳ Info::make_structure(hld.dep[i]));
        seg.build();
    }
    void applySubtreeNode(int u, const Tag &t) {
        hld.for_subtree(u, [&](int L, int R) {
↳ seg.range_apply(L, R, t); });
    }
    void applyPathNode(int u, int v, const Tag
↳ &t) {
        hld.for_path(u, v, [&](int L, int R) {
↳ seg.range_apply(L, R, t); });
    }
    Info querySubtreeNode(int u) {
        Info res = Info::zero();
        hld.for_subtree(u, [&](int L, int R) {
↳ res = Info::merge(res, seg.query(L,
↳ R)); });
        return res;
    }
    Info queryPathNode(int u, int v) {
        Info res = Info::zero();
        hld.for_path(u, v, [&](int L, int R) {
↳ res = Info::merge(res, seg.query(L,
↳ R)); });
        return res;
    }
    // 边接口：边 (u, fa[u]) 挂在儿子 u 上；
    ↳ 子树边集 = 子树点集 \ {u}
    void applySubtreeEdge(int u, const Tag &t) {
        applySubtreeNode(u, t);
        int pos = hld.dfn[u];
        seg.range_apply(pos, pos, t); //
↳ 撤销 u 自己（其代表的边不在子树边集中）
    }
};
```

9.15 欧拉回路与欧拉路径

欧拉回路存在的充要条件（无向连通图）：每个顶点的度数都是偶数。

欧拉路径存在的充要条件（无向连通图）：奇数度顶点数等于 0 或 2。0 时是回路，2 时起点和终点恰好是这两个奇度点。

构造时 DFS 在弹栈时（递归回来时）才把当前边塞进 ans_ls，最后整体 reverse。为什么要反着塞：因为最后要 reverse(ans_ls)，所以当前边必须等递归完后续点之后再 push_back，反过来才会得到正确顺序——和正常 DFS 顺序反着写。

```
if (odd_cnt != 0 && odd_cnt != 2) {
    cout << "NO\n";
    return;
}

vector<ll> ans_ls;
auto dfs = [&](auto &&self, ll u) -> void {
    while (!g[u].empty()) {
        auto it = g[u].begin();
        auto [to, id] = *it;
        g[u].erase(it);
        g[to].erase({u, id});
        self(self, to);
        ans_ls.push_back(id); //
↳ 弹栈时才塞，最后 reverse
    }
};
dfs(dfs, max(start_node, g.begin()->first));

// 判断图是否连通：边数对不上就是不连通
if (SZ(ans_ls) != N) {
    cout << "NO\n";
    return;
}
reverse(all(ans_ls));
cout << "YES\n";
for (auto u : ans_ls) cout << u << " ";
cout << "\n";
```

9.16 Color Coding(随机染色 + 状压 DP)

Color Coding 适合处理这类题：图很大，但只需要找一个点数不超过 K 的简单路径或小子图，并且 K 本身很小。它的核心想法是把“判重节点”改写成“判重颜色”。

每一轮先给所有点随机染上 0 到 $K - 1$ 中的一种颜色。之后只允许路径走到新颜色的点上。于是状态里的 sta 不再表示“访问过哪些点”，而是表示“已经用过哪些颜色”。

这样做的好处是非常直接的：如果一条路径上颜色两两不同，那么这些点一定也两两不同，所以它天然就是简单路径。原本要维护一个 n 位的访问集合，现在只需要维护一个 K 位的颜色集合，复杂度就从“看起来完全做不了”变成了“ K 小时可以接受”。

像 1009 走马观花 这题里，点权都在 $[0, K - 1]$ ，还要求路径点权和 $\bmod K = 0$ 。这时还有一个很关键的前置结论：最优解至多只会经过 K 个点。因为如果一条合法路径超过了 K 个点，那么看它的前缀和模 K ，按鸽巢原理一定会有两个前缀余数相同，于是中间那一段的点权和就 $\equiv 0 \pmod K$ 。把这段删掉以后，合法性不变，但边权和更小，因此最优解不可能更长。

实现中状态是 $dp[u][sta]$ ，其中存的是一个二进制组：

- $dp[u][sta][0]$ 表示走到 u 、颜色集合为 sta 时，当前最小的边权和。
- $dp[u][sta][1]$ 表示这条最优方案对应的花韵总值模 K 的结果。

timeId[u][sta] 是一个用来时间戳优化的这个数组。

9.16.1 转移骨架

那么实际上，我们在转移的时候是只对这个最短路进行优化，我们不会对这个花韵值有任何的这个选择，是最短路径，就直接接收其花韵总值。

```
for (int i = 1; i <= N; ++i) {
    color[i] = uniform_int_distribution<int>(0, K
    ↪ - 1)(rng);
}
for (int i = 1; i <= N; ++i) {
    dp[i][1 << color[i]] = {0, A[i]};
    timeId[i][1 << color[i]] = curT;
    if (A[i] == 0) {
        ans = min(ans, 0ll);
        return;
    }
}
for (int use = 1; use <= K; ++use) {
    for (int sta = 1; sta < (1 << K); ++sta) {
        if (__builtin_popcount(sta) != use)
            ↪ continue;
        for (int u = 1; u <= N; ++u) {
            if (!check(u, sta)) continue;
            for (auto [to, w]: g[u]) {
                if (sta & (1 << color[to])) continue;
                ll tow = dp[u][sta][0] + w;
                if (tow >= ans) {
                    break;
                }
                ll toSta = sta | (1 << color[to]);
                if (!check(to, toSta)) {
                    dp[to][toSta] = {INF, -1};
                    timeId[to][toSta] = curT;
                }
                if (tow < dp[to][toSta][0]) {
                    dp[to][toSta][0] = tow;
                    dp[to][toSta][1] = (A[to] +
                    ↪ dp[u][sta][1]) % K;
                    if (dp[to][toSta][1] == 0) {
                        ans = min(dp[to][toSta][0], ans);
                    }
                }
            }
        }
    }
}
```

上面这段代码里，真正的灵魂只有两句：

- if (sta & (1 << color[to])) continue;，表示这个颜色已经用过了，这一轮就不允许再走到这个点。
- dp[to][toSta][1] = (A[to] + dp[u][sta][1]) % K;，表示直接把“花韵和模 K”塞在状态二元组的第二项里维护，而不是额外拆出一维。

9.16.2 剪枝技巧

这份代码里还有两个很值得加进模板的剪枝和优化细节。

- 邻接表先按边权升序排序。这样在枚举 u 的出边时，一旦发现 $d + w \geq ans$ ，后面的边只会更大，可以直接 break。（实测这个效果比较好）
- 不要每一轮都整块清空 DP。可以采用时间戳优化。

9.16.3 复杂度

如果一轮染色的状态是 $dp[u][sta]$ ，复杂度大致是 $O(2^K(N + M))$ 。像这题再加一维 sum 以后，可以记成 $O(K \cdot 2^K(N + M))$ 。单轮只会在目标路径刚好被染成“彩虹路径”时成功，所以要独立重复若干轮。最坏情况下，长度恰好为 K 的目标路径在一轮里全异色的概率是 $\frac{K!}{K^K}$ ，因此 $K = 6$ 时通常要跑几百轮才稳。

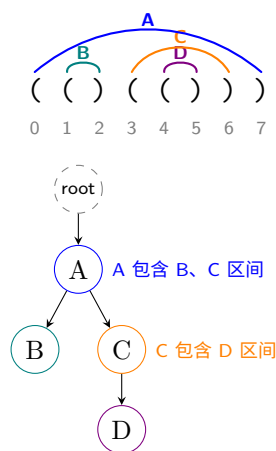
9.16.4 一句话总结

Color Coding 的本质，就是用“K 个颜色的 bitmask”去近似代替“真实访问点集合”。只要题目的目标结构规模小，且简单路径这个限制卡得很死，它往往就是那种平时用不上，一旦出现就很致命的技巧。

9.17 合法括号序列 ↔ 有根有序树

结论：长度为 $2n$ 的合法括号序列与 $n + 1$ 个结点的有根有序树一一对应；计数公式见前文“Catalan 数（合法括号串数量）”小节。扫描建树的过程就是一次 DFS 的进栈/出栈轨迹——遇到（在当前结点下新建一个“最右儿子”并下去，遇到）回到父结点，扫完恰好回到根。有一个“虚根”承担最外层括号，方便统一处理。

具体例子：序列 $((()(()))$ ($n = 4$)。上方是序列与配对弧（同色弧属同一对），下方是对应的 5 结点有根有序树（root 是虚根）。每个实心结点对应一对配对括号：



位置 ↔ 结点：A 对应位置 0-7，B 对应 1-2，C 对应 3-6，D 对应 4-5。注意 A 的两个儿子 B，C 就是位置上相邻的两对最外层配对——“兄弟 ↔ 并排配对”在这张图里一眼能看出。

对照表 (括号序列位置从 0 编号):

为什么有用: 把” 括号序列区间 $[l, r]$ ” 翻译成” 子树” 之后, 问题立刻具备” 子树内独立、跨子树只通过根交互” 的分治/树形 DP 结构。像 CF 2210D 这类” 对合法括号序列做变换后再计数/极值” 的题, 先把序列想成树几乎是本能动作。

配对位置预处理 ($O(n)$, 栈一遍扫):

```
// match[i]: 位置 i 的括号与哪个位置配对
vector<int> build_match(const string &s) {
    int n = s.size();
    vector<int> match(n);
    stack<int> st;
    for (int i = 0; i < n; ++i) {
        if (s[i] == '(') st.push(i);
        else {
            int j = st.top(); st.pop();
            match[i] = j;
            match[j] = i;
        }
    }
    return match;
}
```

顺手建真树: 如果题目要的不止 match[], 而要父指针、儿子列表, 只需在扫的时候每遇 (就 ++cnt 开一个新结点并挂到栈顶结点儿子末尾, 同样 $O(n)$:

```
// 返回 {par, children}; 结点从 0 开始, 0 号为虚根
// 约定输入 s 合法 (可外面先 assert)
struct ParenTree {
    vector<int> par;
    vector<vector<int>> ch;
};

ParenTree build_tree(const string &s) {
    ParenTree t;
    t.par.push_back(-1);
    t.ch.emplace_back();
    stack<int> st;
    st.push(0);
    for (char c : s) {
        if (c == '(') {
            int u = (int)t.par.size();
            t.par.push_back(st.top());
            t.ch.emplace_back();
            t.ch[st.top()].push_back(u);
            st.push(u);
        } else {
            st.pop();
        }
    }
    return t;
}
```

常见踩坑: 很多题不需要显式建树, 只要” 配对数组 match[] + 深度数组” 就足够做区间 $\rightarrow$ 子树的翻译; 显式建树往往只在需要跨子树合并信息 (例如儿子之间按下标有序 DP) 时才值得做。

9.18 网络流 (Dinic): 最大流 / 最小割容量及割边集合

Dinic 求最大流 $O(V^2E)$, 二分图匹配 $O(E\sqrt{V})$ 。核心思路: BFS 在残量网络上分层, DFS 只沿层数 +1 的边增广; cur[u] 当前弧优化保证每条边在一轮里只被无效访问一次。

典型用法:

- 最大流 / 最小割容量: maxFlow(s, t)。由最大流最小割定理, 返回值同时是 s - t 最小割容量。
- 最小割顶点集: 跑完 maxFlow 后调 minCutSide(s), 返回 vis[]: vis[u]=1 表示 u 在 S 侧。原理是残量网络里从 s 沿剩余容量 > 0 的边能到的就是 S 侧。
- 最小割边集: minCutEdges(s) 返回所有”from 在 S 侧、to 在 T 侧” 的原图边的输入编号。前提是 addEdge 时传过 id。
- 某条边实际流量: getFlow(id) 返回 originCap - cap。

建图约定: 点编号 1-based, Dinic dinic(n) 后 0 号点空着不用。无向容量边用 addUndirectedEdge(u, v, c), 等价于两个方向各加一条容量 c 的有向边。多测用 dinic.init(n) 重置。

```
class Dinic {
private:
    struct Edge {
        int from, to;
        ll cap; // 当前剩余容量
        ll originCap; // 原始容量, 用于求割边代价 / 查询流量
        int id; // 输入边编号, 不需要输出割边时可以不管
    };

    int n;
    vector<Edge> edges;
    vector<vector<int>> g;
    vector<int> level, cur;
    vector<int> edgeIdToIndex;

    bool bfs(int s, int t) {
        fill(all(level), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int edgeIndex : g[u]) {
                Edge &e = edges[edgeIndex];
                if (e.cap > 0 && level[e.to] == -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;
    }
};
```



```

}

ll dfs(int u, int t, ll pushed) {
    if (u == t) return pushed;
    if (pushed == 0) return 0;
    for (int &i = cur[u]; i <
        → (int)g[u].size(); i++) {
        int edgeIndex = g[u][i];
        Edge &e = edges[edgeIndex];
        if (e.cap == 0) continue;
        if (level[e.to] != level[u] + 1)
            → continue;
        ll tr = dfs(e.to, t, min(pushed,
            → e.cap));
        if (tr == 0) continue;
        e.cap -= tr;
        edges[edgeIndex ^ 1].cap += tr;
        return tr;
    }
    return 0;
}

public:
    // 建立一个 1-base 流图, 可用点编号 1..n, 0
    → 号点空着不用。
    Dinic(int n = 0) { init(n); }

    // 多测时用 dinic.init(n) 清空旧图。
    void init(int n_) {
        n = n_;
        edges.clear();
        g.assign(n + 1, {});
        level.assign(n + 1, -1);
        cur.assign(n + 1, 0);
        edgeIdToIndex.clear();
    }

    // 加有向边 from → to, 容量 cap。
    // 只求最大流不需要传 id;
    → 要输出最小割边就把输入编号 i 传进来。
    void addEdge(int from, int to, ll cap, int
        → id = -1) {
        if (id >= 0) {
            if ((int)edgeIdToIndex.size() <=
                → id) {
                edgeIdToIndex.resize(id + 1,
                    → -1);
            }
            edgeIdToIndex[id] =
                → (int)edges.size();
        }
        edges.push_back({from, to, cap, cap,
            → id});
        g[from].push_back((int)edges.size() -
            → 1);
        edges.push_back({to, from, 0, 0, -1});
        g[to].push_back((int)edges.size() - 1);
    }

    // 加无向容量边: u → v 和 v → u 都加容量
    → cap。
    // 普通无向图最小割通常这样建。
    void addUndirectedEdge(int u, int v, ll
        → cap, int id = -1) {

```

```

        addEdge(u, v, cap, id);
        addEdge(v, u, cap, id);
    }

    // s 到 t 的最大流; 由最大流最小割定理也是 s-t
    → 最小割容量。
    ll maxFlow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) {
            fill(all(cur), 0);
            while (true) {
                ll pushed = dfs(s, t, INF);
                if (pushed == 0) break;
                flow += pushed;
            }
        }
        return flow;
    }

    // 跑完 maxFlow(s, t) 后调用。
    // vis[u] = 1 表示 u 在最小割 S 侧; vis[u] =
    → 0 表示在 T 侧。
    // 原理: 残量网络里从 s 沿剩余容量 > 0
    → 的边能到的点。
    vector<int> minCutSide(int s) {
        vector<int> vis(n + 1, 0);
        queue<int> q;
        vis[s] = 1;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int edgeIndex : g[u]) {
                Edge &e = edges[edgeIndex];
                if (e.cap > 0 && !vis[e.to]) {
                    vis[e.to] = 1;
                    q.push(e.to);
                }
            }
        }
        return vis;
    }

    // 跑完 maxFlow(s, t) 后调用,
    → 返回一组最小割边的输入编号。
    // 一条原图边 u → v 属于最小割 <=> u 在 S
    → 侧且 v 在 T 侧。
    // 使用前提: addEdge 时传过 id。
    vector<int> minCutEdges(int s) {
        vector<int> vis = minCutSide(s);
        vector<int> cutEdges;
        for (int edgeIndex = 0; edgeIndex <
            → (int)edges.size(); edgeIndex += 2) {
            Edge &e = edges[edgeIndex];
            if (e.id != -1 && vis[e.from] &&
                → !vis[e.to]) {
                cutEdges.push_back(e.id);
            }
        }
        return cutEdges;
    }

    // 跑完 maxFlow(s, t) 后调用,
    → 返回当前这组最小割边的原始容量之和。

```

```

// 正常情况下 dinic.minCutCost(s) ==
//  $\rightarrow$  dinic.maxFlow(s, t)。
ll minCutCost(int s) {
    vector<int> vis = minCutSide(s);
    ll cost = 0;
    for (int edgeIndex = 0; edgeIndex <
         $\rightarrow$  (int)edges.size(); edgeIndex += 2) {
        Edge &e = edges[edgeIndex];
        if (vis[e.from] && !vis[e.to]) {
            cost += e.originCap;
        }
    }
    return cost;
}

// 查询某条输入边实际流了多少; 前提是 addEdge
//  $\rightarrow$  时传过 id。
ll getFlow(int id) {
    int edgeIndex = edgeIdToIndex[id];
    return edges[edgeIndex].originCap -
         $\rightarrow$  edges[edgeIndex].cap;
}
};

```

9.19 最小费用最大流 (SPFA 势能 + Dijkstra 增广)

在最大流的基础上, 每条边带单位流量费用, 要求在所有最大流方案中费用最小的那个。核心思路: 每次沿费用最短的增广路推流。直接 SPFA 找最短路也对, 但残量网络可能出现负边权 (反向边费用为 $-cost$), 所以引入势能 (Johnson 思想) 做边权修正, 让 Dijkstra 可用: 把边 $u \rightarrow v$ 的权由 $cost$ 改成 $cost + h_u - h_v$, 正确势能下这个修正后的权一定非负, 每次 Dijkstra 跑完用 $h_v += dist_v$ 同步即可。

复杂度: $O(SPFA + A \cdot m \log n)$, A 为增广次数 (每条增广路至少一条边饱和)。

典型用法:

- **原始边费用非负:** 直接 `minCostMaxFlow(s, t)`, h 初始化为 0, 第一次 Dijkstra 等价于在原图上跑费用最短路, 势能自然生效。
- **原始边费用可能为负:** 传 `useSPFA=true`, 先用 SPFA 跑出从 s 到每点的初始势能再走 Dijkstra。注意是原始边出现负费用; 反向边一开始 `cap=0` 不参与最短路。

建图约定: 点编号 1-based, 反向边随 `addEdge` 自动加 (容量 0、费用 $-cost$)。返回 {最大流, 最小费用} 的二元组。

```

class MinCostMaxFlow {
private:
    struct Edge {
        int from, to; //
         $\rightarrow$  边的起点和终点
        ll cap; //
         $\rightarrow$  当前残余容量
        ll cost; //
         $\rightarrow$  单位流量费用
    };
};

```

```

};

int n;
vector<Edge> edges;
vector<vector<int>> g;

// SPFA 初始化势能  $h$ 
// 作用: 当原始费用可能为负时, 先算出从  $s$ 
//  $\rightarrow$  到每个点的最短路势能
void initPotentialBySPFA(int s, vector<ll>
     $\rightarrow$  &h) {
    vector<int> inq(n + 1, 0);
    queue<int> q;
    fill(all(h), INF);
    h[s] = 0;
    q.push(s);
    inq[s] = 1;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        inq[u] = 0;
        for (int id : g[u]) {
            Edge &e = edges[id];
            if (e.cap <= 0) continue;
            int v = e.to;
            if (h[v] > h[u] + e.cost) {
                h[v] = h[u] + e.cost;
                if (!inq[v]) {
                    inq[v] = 1;
                    q.push(v);
                }
            }
        }
    }
}

// 不可达点的势能设为 0, 避免后续计算 INF
//  $\rightarrow$  参与运算
for (int i = 1; i <= n; i++) {
    if (h[i] == INF) h[i] = 0;
}

public:
    MinCostMaxFlow(int n) : n(n), g(n + 1) {}

    // 加一条  $u \rightarrow v$  的边, 容量为  $cap$ , 费用为  $cost$ 
    // 同时加反向边  $v \rightarrow u$ , 容量为 0, 费用为
    //  $\rightarrow -cost$ , 用于撤销流量
    void addEdge(int u, int v, ll cap, ll cost)
    {
        int id = edges.size();
        edges.push_back({u, v, cap, cost});
        edges.push_back({v, u, 0, -cost});
        g[u].push_back(id);
        g[v].push_back(id ^ 1);
    }

    // 返回 {最大流, 最小费用}
    // useSPFA 表示是否使用 SPFA 初始化势能
    pair<ll, ll> minCostMaxFlow(int s, int t,
         $\rightarrow$  bool useSPFA = false) {
        ll maxFlow = 0;
        ll minCost = 0;
        vector<ll> h(n + 1, 0); //
         $\rightarrow$  势能, 让 Dijkstra
         $\rightarrow$  可处理残量网络中的负边
    }
};

```

```

vector<ll> dist(n + 1); //
↳ 最短路距离
vector<int> pre(n + 1); //
↳ pre[v] = 最短路到达 v 的边编号

if (useSPFA) {
    initPotentialBySPFA(s, h);
}

while (true) {
    fill(all(dist), INF);
    fill(all(pre), -1);
    priority_queue<pair<ll, int>,
        ↳ vector<pair<ll, int>>,
        ↳ greater<pair<ll, int>>> pq;
    dist[s] = 0;
    pq.push({0, s});

    // 在残量网络上跑 Dijkstra,
    ↳ 找费用最短的增广路
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        ↳ pq.pop();
        if (d != dist[u]) continue;
        for (int id : g[u]) {
            Edge &e = edges[id];
            if (e.cap <= 0) continue;
            int v = e.to;
            // 势能修正后的边权,
            ↳ 正确势能下一定 >= 0
            ll newCost = e.cost + h[u]
                ↳ - h[v];
            if (dist[v] > dist[u] +
                ↳ newCost) {
                dist[v] = dist[u] +
                    ↳ newCost;
                pre[v] = id;
                pq.push({dist[v], v});
            }
        }
    }

    // t 不可达, 说明已经没有增广路
    if (pre[t] == -1) break;

    // 更新势能
    for (int i = 1; i <= n; i++) {
        if (dist[i] < INF) {
            h[i] += dist[i];
        }
    }

    // 找这条增广路上的瓶颈容量
    ll addFlow = INF;
    for (int cur = t; cur != s; cur =
        ↳ edges[pre[cur]].from) {
        addFlow = min(addFlow,
            ↳ edges[pre[cur]].cap);
    }

    // 沿着最短费用路增广
    for (int cur = t; cur != s; cur =
        ↳ edges[pre[cur]].from) {
        int id = pre[cur];

```

```

        edges[id].cap -= addFlow;
        edges[id ^ 1].cap += addFlow;
        minCost += addFlow *
            ↳ edges[id].cost;
    }

    maxFlow += addFlow;
}

return {maxFlow, minCost};
};

```

9.20 图论常见结论

9.20.1 树直径端点的”远点”性质

对于任意点 a , 树直径的两个端点中至少有一个是距离 a 最远的点。这就是为什么”两遍 BFS 求树直径”成立: 第一遍从任意点 a 出发找最远点 u , u 一定是直径端点; 第二遍从 u 出发找最远点 v , (u, v) 就是一条直径。可以用反证法证明。例题 [ABC 428](#)。

9.20.2 相邻点单源最短路差不超过 1

在无权图里, 对任意一条边 (u, v) , 恒有 $|dis_u - dis_v| \leq 1$ (dis_u 是某固定根到 u 的最短路长度, BFS 层数)。看起来像废话, 但偶尔确实有用, 如 [CF 2179](#)。

进一步地, 若图是二分图, 可以加强为 $|dis_u - dis_v| = 1$ 。原因: 二分图 $\iff$ 所有环长度为偶数 $\iff$ 从固定根出发的最短路距离的奇偶性给出一个合法二分划分。所以对任意边 (u, v) , dis_u 与 dis_v 的奇偶性必相反, 因此不可能相等。

9.20.3 广义凯莱公式 (森林连通块版)

给定一个 n 个顶点 m 条边的森林, 设有 k 个连通块, 大小分别为 $s_1, s_2, \dots, s_k$ 。把它们加 $k-1$ 条边合成一棵树的方案数为:

$$n^{k-2} \prod_{i=1}^k s_i.$$

直观理解: 长度为 $k-2$ 的 prufer 序列贡献 n^{k-2} ; 乘以 $\prod_{i=1}^k s_i$ 是因为对所有非根连通块要确定它的”发送端” (每块只有一个发送端, 否则会成环), 还要确定根连通块的”接收端”——这个在 prufer 序列中没有直接体现, 严格讲是最后一个被删除的连通块所连到的根连通块的接收点。

9.20.4 两条树上路径是否相交 (LCA 判定)

记两条路径的端点为 (a, b) 与 (c, d) , $p = \text{LCA}(a, b)$, $q = \text{LCA}(c, d)$, 则

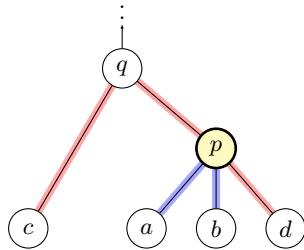
两路径相交 $\iff p \in \text{路径}(c, d)$ 或 $q \in \text{路径}(a, b)$ 。

证明: 路径 $a-b$ 是” a 上升到 p 再下降到 b ”的 Λ 形, p 是其上深度最小的点, 且路径上每个点都是 p 的后代。设 x 是两条路径的公共点, 那么 p 与 q 都是 x 的祖先 (含自身), 故 p, q 必有祖先后代关系。不妨设

$\text{dep}(p) \geq \text{dep}(q)$, 则 q 是 p 的祖先, 而 p 又是 x 的祖先, 所以 p 落在 x 上升到 q 的那一段上——这一整段都属于路径 $c-d$, 故 p 在 $c-d$ 上。反向显然。□

推论: 只需检验 p, q 中深度较大的那个是否落在另一条路径上; 实现时懒得比深度就两个方向各判一遍。

判定“ x 是否在路径 $u-v$ 上”: $\text{LCA}(u, x) = x$ 且 $\text{LCA}(x, v) = \text{LCA}(u, v)$ (或把 u, v 对调); 等价地 $\text{dis}(u, x) + \text{dis}(x, v) = \text{dis}(u, v)$ 。模板与例题见本章开头「倍增法求 LCA」的 P3398 仓鼠找 sugar。



图中 $\text{dep}(p) > \text{dep}(q)$, 两条路径的交集恰是 $\{p\}$: 蓝路径 $a-b$ 以 p 为顶, 红路径 $c-d$ 从 q 下降时正好经过 p 。若把 p 从红路径上挪开 (例如 d 改挂到 c 那一侧), 两条路径就不相交了。

10 字符串算法

10.1 Manacher 算法

Manacher 算法在 $O(n)$ 时间内求出字符串每个位置为中心的最长回文半径。核心思想是利用已经求出的回文区间 $[L, R]$, 把当前位置 cur 关于中心对称到 $R + L - cur$, 借助对称位置已经计算好的半径作为初值, 避免从 1 开始重新匹配。

下面分别给出奇回文和偶回文两个版本。

```
string s;
cin >> s;
const int n = s.size();
vector<int> manacher(n);
// 奇回文半径, manacher[cur] 表示以 cur
// 为中心的最大回文半径
for (int cur = 0, L = 0, R = 0, k = 0; cur < n;
    cur++)
{
    if (cur <= R)
        k = min(manacher[R + L - cur], R - cur + 1);
    else
        k = 1;
    while (cur + k < n && cur - k >= 0 && s[cur +
        k] == s[cur - k])
        k++;
    if (cur + k - 1 > R) // 一定保证右端点最右
        R = cur + k - 1, L = cur - k + 1;
    manacher[cur] = k;
}
fill(all(manacher), 0);
// 偶回文半径, manacher[cur] 表示以 {cur - 1,
// cur} 为中心的最大回文半径
for (int cur = 0, L = 0, R = 0, k = 0; cur < n;
    cur++)
{
    if (cur <= R)
        k = min(manacher[R + L - cur + 1], R - cur
        + 1);
    else
        k = 0; // 注意这里要写成
        // 0, 因为第一个位置初始不合法
    while (cur + k < n && cur - 1 - k >= 0 &&
        s[cur + k] == s[cur - 1 - k])
        k++;
    if (cur + k - 1 > R) // 一定保证右端点最右
        R = cur + k - 1, L = cur - 1 - k + 1;
    manacher[cur] = k;
}
```

10.2 Z 函数 (扩展 KMP)

Z 函数 $z[i]$ 定义为字符串 s 与其后缀 $s[i..n-1]$ 的最长公共前缀长度, 其中约定 $z[0] = n$ 。算法在 $O(n)$ 时间内维护一个最右匹配区间 $[l, r]$ (即所有已计算的 i 中使 $i + z[i] - 1$ 最大的那个 i 给出的区间): 当 cur 落在 $[l, r]$ 内时, 可以用对称位置 $cur - l$ 处已经算出的 z 值作为初值, 否则从 0 暴力匹配; 任何一次扩展都不会让 r 后退, 故总匹配次数线性。

// Z 函数: 求字符串每一个后缀和原字符串的 LCP

```
auto Z = [&](string s) -> vector<int>
{
    int n = s.size();
    vector<int> Z(n);

    // 默认第一个位置是全匹配
    Z[0] = n;
    for (int cur = 1, l = 0, r = 0, k = 0; cur <
        n; cur++)
    {
        if (cur <= r)
            k = min(Z[cur - l], r - cur + 1); //
            // 复用维护的匹配区间信息
        else
            k = 0;
        // 暴力扩展
        while (cur + k < n && s[cur + k] == s[k])
            k++;

        // 当且仅当新匹配区间右端点更远的时候更新匹配
        // 区间
        if (cur + k - 1 > r)
            l = cur, r = cur + k - 1;
        // 记录 Z 函数的值
        Z[cur] = k;
    }
    return Z;
};
```

10.3 KMP (前缀函数 pi)

KMP 的核心是 前缀函数: 对串 A , $\pi[i]$ 表示 $A[0..i]$ 这一段中最长 真前缀 = 真后缀 的长度 (约定 $\pi[0] = 0$)。求出 π 后做模式匹配的常用技巧是把模式串 s_2 、文本串 s_1 拼成 $s_2 + \# + s_1$ 一起跑前缀函数, 凡是 $\pi[i] = |s_2|$ 的位置就对应一次匹配。

```
/*
 * 采用 0-based 设计
 * 模式串 (s2) + '#' + 文本串 (s1)
 * 需要这样子传入, 如果想要执行 kmp 的话
 * kmp 模板题 AC
 * https://www.luogu.com.cn/record/269142391
 */
vector<ll> kmp(const string &A) {
    // 返回总前缀数组 pi
    // 其定义是 [0, i] 真前后缀相同的最长长度
    // 特别的, pi[0] = 0
    vector<ll> pi(SZ(A));
    for (int i = 1; i < SZ(A); ++i) {
        ll len = pi[i - 1];
        while (len > 0 && A[len] != A[i]) {
            len = pi[len - 1];
        }
        if (A[len] == A[i]) {
            pi[i] = len + 1;
        }
    }
    return pi;
}
```

典型调用方式:

```
void Solve() {
    string s1, s2;
    cin >> s1 >> s2;
    vector<ll> pi = kmp(s2 + "#" + s1);
    // 输出所有匹配的起始位置
    for (int i = SZ(s2); i < SZ(pi); ++i) {
        if (pi[i] == SZ(s2)) {
            ll ans = i - SZ(s2) - SZ(s2);
            ++ans;
            cout << ans << "\n";
        }
    }
    // 输出 s2 第 i 位的最长 border 长度
    for (int i = 0; i < SZ(s2); ++i) {
        cout << pi[i] << " ";
    }
    cout << "\n";
}
```

10.4 字符串哈希 (mod $2^{61} - 1$ 版本)

一个用 Mersenne 素数 $2^{61} - 1$ 作模的字符串哈希, 整数运算用 `__uint128_t` 中转、做一次「低 61 位 + 高位」折叠后只需最多一次减法即可归约到 $[0, M)$ 。配合 HS 类提供的前缀哈希、动态 push / pop、以及一个简单的 find 子串计数接口, 覆盖比赛里绝大多数字符串哈希需求。

```
struct U {
    using u128 = __uint128_t;
    static constexpr ll MOD = (1ULL << 61) - 1;

    ll v; // always in [0, MOD)

    U() : v(0) {}
    U(ll x) {
        ll t = x % MOD;
        if (t < 0) t += MOD;
        v = (ll) t;
    }

    static inline ll add(ll a, ll b) {
        ll s = a + b;
        if (s >= MOD) s -= MOD;
        return s;
    }
    static inline ll sub(ll a, ll b) {
        return (a >= b) ? (a - b) : (a + MOD - b);
    }
    static inline ll mul(ll a, ll b) {
        u128 t = (u128) a * b; // <
        // 2^122
        ll s = ll(t & MOD) + ll(t >> 61); // 1-step
        // fold
        if (s >= MOD) s -= MOD; // at
        // most one subtract needed
        return s;
    }

    U &operator+=(const U &o) { v = add(v, o.v);
    return *this; }
```

```
U &operator-=(const U &o) { v = sub(v, o.v);
    return *this; }
U &operator*=(const U &o) { v = mul(v, o.v);
    return *this; }

friend U operator+(U a, const U &b) { a += b;
    return a; }
friend U operator-(U a, const U &b) { a -= b;
    return a; }
friend U operator*(U a, const U &b) { a *= b;
    return a; }

friend bool operator==(const U &a, const U
    &b) { return a.v == b.v; }
friend bool operator!=(const U &a, const U
    &b) { return a.v != b.v; }
friend bool operator<(const U &a, const U
    &b) { return a.v < b.v; }
friend bool operator>(const U &a, const U
    &b) { return a.v > b.v; }
friend bool operator<=(const U &a, const U
    &b) { return a.v <= b.v; }
friend bool operator>=(const U &a, const U
    &b) { return a.v >= b.v; }
};
```

```
// base 运行时随机, 防 anti-hash (固定 base
    // 容易被离线预计算碰撞)
const U base = []{
    mt19937_64 rng(chrono::steady_clock::now().ti
        // me_since_epoch().count());
    uniform_int_distribution<ll> dist(257, U::MOD
        // - 1);
    return U(dist(rng));
}();
```

```
// AC https://vjudge.net/solution/65151535
    // string matching 测试 find
// AC https://www.luogu.com.cn/record/245201000
    // 测试 push / pop / 增强 get
// 字符串哈希约定: 越前面是越高位 (和数字一样)
struct HS {
    vector<U> hs, powBase;
    U hashval;
    ll n;
    string s;
    // 内部 0-based, hs[i] 表示 s[0..i] 的哈希
    HS(const string &ls) {
        n = SZ(ls);
        s = ls;
        hs.assign(n, 0);
        powBase.assign(n + 1, 0);
        powBase[0] = 1;
        for (ll i = 1; i <= n; ++i) {
            powBase[i] = powBase[i - 1] * base;
        }
        if (n > 0) {
            hs[0] = U(s[0]);
            for (ll i = 1; i < n; ++i) {
                hs[i] = hs[i - 1] * base + U(s[i]);
            }
            hashval = hs[n - 1];
        } else {
            hashval = 0;
        }
    }
};
```



```

    hashval = 0;
}
}

// get(l, r): 返回 s[l..r] 的哈希 (0-based,
// ↪ inclusive)
// 自动把越界端点 clamp 到合法范围
U get(ll l, ll r) {
    if (n == 0) return 0;
    l = max(0ll, l);
    r = min(n - 1, r);
    if (l > r) return 0;
    if (l == 0) return hs[r];
    return hs[r] - hs[l - 1] * powBase[r - l +
    ↪ 1];
}

U get(ll l) { return get(l, n - 1); }

// 返回 ls 在 s 中的出现次数
ll find(const string &ls) {
    U hxfind = 0;
    ll m = SZ(ls);
    if (m == 0) return n + 1;
    if (m > n) return 0;
    for (ll i = 0; i < m; ++i) {
        hxfind = hxfind * base + ls[i];
    }
    ll res = 0;
    for (ll r = m - 1; r < n; ++r) {
        U hx = get(r - m + 1, r);
        if (hx == hxfind) res++;
    }
    return res;
}

void pop() {
    if (n == 0) return;
    n--;
    s.pop_back();
    if (n == 0) {
        hs.clear();
        hashval = 0;
        return;
    }
    hs.pop_back();
    hashval = hs.back();
}

void push(char c) {
    s.push_back(c);
    if (n == 0) {
        hs.push_back(U(c));
    } else {
        hs.push_back(hs.back() * base + U(c));
    }
    n++;
    hashval = hs.back();
    if (SZ(powBase) <= n) {
        powBase.push_back(powBase.back() * base);
    }
}
};

```

10.5 子序列自动机

子序列自动机预处理 $\text{next}[i][c]$: 从位置 i 起字符 c 下一次出现的下标 (含 i)。建出来之后单串子序列匹配可以做到 $O(|t|)$ 一次回答 t 是不是 s 的子序列。下面这份适配小写字母 / 大写字母 / 数字, 换字符集时只需调整 base 与第二维大小。

```

struct seqAuto {
    // 子序列自动机 subsequence automaton
    // AC
    ↪ https://www.luogu.com.cn/record/218545124
    vector<vector<int>> next;
    // 从 i 开始, 字符下一次出现的位置 (含 i)
    char base = 'a';
    seqAuto(const string &s) {
        next.assign(SZ(s) + 5, vector<int>(26, -1));
        ROF(i, SZ(s) - 1, 0) {
            FOR(j, 0, 26) {
                next[i][j] = next[i + 1][j];
            }
            next[i][s[i] - base] = i;
        }
    }
    inline bool isFind(const string &s) {
        ll idx = 0;
        FOR(i, 0, SZ(s) - 1) {
            if (next[idx][s[i] - base] != -1) {
                idx = next[idx][s[i] - base] + 1;
            } else {
                return false;
            }
        }
        return true;
    }
};

```

10.6 Trie 树

普通 26 叉 Trie, 节点上挂 cnt 表示有多少串经过该节点; $\text{query}(s)$ 返回已有集合中所有串与 s 的最长公共前缀长度之和 (每个串按其与 s 的 LCP 长度计入一次)。remove 用懒计数撤回, 不释放节点。

```

/*
 * AC https://codeforces.com/gym/105941/submission/367570263
 * 统计已有集合中所有字符串与 s 的相同前缀长度之和
 */
class Trie {
    vector<array<int, 26>> tree;
    vector<int> cnt;
    int idx, n;

public:
    Trie(int N = 2e6 + 1) : n(N), tree(N),
    ↪ cnt(N), idx(0) {}

    void insert(const string s) {
        int cur = 0;
        for (const char ch : s) {

```

```

    int pos = ch - 'a';
    if (tree[cur][pos] == 0) {
        tree[cur][pos] = ++idx;
    }
    cur = tree[cur][pos];
    cnt[cur]++;
}

void remove(const string s) {
    int cur = 0;
    for (const char ch : s) {
        int pos = ch - 'a';
        if (tree[cur][pos] == 0) {
            assert(0);
        }
        cur = tree[cur][pos];
        cnt[cur]--;
    }
}

ll query(const string &s) {
    int cur = 0;
    ll res = 0;
    for (const char ch : s) {
        int pos = ch - 'a';
        if (tree[cur][pos] == 0) break;
        cur = tree[cur][pos];
        res += cnt[cur];
    }
    return res;
}
};

```

10.7 01 Trie (XOR Trie)

处理一族 unsigned long long 上的最大 / 最小异或问题。max_xor(x) / min_xor(x) 返回与 x 异或的极值；argmax_xor / argmin_xor 返回取得这个极值的原数本身。erase 走「沿路径减 cnt，遇到 cnt 归零的子树则懒摘除指针」的写法，避免正经回收带来的额外管理。

```

// AC https://judge.yosupo.jp/submission/308823
↪ min_xor / erase / insert
// AC https://codeforces.com/contest/706/submission/334623853
↪ argmax / max
// AC https://www.codechef.com/viewsolution/1183578991
↪ 综合
struct XORTrie {
    int B; // 最高位 (含)
    struct Node {
        int nxt[2];
        int cnt; // 经过该节点的数的个数 (叶子 = 出现次数)
        Node() { nxt[0] = nxt[1] = -1; cnt = 0; }
    };
    vector<Node> tr;
    explicit XORTrie(int max_bit = 63) :
        B(max_bit) {
        tr.reserve(1 << 20);
        tr.emplace_back(); // root
    }
}

```

```

void clear() { tr.clear(); tr.emplace_back();
↪ }

inline void insert(unsigned long long x) {
    int u = 0;
    for (int b = B; b >= 0; --b) {
        int bit = int((x >> b) & 1ull);
        if (tr[u].nxt[bit] == -1) {
            tr[u].nxt[bit] = (int) tr.size();
            tr.emplace_back();
        }
        u = tr[u].nxt[bit];
        ++tr[u].cnt;
    }
}

inline int count(unsigned long long x) const {
    int u = 0;
    for (int b = B; b >= 0; --b) {
        int bit = int((x >> b) & 1ull);
        int v = tr[u].nxt[bit];
        if (v == -1) return 0;
        u = v;
    }
    return tr[u].cnt;
}

inline bool erase(unsigned long long x) {
    if (tr.size() == 1) return false;
    static int pathBit[70];
    int u = 0, depth = 0;
    for (int b = B; b >= 0; --b) {
        int bit = int((x >> b) & 1ull);
        int v = tr[u].nxt[bit];
        if (v == -1 || tr[v].cnt == 0) return
        ↪ false;
        pathBit[depth++] = bit;
        u = v;
    }
    if (tr[u].cnt == 0) return false;
    u = 0;
    for (int i = 0; i < depth; ++i) {
        int bit = pathBit[i];
        int v = tr[u].nxt[bit];
        --tr[v].cnt;
        if (tr[v].cnt == 0) tr[u].nxt[bit] = -1;
        ↪ // 懒回收
        u = v;
    }
    return true;
}

inline unsigned long long max_xor(unsigned
↪ long long x) const {
    if (tr.size() == 1) return 0ull;
    int u = 0;
    unsigned long long ans = 0;
    for (int b = B; b >= 0; --b) {
        int bit = int((x >> b) & 1ull);
        int prefer = tr[u].nxt[bit ^ 1];
        int same = tr[u].nxt[bit];
        if (prefer != -1 && tr[prefer].cnt > 0) {
            u = prefer;
            ans |= (1ull << b);
        } else {
            u = same; // 至少有一条有效路
        }
    }
}

```



```

    }
    }
    return ans;
}
inline unsigned long long min_xor(unsigned
↪ long long x) const {
    if (tr.size() == 1) return 0ull;
    int u = 0;
    unsigned long long ans = 0;
    for (int b = B; b >= 0; --b) {
        int bit = int((x >> b) & 1ull);
        int same = tr[u].nxt[bit];
        int other = tr[u].nxt[bit ^ 1];
        if (same != -1 && tr[same].cnt > 0) {
            u = same;
        } else {
            u = other;
            ans |= (1ull << b);
        }
    }
    return ans;
}
inline unsigned long long argmax_xor(unsigned
↪ long long y) const {
    if (tr.size() == 1) return 0ull;
    int u = 0;
    unsigned long long chosen = 0;
    for (int b = B; b >= 0; --b) {
        int bit = int((y >> b) & 1ull);
        int prefer = tr[u].nxt[bit ^ 1];
        int same = tr[u].nxt[bit];
        if (prefer != -1 && tr[prefer].cnt > 0) {
            u = prefer;
            chosen |= (unsigned long long) (bit ^
↪ 1) << b;
        } else {
            u = same;
            chosen |= (unsigned long long) bit << b;
        }
    }
    return chosen;
}
inline unsigned long long argmin_xor(unsigned
↪ long long x) const {
    if (tr.size() == 1) return 0ull;
    int u = 0;
    unsigned long long chosen = 0;
    for (int b = B; b >= 0; --b) {
        int bit = int((x >> b) & 1ull);
        int same = tr[u].nxt[bit];
        int other = tr[u].nxt[bit ^ 1];
        if (same != -1 && tr[same].cnt > 0) {
            u = same; // 让该位 XOR=0
            if (bit) chosen |= (1ull << b);
        } else {
            u = other; // 只能走相反位, 该位 XOR=1
            if (bit ^ 1) chosen |= (1ull << b);
        }
    }
    return chosen;
}
} tr;
// 处理无符号数

```

10.8 字符串分词与输出技术 (大模拟读入)

getline + stringstream 是处理一行内含混合分隔符的标准搭配: 先 getline 整行, 再丢进 stringstream 按空白 / 自定义字符切分。下面两段一段是以 ':' / '+' 等多种字符做模板化分词的「跨天时间均值」例题, 另一段是只用 '+' 切的最简加法器。

```

// 跨天时间区间均值: 行内固定用 ':' / 空格分隔,
↪ 末尾可选 (+day)
inline void solve() {
    init();
    FOR(i, 1, 2) {
        string line; getline(cin, line);
        stringstream sst(line); char fen;
        sst >> hh[1] >> fen >> mm[1] >> fen >> ss[1]
            >> hh[2] >> fen >> mm[2] >> fen >>
            ↪ ss[2];
        string dayst; ll day = 0;
        if (sst >> dayst) {
            stringstream st(dayst);
            st >> fen >> fen >> day;
        }
        sec[i] = day * 86400 + 3600 * (hh[2] -
            ↪ hh[1])
            + 60 * (mm[2] - mm[1]) + (ss[2] -
            ↪ ss[1]);
    }
    ll fsec = (sec[1] + sec[2]) / 2;
    ll h = fsec / 3600; fsec %= 3600;
    ll m = fsec / 60; fsec %= 60;
    ll s = fsec;
    cout << setw(2) << setfill('0') << h << ":"
        << setw(2) << setfill('0') << m << ":"
        << setw(2) << setfill('0') << s << "\n";
}

```

```

// 用 '+' 作分词符的简单加法器
inline void __() {
    stringstream ss(line);
    string num;
    ll ans = 0;
    while (getline(ss, num, '+')) {
        ans += stoll(num);
    }
    cout << ans << "\n";
}

```

11 动态规划 (dp)

11.1 二进制子集枚举技巧

如果要枚举 n 个元素的所有子集，最直接的写法是把每个集合看成一个二进制数，范围就是 $[0, 2^n - 1]$ 。

```
for (int s = 0; s < (1 << n); s++) {  
    // s 的二进制表示对应一个子集  
}
```

11.1.1 枚举某个集合的所有子集

如果已经有一个集合 $mask$ ，想要只枚举它的所有子集，常用下面这个写法。

```
for (int sub = mask; sub; sub = (sub - 1) &  
    mask) {  
    // sub 是 mask 的一个非空子集  
}
```

它的原理是， $sub - 1$ 会把 sub 最低位的一个 1 变成 0，并把它右边的若干位全部变成 1。随后再与 $mask$ 做一次按位与，就会把不属于 $mask$ 的位置全部清掉，于是得到一个更小但仍然合法的子集。

因此，这个过程会按照从大到小的顺序，不重不漏地枚举出 $mask$ 的所有非空子集。

如果还需要处理空集，可以写成下面这样。注意 $sub == 0$ 时必须及时跳出，否则会死循环。

```
for (int sub = mask;; sub = (sub - 1) & mask) {  
    // sub 是 mask 的一个子集，包含空集  
    if (sub == 0) {  
        break;  
    }  
}
```

这个技巧在状压 DP 中非常常见。若外层再枚举所有 $mask$ ，内层枚举它的全部子集，总复杂度是 $O(3^n)$ 。

11.1.2 枚举固定大小的子集

如果只关心恰好选了 use 个元素的状态，就没必要把所有状态都扫一遍再用 `popcount` 过滤。可以直接枚举二进制中恰好有 use 个 1 的状态。

```
for (int use = 0; use <= K; ++use) {  
    long long sta = (1LL << use) - 1;  
    while (sta < (1LL << K)) {  
        // sta 的二进制中恰好有 use 个 1  
  
        if (sta == 0) break; // use=0  
        // 时只对应空集这一个状态，避免下面除零  
        long long c = sta & -sta;  
        long long r = sta + c;  
        sta = (((r ^ sta) >> 2) / c) | r;  
    }  
}
```

这里先令 $sta = (1LL << use) - 1$ ，也就是二进制下最小的、恰好含有 use 个 1 的状态。后面的转移则负责直接跳到“下一个同样有 use 个 1 的状态”。

$c = sta \& -sta$ 会取出最低位的 1， $r = sta + c$ 可以理解为把最靠右、还能向左移动的一位 1 往前推一格。这样一来，右侧结构会被打乱，再用最后一行把剩余的若干个 1 重新挪到最右边，于是就得到了下一个更大的合法状态。

这个写法常被称为 Gosper's hack。它会按照从小到大的顺序，直接跳到下一个二进制中有相同个数 1 的状态。若 $use = 0$ ，则只对应空集这一个状态。

11.2 SOS DP (Sum / Max over Subsets, 高维前缀和)

给定一个长度为 2^N 的数组 $f[\cdot]$ (下标看成 N 位二进制掩码)，希望对每个 $mask$ 求出

$$F[mask] = \bigoplus_{sub \subseteq mask} f[sub]$$

其中 $\bigoplus$ 可以是 $+$ / $\max$ / $\min$ / 异或等任何么半群运算。直接枚举子集是 $O(3^N)$ ；SOS DP 通过逐位扩展把它压到 $O(N \cdot 2^N)$ 。

核心思想是把每一位看成一个维度，依次做「沿第 i 维的一维前缀」：当处理到第 i 位时，若 $mask$ 第 i 位是 1，就把 $mask$ 与 $mask \oplus (1 \ll i)$ (去掉第 i 位) 合并。等价于沿 N 个超立方体维度各做一遍一维前缀和。

对偶版：超集而非子集，即 $F[mask] = \bigoplus_{mask \subseteq sup} f[sup]$ 。把转移方向反过来，把「有第 i 位的从无第 i 位的吸收」改成「无第 i 位的从有第 i 位的吸收」即可。

11.2.1 典型应用：按位无交最大值

问题：给定 n 个非负整数 $a_1, \dots, a_n$ ，对每个 i 找出最大的 a_j 使得 $a_i \& a_j = 0$ (按位无交集，二进制位上互不相交)。

做法：

- 用 $dp[mask] = \max\{a_j : a_j \subseteq mask\}$ (位掩码意义下的子集)，初值 $dp[a_j] = a_j$ 。
- 跑一遍子集 $\max$ 的 SOS DP，得到「掩码 $mask$ 下所有子集中最大的 a_j 」。
- 对询问 i ，答案 $= dp[\bar{a}_i]$ ，因为 a_j 与 a_i 按位无交 $\iff a_j \subseteq \bar{a}_i$ 。

复杂度 $O((V + n) \cdot \log V)$ ，其中 $V = \max a_i$ 。

```
void solve() {  
    int n;  
    cin >> n;  
    vector<int> a(n);  
    for (int &x : a) cin >> x;  
    const int V = ranges::max(a);  
    const int N = 32 - __builtin_clz(V); //  
    // 用到的位数
```

```

vector<int> dp(1 << N);
for (int x : a) dp[x] = x;          //
↳ 标记原始集合

// SOS DP: dp[mask] = max{a_j : a_j 的位是 mask
↳ 的子集}
for (int i = 0; i < N; i++) {
    for (int mask = (1 << N) - 1; mask >= 0;
        ↳ mask--) {
        if ((mask >> i) & 1) {
            dp[mask] = max(dp[mask],
                            dp[mask ^ (1 << i)]);
        }
    }
}

// 查询: 与 a[i] 按位无交 等价于 是 ~a[i] 的子集
for (int i = 0; i < n; i++) {
    int q = (~a[i]) & ((1 << N) - 1);
    cout << (dp[q] == 0 ? -1 : dp[q]) << "
    ↳ \n" [i == n - 1];
}
}

```

要点:

- 第二重循环方向无关 (顺序、逆序都对), 因为转移只涉及「同一 i 位下, $mask$ 与去掉该位的 $mask \oplus (1 \ll i)$ 」, 去掉位的 $mask$ 在本轮不会被本轮的更新二次覆盖 (它的第 i 位是 0, 进不了 if 分支)。
- 用「0 表示未出现」的哨兵时, 要确保 $a_j \neq 0$; 否则要换 $-INF$ 或者额外的 exists 标志位。
- 改成求方案数就把 max 换成 +, 求异或就换成 ^, 模板完全一致。

11.3 线段树维护 dp 求最大独立集

把「区间最大独立集」拍成可合并信息: 每个区间维护 $dp[i][j]$, $i, j \in \{0, 1\}$ 表示该区间左端点是否选、右端点是否选时的最大权和。合并时若左区间右端 $j = 1$ 且右区间左端 $x = 1$ 则非法 (相邻同选)。单点 $\text{Info}(x)$: $dp[1][1] = x$, $dp[0][0] = 0$, 对角 $-\infty$ 。区间查询直接 $\text{seg.query}(l, r)$ 得到该段最大独立集 (四个状态取 max)。

参考提交: [ABC 456 提交 75466400](#)。

```

struct Info {
    array<array<ll, 2>, 2> dp;
    Info(ll x = 0) {
        dp[0][1] = dp[1][0] = -INF;
        dp[1][1] = x;
        dp[0][0] = 0;
    }
};

Info operator+(const Info &a, const Info &b) {
    Info fa;
    for (int i = 0; i <= 1; i++) {
        for (int j = 0; j <= 1; j++) {

```

```

            if (a.dp[i][j] == -INF) continue;
            for (int x = 0; x <= 1; x++) {
                for (int y = 0; y <= 1; y++) {
                    if (b.dp[x][y] == -INF) continue;
                    if (j == 1 && x == 1) continue;
                    fa.dp[i][y] = max(a.dp[i][j] +
                                       ↳ b.dp[x][y], fa.dp[i][y]);
                }
            }
        }
    }
    return fa;
}

template <class InfoType = Info>
class LazySegmentTree {
    int n;
    vector<InfoType> tree;

    void pushUp(int id) {
        tree[id] = tree[id << 1] + tree[id << 1 |
        ↳ 1];
    }

    void build(int id, int l, int r, const
    ↳ vector<InfoType> &a) {
        if (l == r) { tree[id] = a[l]; return; }
        int mid = (l + r) >> 1;
        build(id << 1, l, mid, a);
        build(id << 1 | 1, mid + 1, r, a);
        pushUp(id);
    }

    InfoType query(int id, int l, int r, int ql,
    ↳ int qr) {
        if (ql <= l && r <= qr) return tree[id];
        int mid = (l + r) >> 1;
        if (qr <= mid) return query(id << 1, l,
        ↳ mid, ql, qr);
        if (ql > mid) return query(id << 1 | 1,
        ↳ mid + 1, r, ql, qr);
        return query(id << 1, l, mid, ql, qr) +
        ↳ query(id << 1 | 1, mid + 1, r, ql,
        ↳ qr);
    }

public:
    LazySegmentTree(int n = 0) { init(n); }
    LazySegmentTree(const vector<InfoType> &a) {
        ↳ build(a); }

    void init(int n_) {
        n = n_;
        tree.assign(4 * n + 5, InfoType());
    }

    void build(const vector<InfoType> &a) {
        init((int)a.size() - 1);
        if (n > 0) build(1, 1, n, a);
    }

    InfoType query(int l, int r) {
        if (l > r) return 0;
        return query(1, 1, n, l, r);
    }
}

```

```

    }
};

void solve() {
    int n, k;
    cin >> n >> k;
    vector<ll> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    vector<Info> b(n + 1);
    for (int i = 1; i <= n; i++) b[i] =
        ↳ Info(a[i]);
    for (int i = 1; i <= n; i++) a[i] = a[i] +
        ↳ a[i - 1];
    LazySegmentTree<Info> seg;
    seg.build(b);
    ll ans = INF;
    for (int len = k - 1; len <= k; len++) {
        for (int l = 1; l + len <= n; l++) {
            int r = l + len;
            ll candidate = a[r] - a[l - 1];
            auto info = seg.query(l + 1, r - 1);
            ll maxi = -INF;
            for (int i = 0; i <= 1; i++)
                for (int j = 0; j <= 1; j++)
                    maxi = max(maxi, info.dp[i][j]);
            candidate -= maxi;
            ans = min(ans, candidate);
        }
    }
    cout << ans << "\n";
}

```

12 计算几何

12.1 点线基础模板

以下是计算几何最底层的几个基础类型：符号判定 `sgn`、点 / 向量 `Point`（点和向量同一类型，向量当成「以原点为起点」的点）、叉乘 / 点乘、两点式直线 `Line`。后续所有子节的代码都建立在这些定义之上，每节顶部会以注释形式列出本节用到的依赖（具体实现见此处）。

`Point / Line` 都是 `template<class T>` 泛型（无默认），调用方在 call site 显式给 `T`（如 `Point<ll>`、`Point<DB>`）；同一份模板可同时被整数 `Point<ll>` 与浮点 `Point<DB>` 实例化（对拍 generator 常见需求）。向量类型不另起别名，下游函数体里取边向量直接 `auto v = poly[i+1] - poly[i]`，让编译器把它推成 `Point<T>`（点和向量同形）。当 `T` 为整数类型（含 `ll` 与 `__int128`）时，`cross / norm` 自动放大到 `__int128_t` 计算以避免溢出。`Point::abs()` 走自己写的 `sqrtL`：浮点直接 `sqrtl`，整数走 `isqrt + 修正`，避免 `(long double) i128` 在 $n > 2^{53}$ 时丢整数精度。

[change log](#)

```
// floor(sqrt(n)), n >= 0; 二分 + 防溢出 (mid <= n/mid 等价 mid*mid <= n)
i128 isqrt(i128 n) {
    if (n <= 0) return 0;
    i128 lo = 0, hi = (i128) 2e18; // sqrt(4e36) <= 2e18
    while (lo < hi) {
        i128 mid = lo + (hi - lo + 1) / 2;
        if (mid <= n / mid) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}

// 统一 sqrt 入口：浮点直接 sqrtl；整数（含 i128）走 isqrt + 修正，
// 避免 (long double) i128 在 n > 2^53 时丢整数精度
template<typename T>
long double sqrtL(T n) {
    if constexpr (is_floating_point_v<T>) {
        return sqrtl((long double) n);
    } else {
        i128 ni = (i128) n;
        if (ni <= 0) return 0;
        i128 i = isqrt(ni);
        long double id = (long double) i;
        long double rd = (long double) (ni - i * i);
        // sqrt(i^2 + r) = i * sqrt(1 + r/i^2)
        return id * sqrtl(1.0L + rd / (id * id));
    }
}

// x*x 安全平方：整数升 i128 防溢出，浮点原样 T*T (i128 没浮点意义)。
template<class T>
auto squared(T x) {
    if constexpr (is_floating_point_v<T>) {
        return x * x;
    } else {
        return (i128) x * (i128) x;
    }
}

template<class T>
int sgn(T x) {
    if (-eps < x && x < eps) return 0;
    return x < 0 ? -1 : 1;
}

template<class T>
struct Point {
    T x, y;
```

```

int id = -1; // 原始下标. -1 表示派生点 (+ - * 等算术结果) 或未赋值, id 无意义

explicit Point(T x_ = 0, T y_ = 0) : x(x_), y(y_) {}

// Point<U> -> Point<T> 隐式 (common_type 单调升档方向, U -> T 必须无损):
// common_type_t<U, T> == T 才放行 (例 ll -> DB 通; DB -> ll 拦, 防 truncation)
template<class U, class = enable_if_t<is_same_v<common_type_t<U, T>, T>>>
Point(const Point<U> &o) : x((T) o.x), y((T) o.y), id(o.id) {}

// 只对左值生效的复合运算符
Point &operator+=(Point p) & { x += p.x, y += p.y; return *this; }
Point &operator-=(Point p) & { x -= p.x, y -= p.y; return *this; }
Point &operator*=(T t) & { x *= t, y *= t; return *this; }
Point &operator/=(T t) & { x /= t, y /= t; return *this; }

Point operator-() const { return Point(-x, -y); }

friend Point operator+(Point a, Point b) { return a += b; }
friend Point operator-(Point a, Point b) { return a -= b; }

// 标量乘除: 返回 Point<common_type_t<T, U>> (C++14), 自动算公共类型
// <ll,ll>=ll / <ll,DB>=DB / <DB,ll>=DB; R==T 走 copy ctor, R!=T 走 converting ctor
template<class U>
friend auto operator*(Point a, U b) {
    Point<common_type_t<T, U>> r = a; r *= b; return r;
}
template<class U>
friend auto operator*(U a, Point b) {
    Point<common_type_t<T, U>> r = b; r *= a; return r;
}
template<class U>
friend auto operator/(Point a, U b) {
    Point<common_type_t<T, U>> r = a; r /= b; return r;
}

friend bool operator<(Point a, Point b) {
    int sx = sgn(a.x - b.x);
    if (sx != 0) return a.x < b.x;
    return a.y < b.y;
}
friend bool operator>(Point a, Point b) { return b < a; }
friend bool operator==(Point a, Point b) {
    return sgn(a.x - b.x) == 0 && sgn(a.y - b.y) == 0;
}
friend bool operator!=(Point a, Point b) { return !(a == b); }
friend istream &operator>>(istream &is, Point &p) { return is >> p.x >> p.y; }

// !is_floating_point_v 而不是 is_integral_v: 让 i128 / __int128 也走整数分支
auto norm() const {
    if constexpr (!is_floating_point_v<T>) {
        return (__int128_t) x * x + (__int128_t) y * y;
    } else {
        return x * x + y * y;
    }
}
DB abs() const { return sqrtL(norm()); }

Point rot90() const { return Point(-y, x); }
Point<DB> rot(DB ang) const {
    return Point<DB>(cosl(ang) * x - sinl(ang) * y,
                    sinl(ang) * x + cosl(ang) * y);
}

```

```

// L1 范数 (曼哈顿) =  $|x| + |y|$ . 整型分支走 __int128_t 防溢出.
auto normL1() const {
    auto ax = (x < 0 ? -x : x), ay = (y < 0 ? -y : y);
    if constexpr (!is_floating_point_v<T>) {
        return (__int128_t) ax + (__int128_t) ay;
    } else {
        return ax + ay;
    }
}

// Linf 范数 (切比雪夫) =  $\max(|x|, |y|)$ . 返回类型与 T 同.
T normLinf() const {
    auto ax = (x < 0 ? -x : x), ay = (y < 0 ? -y : y);
    return ax > ay ? ax : ay;
}
};

template<class T>
auto cross(Point<T> a, Point<T> b) {
    if constexpr (!is_floating_point_v<T>) {
        return (__int128_t) a.x * b.y - (__int128_t) a.y * b.x;
    } else {
        return a.x * b.y - a.y * b.x;
    }
}

// 三参数重载: 以 o 为原点算  $(a - o) \times (b - o)$ 
// 用途: 判 3 点 turn (>0 左转 / <0 右转 / =0 共线) / 2 倍三角形有向面积
template<class T>
auto cross(Point<T> o, Point<T> a, Point<T> b) { return cross(a - o, b - o); }

template<class T, class U>
auto dot(Point<T> a, Point<U> b) {
    // 在 T 为整数的时候, 使用 __int128_t 整型进行计算
    if constexpr (!is_floating_point_v<T> && !is_floating_point_v<U>) {
        return (__int128_t) a.x * b.x + (__int128_t) a.y * b.y;
    } else {
        return a.x * b.x + a.y * b.y;
    }
}

// 两点式直线
template<class T>
struct Line {
    Point<T> p1, p2;

    Line() = default;

    Line(Point<T> a, Point<T> b) : p1(a), p2(b) {}

    // Line<U> -> Line<T> 隐式 (common_type 单调升档方向, 同 Point converting ctor)
    template<class U, class = enable_if_t<is_same_v<common_type_t<U, T>, T>>>
    Line(const Line<U> &o) : p1(o.p1), p2(o.p2) {}

    // AC https://qoj.ac/submission/2146870
    Line(T k, T c) : p1(Point<T>(0, c)), p2(Point<T>(1, k + c)) {}

    friend bool operator<(Line a, Line b) {
        if (a.p1 != b.p1) return a.p1 < b.p1;
        return a.p2 < b.p2;
    }

    friend bool operator==(Line a, Line b) {
        return a.p1 == b.p1 && a.p2 == b.p2;
    }
}

```

```

friend istream &operator>>(istream &is, Line &e) {
    T a, b, c, d;
    is >> a >> b >> c >> d;
    e = Line(Point<T>(a, b), Point<T>(c, d));
    return is;
}
};

Line<DB> gen_line_from_general(DB a, DB b, DB c) {
    // 方向向量 (b, -a)
    Point<DB> p1, p2;
    if (b != 0) {
        p1 = Point<DB>(0, -c / b);
    } else {
        p1 = Point<DB>(-c / a, 0);
    }
    p2 = Point<DB>(p1.x + b, p1.y - a);
    return Line<DB>(p1, p2);
}

mt19937_64 rng(std::chrono::steady_clock::now().time_since_epoch().count());

```

参考题单：计算几何模板题目总结 (vjudge) 以及 QOJ 模板赛 2646。

12.2 点在线上的投影点

vjudge 提交。

设 $\text{vec} = l.p2 - l.p1$ ，把向量 $p - l.p1$ 在 vec 方向上的归一化投影长度记为 r ，于是投影点等于 $l.p1 + r * \text{vec}$ 。返回值固定为 $\text{Point}<\text{DB}>$ （除法走浮点，输入 T / U 是 ll 还是 DB 都向 DB 收敛），不再 `assert` 浮点。

```

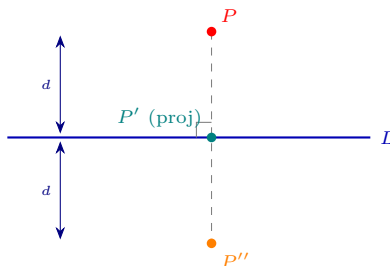
// 依赖（基础模板已定义）：
// struct Point, struct Line
// dot(Point, Point)
// 使用这个东西，Point 必须是 double 等浮点类型
template<class T, class U>
Point<DB> project(Point<T> p, Line<U> l) {
    Point<DB> vec = l.p2 - l.p1;
    // 这个距离反正早除晚除都要除掉
    DB r = dot(vec, p - l.p1) / ((DB) vec.x * vec.x + vec.y * vec.y);
    return l.p1 + vec * r;
}

```

12.3 对称点

vjudge 提交。

设 proj 为投影点，则对称点 = $\text{proj} + \text{proj} - p$ 。直观上：以投影点为中点，原点和对称点处于直线两侧、距离相等。



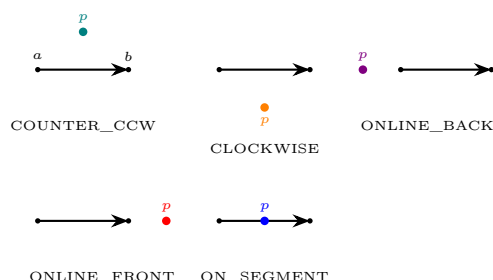
图注： P' （投影）是 P 到 L 的垂足； P'' （对称点）满足 P' 是 P, P'' 中点，从而 $P'' = 2P' - P$ 。


```
// 依赖:
// struct Point, struct Line
// project(Point, Line) (见前一节)
template<class T, class U>
Point<DB> reflect(Point<T> p, Line<U> l) {
    Point<DB> proj = project(p, l);
    return proj + proj - p;
}
```

12.4 点线位置判断 (CCW)

[vjude 提交](#)。

给一条有向直线 $\overrightarrow{a \rightarrow b}$, 目标点 p 落在何处由 $\text{cross}(\vec{ab}, \vec{ap})$ 与 $\text{dot}(\vec{ab}, \vec{ap})$ 决定, 共五种情况:



图注: 先看 $\text{cross}(\vec{ab}, \vec{ap})$ 的符号——正左 / 负右; 为零再看 dot 与 $|\vec{ab}|, |\vec{ap}|$ 的比较, 区分 p 落在线段反方向延长 (BACK)、正方向延长 (FRONT) 还是线段内部 (ON)。

```
// 依赖:
// struct Point, struct Line
// sgn, cross, dot
// Point::norm()
constexpr int ON_SEGMENT = 0;
constexpr int COUNTER_CLOCKWISE = 1;
constexpr int CLOCKWISE = -1;
constexpr int ONLINE_BACK = 2;
constexpr int ONLINE_FRONT = -2;

template<class T, class U>
int CCW(Point<T> p, Line<U> l) {
    auto a = l.p2 - l.p1;
    auto b = p - l.p1;
    if (sgn(cross(a, b)) > 0) {
        return COUNTER_CLOCKWISE;
    }
    if (sgn(cross(a, b)) < 0) {
        return CLOCKWISE;
    }
    if (sgn(dot(a, b)) < 0) {
        return ONLINE_BACK;
    }
    if (a.norm() < b.norm()) return ONLINE_FRONT;
    return ON_SEGMENT;
}
```

12.4.1 逆时针弧角 ccw_angle (标量 / 向量两版)

求「从 from 逆时针转到 to」的有向夹角, 结果恒落在 $[0, 2\pi)$, 而不是 $|\text{to} - \text{from}|$ (后者在跨 $0/2\pi$ 处会算错)。两个重载: 标量版输入两个极角, 向量版输入两个方向向量、用单次 $\text{atan2}(\text{cross}, \text{dot})$ (叉积当 $\sin$ 分量、点积当 $\cos$ 分量) 一步求出, 比两次 atan2 相减更稳。

```

// 依赖:
// struct Point, cross, dot (向量版用到)
// 圆弧 from -> to 的逆时针弧角差 [0, 2pi), 不是 abs(to - from)
static DB ccw_angle(DB theta_from, DB theta_to) {
    DB d = std::fmod(theta_to - theta_from, 2 * PI);
    if (d < 0) d += 2 * PI;
    return d;
}

// 向量版: 从方向向量 from 逆时针转到方向向量 to 的角 [0, 2pi).
// 用单次 atan2(cross, dot) (cross 当 sin 分量 / dot 当 cos 分量), 比两次 atan2 相减稳.
template<class T, class U>
static DB ccw_angle(Point<T> from, Point<U> to) {
    DB d = std::atan2((DB)cross(from, to), (DB)dot(from, to));
    if (d < 0) d += 2 * PI;
    return d;
}

```

12.5 判断两直线是否垂直 / 平行

[vjudge 提交](#)。

分别取两条直线的方向向量, 看 dot 是否为零 (垂直) 或 cross 是否为零 (平行)。

```

// 依赖:
// struct Line
// sgn, cross, dot
template<class T, class U>
bool isOrthogonal(Line<T> a, Line<U> b) {
    auto c = a.p2 - a.p1, d = b.p2 - b.p1;
    return !sgn(dot(c, d));
}

template<class T, class U>
bool isParallel(Line<T> a, Line<U> b) {
    auto c = a.p2 - a.p1, d = b.p2 - b.p1;
    return !sgn(cross(c, d));
}

```

12.6 判断两线段是否相交

[vjudge 提交](#)。

a 与 b 相交当且仅当 b 的两端点关于 a 异侧 (含端点重合) 且 a 的两端点关于 b 异侧。利用 CCW 的符号相乘 ≤ 0 即可。

```

// 依赖:
// struct Line
// CCW(Point, Line)
template<class T, class U>
bool isSegIntersect(Line<T> a, Line<U> b) {
    // 判断线段相交
    return CCW(b.p1, a) * CCW(b.p2, a) <= 0
        && CCW(a.p1, b) * CCW(a.p2, b) <= 0;
}

```

命名提示: 函数名特意叫 isSegIntersect (不是 isIntersect) —— Line<T> 在本模板里同时承担「直线」与「线段」两种语义, 这里 a, b 是被当成线段判定的, 函数名前加 Seg 防止与「两直线是否相交」(任意非平行直线都相交) 混淆。

12.7 两条直线交点

[vjudge 提交](#)。

设两条直线分别用方向式 $p + t\vec{v}$ 、 $q + s\vec{w}$ 。在交点处：

$$p + t\vec{v} = q + s\vec{w}$$

$$(p - q) \times \vec{w} + t\vec{v} \times \vec{w} = 0 \quad (\text{两边叉乘 } \vec{w}, \text{ 自身叉乘消去 } s\vec{w})$$

$$t = \frac{\vec{w} \times (q - p)}{\vec{v} \times \vec{w}}$$

交点即 $p + t\vec{v}$ 。返回值固定为 `Point<DB>`（除法走浮点，不论输入 `T` 是什么都向 `DB` 收敛）。函数体内先把所有 `Point<T>` 走 promotion 提到 `Point<DB>`，整数 `T` 也合法（实参为整数 `Line<ll>` 时不再 `assert`）。

```
// 依赖:
// struct Point, struct Line
// cross(Point, Point)
// p+tv=q+sw
// (p-q) x w + tv x w = 0 (利用叉乘乘以自身为 0, 消掉 sw)
// t=-(p-q)x w/(vxw)
// 传入的是方向式 拿到的是两直线交点
template<class T, class U>
Point<DB> getintersect(const Line<T> &a, const Line<U> &b) {
    Point<DB> p = a.p1, v = a.p2 - a.p1, q = b.p1, w = b.p2 - b.p1;
    Point<DB> u = p - q;
    DB t = (DB) cross(w, u) / cross(v, w);
    return p + v * t;
}
```

12.8 两线段间最小距离

[vjude 提交](#)。

若两线段相交则距离为 0；否则取四个「端点 → 对方线段」距离的最小值。点到线段距离 `distancePS` 中要先判断投影是否落在线段两端之外，落到外面就退化为对应端点的距离。

```
// 依赖:
// struct Point, struct Line
// sgn, cross, dot
// Point::abs()
// isSegIntersect(Line, Line)
template<class T, class U>
DB distancePL(Point<T> p, Line<U> l) {
    auto val = cross(l.p1, l.p2, p); // 三参数: 以 l.p1 为原点
    return sgn(val) * val / (l.p2 - l.p1).abs();
}

// AC https://qoj.ac/submission/2001668
template<class T, class U>
DB distancePS(Point<T> p, Line<U> l) {
    if (l.p1 == l.p2) return (p - l.p1).abs(); // 退化为点
    auto t = l.p2 - l.p1;
    if (sgn(dot(t, p - l.p1)) < 0) return (p - l.p1).abs();
    if (sgn(dot(t, p - l.p2)) > 0) return (p - l.p2).abs();
    return distancePL(p, l);
}

// AC https://qoj.ac/submission/2001588
template<class T, class U>
DB distanceSS(Line<T> a, Line<U> b) {
    if (isSegIntersect(a, b)) return 0;
    return min({distancePS(a.p1, b), distancePS(a.p2, b),
                distancePS(b.p1, a), distancePS(b.p2, a)});
}
```

12.9 多边形面积

[vjjudge 提交](#)。

用「向量叉乘求三角形有向面积之和」（鞋带公式）：

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} \text{cross}(P_i, P_{(i+1) \bmod n}) \right|$$

逆时针排列时 $\sum$ 本身已为正，顺时针为负，统一取绝对值。

拆出 polygonSignedArea：返回值 auto 跟着 cross 走（整数 $T \rightarrow \text{i128}$ ，浮点 $T \rightarrow T$ ）。**不要**像旧版那样写成 `DB res = 0; res += cross(...)`——在整数 T 下 cross 返回 `__int128_t`，赋给 double 累加器会在 $|val| > 2^{53}$ 时丢精度。除此之外，`reorder_polygon` 也复用 `polygonSignedArea` 取符号判方向，避免「前三点共线」时的局部退化。

```
// 依赖：
// struct Point
// cross(Point, Point)
// Point::abs()

// 多边形 2 倍有向面积（不除 2 以保整数精度）
// 返回类型 auto: 整数 T -> i128, 浮点 T -> T
// > 0: CCW; < 0: CW; == 0: 退化（全共线 / 自交）
template<class T>
auto polygonSignedArea(const vector<Point<T>> &poly) {
    auto res = cross(Point<T>(), Point<T>()); // 零值，类型由 cross 继承
    size_t n = poly.size();
    for (size_t i = 0; i < n; ++i) res += cross(poly[i], poly[(i + 1) % n]);
    return res;
}

template<class T>
DB polygonArea(const vector<Point<T>> &poly) {
    auto s = polygonSignedArea(poly);
    if (s < 0) s = -s;
    return (DB) s / 2.0;
}

template<class T>
DB polygon_perimeter(const vector<Point<T>> &poly) {
    DB res = 0;
    size_t n = poly.size();
    for (int i = 0; i < n; ++i) res += (poly[i] - poly[(i + 1) % n]).abs();
    return res;
}

// 正 n 边形面积:  $A = n * l^2 / (4 * \tan(\pi / n))$ ,  $n \geq 3$ 
DB regular_polygon_area(ll n, DB l) {
    return (DB) n * l * l / (4.0L * tanl(PI / (DB) n));
}
```

12.10 圆 Circle：测度五件套

圆心 c + 半径 r 的最小描述，配套弧长 / 弦长 / 弓形高 / 扇形面积 / 弓形面积五件套测度。

角度约定：入参 θ 是弦所对的圆心角，单位弧度，约束 $\theta \in [0, 2\pi]$ 。 $\theta < \pi$ 是 minor 弓形（小的那块）， $\theta > \pi$ 是 major 弓形（大的那块），公式同一个，**不做内部 deg $\leftrightarrow$ rad 转换**（调用方自己 $\text{deg} * \text{PI} / 180$ ）。`point_at` 接受任意 θ （含负值 / 超 2π ），不抛 `assert`——绕圆走多圈合法。

弓形面积公式：弓形 = 扇形 - 三角形，化简得

$$S_{\text{seg}}(\theta) = \frac{1}{2} r^2 (\theta - \sin \theta).$$

AC 实战：[Gym 105161B Area of the Devil](#)（圆 - 五块 segment - 五块三角形）。

ccw_angle helper：从极角 θ_{from} 转到 θ_{to} 的弧度差，落在 $[0, 2\pi)$ 。**不要**用 `abs(theta_to - theta_from)`——后者在 wrap 处（例如 `from = 352°`, `to = 54°`）会算成 298° 而非真实的 62° 。

```

// 依赖:
// struct Point, DB = long double, PI, eps
template<class T>
struct Circle {
    Point<T> c;
    T r;

    Circle() = default;
    Circle(Point<T> c_, T r_) : c(c_), r(r_) {}

    // Circle<U> -> Circle<T> 隐式 (common_type 单调升档方向, 同 Point converting ctor)
    template<class U, class = enable_if_t<is_same_v<common_type_t<U, T>, T>>>
    Circle(const Circle<U> &o) : c(o.c), r(o.r) {}

    // 圆上极角 theta 处的点 c + (r cos theta, r sin theta)
    // theta 不限范围 (允许负值 / > 2 PI), 始终走 cos/sin 不抛 assert
    Point<DB> point_at(DB theta) const {
        return Point<DB>((DB) c.x + (DB) r * cosl(theta),
                        (DB) c.y + (DB) r * sinl(theta));
    }

    void check_theta(DB theta) const {
#ifdef LOCAL
        assert(theta >= -eps && theta <= 2 * PI + eps);
#endif
        (void) theta;
    }

    // 弧长 = r theta
    DB arc_length(DB theta) const {
        check_theta(theta);
        return (DB) r * theta;
    }

    // 弦长 = 2 r sin(theta / 2)
    DB chord_length(DB theta) const {
        check_theta(theta);
        return 2.0L * (DB) r * sinl(theta / 2);
    }

    // 弓形高 (sagitta) = r (1 - cos(theta / 2))
    DB sagitta(DB theta) const {
        check_theta(theta);
        return (DB) r * (1.0L - cosl(theta / 2));
    }

    // 扇形面积 = (1/2) r^2 theta
    DB sector_area(DB theta) const {
        check_theta(theta);
        return 0.5L * (DB) r * r * theta;
    }

    // 弓形面积 = (1/2) r^2 (theta - sin theta)
    // AC https://vjudge.net/solution/69814399 (Area of the Devil)
    DB segment_area(DB theta) const {
        check_theta(theta);
        return 0.5L * (DB) r * r * (theta - sinl(theta));
    }

    // AC https://vjudge.net/solution/69993234 使用 sgn 避免精度问题
    template<class U>
    bool is_point_in_circle(const Point<U> &p) {
        return sgn(distancePPNorm(p, c) - squared(r)) <= 0;
    }
}

```

```

}
};

```

12.11 两圆位置关系 circle_relation

vjudge 提交 (Aizu CGL_7_A)。

按圆心距 d 与 $r_1 \pm r_2$ 的关系判 5 种位置：相离 / 外切 / 相交 / 内切 / 内含。enum 取值即两圆公切线条数（外离 4 / 外切 3 / 相交 2 / 内切 1 / 内含 0），方便直接拿来当答案输出。

重合圆 sentinel COINCIDENT = -1：同心同径时公切线无穷多，单独一档；调用方按需特判（如题面要求输出 -1 / Inf / 任意大数）。

用 $\text{squared}(d)$ 与 $\text{squared}(r_1 \pm r_2)$ 比较，不直接比 d ：整数情形 $(r_1 + r_2)^2$ 在 1l 边界（输入 1e9 量级）会溢，走 $\text{squared}()$ 自动升 `__int128_t` 防溢出；浮点情形也省一次 sqrt 。

```

// 依赖：
// struct Circle, distancePPNorm, squared, sgn
enum CircleRelation {
    // 表示这个公切线个数；COINCIDENT (-1) 是重合 sentinel (无穷多公切线)
    COINCIDENT = -1,
    ONE_CONTAINS_OTHER = 0,
    INTERNALLY_TANGENT = 1,
    INTERSECTING = 2,
    EXTERNALLY_TANGENT = 3,
    EXTERNALLY_APART = 4,
};

template<class T>
CircleRelation circle_relation(const Circle<T> &a, const Circle<T> &b) {
    auto d2 = distancePPNorm(a.c, b.c);
    auto sum = squared(a.r + b.r);
    auto dif = squared(a.r - b.r);
    // 重合 (同心 + 同径)：无穷多公切线，单独 sentinel
    if (sgn(d2) == 0 && sgn(a.r - b.r) == 0) return COINCIDENT;
    if (sgn(d2 - sum) > 0) return EXTERNALLY_APART;
    if (sgn(d2 - sum) == 0) return EXTERNALLY_TANGENT;
    if (sgn(d2 - dif) > 0) return INTERSECTING;
    if (sgn(d2 - dif) == 0) return INTERNALLY_TANGENT;
    return ONE_CONTAINS_OTHER;
}

```

12.12 三角形内接圆 / 外接圆

内接圆 incircle_triangle: vjudge 提交 (Aizu CGL_7_B)。内心 I 是三条角平分线的交点，用重心式直接写出：

$$I = \frac{|BC| \cdot A + |AC| \cdot B + |AB| \cdot C}{|AB| + |AC| + |BC|}.$$

内切圆半径 r = 内心到任意一条边的距离，取 $\min$ 三条边距离做防御性数值兜底（三个值理论上相等，浮点下取最小避免 r 越界出三角形）。

外接圆 outcircle_triangle: vjudge 提交 (Aizu CGL_7_C)。外心 O 是三条垂直平分线的交点，取 AB 与 AC 两边的中垂线求交即可：

- AB 中点 $M_{AB} = (A + B)/2$ ，中垂线方向 $= (B - A).rot90()$ ；
- AC 中点 $M_{AC} = (A + C)/2$ ，中垂线方向 $= (C - A).rot90()$ ；
- $O = \text{getintersect}(L_{AB}, L_{AC})$ ， $R = |O - A|$ 。

不要用「重心式」($I = (a \cdot BC + b \cdot AC + c \cdot AB)/(AB + AC + BC)$) 求外心——那是内心公式，等腰 / 等边三角形样例下两者重合，会侥幸过样例但 WA。

退化 sentinel: 三点共线 / 任意两点重合 ($\text{cross}(a, b, c) == 0$) 时，内接圆 / 外接圆都不存在，分别返 $r = 0$ 与 $r = -1$ 作 sentinel；调用方用 $\text{ci.r} == 0$ / $\text{co.r} < 0$ 判退化即可，不会再吃到除零 NaN。

```

// 依赖:
// struct Point, struct Line, struct Circle
// distancePP, distancePS, getintersect, Point::rot90, sgn, cross (三参数)
// 退化 (共线 / 任意两点重合, cross == 0): 内接圆不存在, 返回 r=0 sentinel
Circle<DB> incircle_triangle(Point<DB> a, Point<DB> b, Point<DB> c) {
    if (sgn(cross(a, b, c)) == 0) return Circle<DB>(a, 0);
    DB ab = distancePP(a, b), ac = distancePP(a, c), bc = distancePP(b, c);
    Point<DB> I = (a * bc + ac * b + ab * c) / (ab + ac + bc);
    return Circle<DB>(I, min({distancePS(I, Line(a, c)),
                             distancePS(I, Line(a, b)),
                             distancePS(I, Line(b, c))}));
}

// 外心 = 两条中垂线交点
// 退化 (共线 / 任意两点重合, cross == 0): 外接圆不存在 (半径无穷大), 返回 r=-1 sentinel
Circle<DB> outcircle_triangle(Point<DB> a, Point<DB> b, Point<DB> c) {
    if (sgn(cross(a, b, c)) == 0) return Circle<DB>(a, -1);
    Point<DB> abMid = (a + b) / 2, acMid = (a + c) / 2;
    Point<DB> vecAb = (b - a).rot90(), vecAc = (c - a).rot90();
    Point<DB> I = getintersect(Line(abMid, abMid + vecAb),
                              Line(acMid, acMid + vecAc));
    return Circle<DB>(I, distancePP(I, a));
}

```

12.13 直线与圆的关系 / 交点 circle_line_relation / getCrossPointsCL

[vjude 提交 \(Aizu CGL_7_D\)](#); 弦长版 [vjude 提交 \(EOlymp-359\)](#)。

关系判定: 圆心 C 到直线距离 $d = |\text{cross}(p_1, p_2, C)| / |p_2 - p_1|$, 与半径 r 比较。两边平方 + 乘 $|p_2 - p_1|^2$ 走整数: cross^2 vs $r^2 \cdot |p_2 - p_1|^2$, `squared()` 自动升 `__int128_t` 防溢。

交点求法: 先取垂足 $F = \text{project}(C, l)$, 半弦长 $h = \sqrt{r^2 - |CF|^2}$ (斜边 r 与直角边 $|CF|$ 在直角三角形里推另一直角边), 单位方向 $\hat{d} = (p_2 - p_1) / |p_2 - p_1|$, 交点 $F \pm \hat{d} \cdot h$ 。

退化 sentinel: $l.p_1 == l.p_2$ 时直接返 0 / 空集, 不会跑到 `project` 的除零路径。

```

// 依赖:
// struct Point, struct Line, struct Circle
// cross (三参数), squared, sgn, project, Point::abs
template<class T>
int circle_line_relation(const Circle<T> &cir, const Line<T> &l) {
    // 退化直线 (p1 == p2): 不构成有效直线, 视为无交点
    if (l.p1 == l.p2) return 0;
    auto cr2 = squared(cross(l.p1, l.p2, cir.c));
    auto lenSq = (l.p2 - l.p1).norm();
    auto rhs = squared(cir.r) * lenSq;
    int s = sgn(cr2 - rhs);
    if (s > 0) return 0;
    if (s == 0) return 1;
    return 2;
}

template<class T>
vector<Point<DB>> getCrossPointsCL(const Circle<T> &cir, const Line<T> &l) {
    int k = circle_line_relation(cir, l);
    if (k == 0) return {};
    // 得到垂足
    Point<DB> F = project(cir.c, l);
    if (k == 1) return {F};
    Point<DB> d = l.p2 - l.p1;
    Point<DB> dhat = d / d.abs();
    // 斜边 R 与直角边 (圆心和垂足) 得到另外一直角边 (交点与垂足)
    DB h = sqrtl(max((DB)0, (DB)cir.r * cir.r - (cir.c - F).norm()));
    return {F - dhat * h, F + dhat * h};
}

```

```
}

```

12.14 两圆的关系 / 交点 circle_circle_relation / getCrossPointsCC

vjudge 提交 (Aizu CGL_7_E); 含重合 sentinel 版 vjudge 提交 (EOlymp-4779)。

关系判定: d^2 、 $(r_1 + r_2)^2$ 、 $(r_1 - r_2)^2$ 全走整数 squared 比较, 不开方。 $d^2 > (r_1 + r_2)^2$ 相离; $d^2 = (r_1 + r_2)^2$ 外切; $(r_1 - r_2)^2 < d^2 < (r_1 + r_2)^2$ 相交; $d^2 = (r_1 - r_2)^2$ 内切; $d^2 < (r_1 - r_2)^2$ 内含。

交点求法: 连心线方向 $\hat{d} = (c_2.c - c_1.c) / D$ 。弦中点 $M = c_1.c + \hat{d} \cdot a$, 其中 $a = (r_1^2 - r_2^2 + D^2) / (2D)$ (联立两圆方程相减消平方项得到)。沿连心线垂直方向 $\hat{d}.rot90() \pm$ 半弦长 $h = \sqrt{r_1^2 - a^2}$ 即两交点。

重合圆 sentinel: 同心同径时 circle_circle_relation 返 -1 (交点无穷多), getCrossPointsCC 据此返空, 重合语义由 caller 自己处理。

promotion 坑: $(c_2.c - c_1.c) / D$ 若 cN 是 Point<ll>, D 是 DB, 会走 Point<ll>::operator/(Point<ll>, ll) 把 D 截成整数。必须先 Point<DB> $d = c_2.c - c_1.c$;

```
// 依赖:
// struct Point, struct Circle
// distancePPNorm, squared, sgn, Point::abs, Point::rot90
template<class T>
int circle_circle_relation(const Circle<T> &a, const Circle<T> &b) {
    auto d2 = distancePPNorm(a.c, b.c);
    auto sum = squared(a.r + b.r);
    auto dif = squared(a.r - b.r);
    // 重合 (同心 + 同径): 交点无穷多, 返 -1 sentinel
    if (sgn(d2) == 0 && sgn(a.r - b.r) == 0) return -1;
    if (sgn(d2 - sum) > 0) return 0;
    if (sgn(d2 - dif) < 0) return 0;
    if (sgn(d2 - sum) == 0 || sgn(d2 - dif) == 0) return 1;
    return 2;
}

template<class T>
vector<Point<DB>> getCrossPointsCC(const Circle<T> &c1, const Circle<T> &c2) {
    int k = circle_circle_relation(c1, c2);
    // k = -1 重合 (无穷多) / k = 0 不交: 统一返空, 重合语义由 caller 自己处理
    if (k <= 0) return {};
    // 弦中点
    Point<DB> d = c2.c - c1.c;
    DB D = d.abs();
    Point<DB> dhat = d / D;
    DB a = ((DB)c1.r * c1.r - (DB)c2.r * c2.r + D * D) / (2 * D);
    Point<DB> M = (Point<DB>)c1.c + dhat * a;
    if (k == 1) return {M};
    // 沿连心线垂直方向 +/- 半弦长
    DB h = sqrtl(max((DB)0, (DB)c1.r * c1.r - a * a));
    Point<DB> nhathat = dhat.rot90();
    return {M + nhathat * h, M - nhathat * h};
}
```

12.15 过一点圆的切点 getTangentPoints

vjudge 提交 (Aizu CGL_7_F)。

推导: 过外部点 P 作圆 (C, r) 切线时, 切点 T 同时满足 $PT \perp CT$ 与 $|CT| = r$, 即 $\triangle PCT$ 在 T 处是直角三角形, $|PT| = \sqrt{|PC|^2 - r^2}$ 。切点 T 同时落在原圆 (C, r) 与辅圆 $(P, |PT|)$ 上, 对两圆求交即得。

三种退化分类直接 early return: P 在圆内 (含圆心) 返空; P 在圆上唯一切点就是 P 自身; P 在圆外才走辅圆链。比”全部喂给 getCrossPointsCC 让它兜底”显式可读, 也免依赖辅圆 $r = 0$ 时的边界稳定性。

```
// 依赖:
// struct Point, struct Circle
// getCrossPointsCC
// 退化: P 在圆内 (含圆心) 返空; P 在圆上唯一切点就是 P 自身
vector<Point<DB>> getTangentPoints(Point<ll> P, Circle<ll> cir) {
```



```

auto d2 = (P - cir.c).norm();
auto r2 = (__int128_t)cir.r * cir.r;
if (d2 < r2) return {};
if (d2 == r2) return { Point<DB>((DB)P.x, (DB)P.y) };
DB Lsq = (DB)d2 - (DB)r2;
DB L = sqrtl(max((DB)0, Lsq));
Circle<DB> orig((Point<DB>)cir.c, (DB)cir.r);
Circle<DB> aux((Point<DB>)P, L);
return getCrossPointsCC(orig, aux);
}

```

12.16 两圆公切线 getCommonTangentPoints

[vjjudge 提交 \(Aizu CGL_7_G\)](#)。函数返回的是公切线在 c_1 上的切点列表（每条公切线一个点；对应 c_2 上的切点由切线本身确定）。

推导： 设 $T_1 = c_1.c + r_1 \cdot \vec{n}$, $\vec{n}$ 是单位法向量。令 $\vec{u} = (c_2.c - c_1.c)/D$, 分外 / 内公切两类：

- 外公切：两边法向同向, $T_2 = c_2.c + r_2 \cdot \vec{n}$ 且 $(T_2 - T_1) \perp \vec{n}$, 得 $(c_2.c - c_1.c) \cdot \vec{n} = r_1 - r_2$, 故 $\cos \alpha = (r_1 - r_2)/D$;
- 内公切：两边法向反向, $T_2 = c_2.c - r_2 \cdot \vec{n}$, 得 $\cos \alpha = (r_1 + r_2)/D$ 。

然后 $\vec{n} = \cos \alpha \cdot \vec{u} + \sin \alpha \cdot \vec{u}_\perp$, $\sin \alpha = \pm \sqrt{1 - \cos^2 \alpha}$ 取两组。

退化： $|\cos \alpha| > 1$ 该类无切（内含 / 包含跳过）； $\sin \alpha = 0$ 该类只 1 条（内 / 外切临界）； $D = 0$ （同心，含重合）统一返空。最后 sort + unique 去重，应对内 / 外切临界 + $r = 0$ 退化下 4 个候选近重合的情形。

```

// 依赖:
// struct Point, struct Circle, eps
// Point::abs, Point::rot90
vector<Point<DB> > getCommonTangentPoints(Circle<ll> c1, Circle<ll> c2) {
    vector<Point<DB> > ret;
    Point<DB> d = c2.c - c1.c;
    DB D = d.abs();
    if (D < eps) return ret;
    Point<DB> u = d / D;
    Point<DB> u_perp = u.rot90();
    DB r1 = c1.r, r2 = c2.r;
    auto add = [&](DB cos_a) {
        if (cos_a > 1 + eps || cos_a < -1 - eps) return;
        DB s = sqrtl(max((DB)0, 1 - cos_a * cos_a));
        Point<DB> c1d = c1.c;
        if (s < eps) {
            Point<DB> n = u * cos_a;
            ret.push_back(c1d + n * r1);
        } else {
            ret.push_back(c1d + (u * cos_a + u_perp * s) * r1);
            ret.push_back(c1d + (u * cos_a - u_perp * s) * r1);
        }
    };
    add((r1 - r2) / D); // 外公切
    add((r1 + r2) / D); // 内公切
    sort(ret.begin(), ret.end());
    // 内/外切临界 + r=0 退化时 4 候选可能近重合, unique 去重
    ret.erase(unique(ret.begin(), ret.end()), ret.end());
    return ret;
}

```

12.17 两圆交集面积 two_circle_intersect_area

[vjjudge 提交 \(Aizu CGL_7_I\)](#)。

推导： 两圆相交时公共弦把每个圆切成两个弓形（segment / cap）。交集面积 = c_1 靠 c_2 那侧的弓形 + c_2 靠 c_1 那侧的弓形。圆心角由余弦定理给出：

$$\theta_1 = 2 \arccos \frac{D^2 + r_1^2 - r_2^2}{2Dr_1}, \quad \theta_2 = 2 \arccos \frac{D^2 + r_2^2 - r_1^2}{2Dr_2}.$$

单个弓形面积 = $\frac{1}{2}r^2(\theta - \sin\theta)$ 。

边界: $D \geq r_1 + r_2$ 不交 $\rightarrow 0$; $D \leq |r_1 - r_2|$ 一圆完全包含另一圆 $\rightarrow \pi \cdot r_{\min}^2$ (重合圆自然落入此档, 无需特判)。

```
// 依赖:
// struct Circle
// distancePP, sgn, PI
template<class T>
DB two_circle_intersect_area(Circle<T> c1, Circle<T> c2) {
    DB D = distancePP(c1.c, c2.c);
    DB r1 = c1.r, r2 = c2.r;
    if (sgn(D - r1 - r2) >= 0) return 0;
    DB diff = abs(r1 - r2);
    if (sgn(D - diff) <= 0) {
        DB rmin = min(r1, r2);
        return PI * rmin * rmin;
    }
    DB t1 = 2 * acosl((D * D + r1 * r1 - r2 * r2) / (2 * D * r1));
    DB t2 = 2 * acosl((D * D + r2 * r2 - r1 * r1) / (2 * D * r2));
    return 0.5L * r1 * r1 * (t1 - sinl(t1)) + 0.5L * r2 * r2 * (t2 - sinl(t2));
}
```

12.18 简单多边形与圆的交集面积 polygon_circle_intersect_area

[vjudge 提交 \(Aizu CGL 7_H\)](#)。

核心: 把多边形按边累加「三角形 (O, A_i, A_{i+1}) 与 $\text{disk}(O, r)$ 的有向面积」, O 是圆心。每条边的贡献由 $\text{tri_disk_signed_area}$ 计算: 先把 A, B 平移到以 O 为原点的坐标系; 然后按 A, B 是否在圆内分四种 case:

- 两端都在圆内: 整段就是真三角形, 有向面积 = $\frac{1}{2}\text{cross}(A, B)$;
- 两端都在圆外但弦穿圆得入 / 出两点 P_1, P_2 : 扇形 OAP_1 + 真三角形 OP_1P_2 + 扇形 OP_2B ;
- 两端都在圆外且不穿圆: 整段对应一个扇形 (从 $\vec{OA}$ 转到 $\vec{OB}$);
- A 内 B 外 (或反之): 在边上找穿圆点 P , OAP 是真三角形 + OPB 是扇形。

最后整圈累加取 abs , 输入多边形 CW / CCW 都可。**扇形角度**用 $\text{atan2}(\text{cross}, \text{dot})$ 算: 能拿到有向角度 $\in (-\pi, \pi]$, 正负自动包含弧的方向 (不像 acos 只给 $[0, \pi]$ 丢方向信息)。

退化: $\text{tri_disk_signed_area}$ 在 $A = B$ (多边形相邻顶点重合) 时直接返 0, 整体也吃得到。

```
// 依赖:
// struct Point, struct Circle
// cross (两参数), dot, Point::abs, sgn, sqrtl, atan2l
DB tri_disk_signed_area(Point<DB> A, Point<DB> B, DB r) {
    if (A == B) return 0; // 零长度边, 三角形退化, 有向面积为 0
    DB rA = A.abs(), rB = B.abs();
    bool aIn = sgn(rA - r) <= 0;
    bool bIn = sgn(rB - r) <= 0;
    auto ang = [](Point<DB> P, Point<DB> Q) {
        return atan2l((DB)cross(P, Q), (DB)dot(P, Q));
    };
    if (aIn && bIn) {
        // 整段都在圆内, 真三角形面积
        return 0.5L * (DB)cross(A, B);
    }
    // 求 A + t*(B-A) 落圆上的 t: |A + t d|^2 = r^2
    Point<DB> d = B - A;
    DB dd = (DB)dot(d, d);
    DB dA = (DB)dot(d, A);
    DB AA = (DB)dot(A, A);
    DB disc = dA * dA - dd * (AA - r * r);
    disc = max(disc, (DB)0);
    DB sq = sqrtl(disc);
    DB t1 = (-dA - sq) / dd, t2 = (-dA + sq) / dd;
```

```

if (!aIn && !bIn) {
    // 两端都在外, 看弦区间  $[t1, t2]$  与段  $[0, 1]$  是否相交
    if (sgn(t2) < 0 || sgn(t1 - 1) > 0 || sgn(disc) <= 0) {
        // 不穿圆, 整段对应一个扇形 (从  $OA$  转到  $OB$ )
        return 0.5L * r * r * ang(A, B);
    }
    t1 = max(t1, (DB)0);
    t2 = min(t2, (DB)1);
    Point<DB> P1 = A + d * t1;
    Point<DB> P2 = A + d * t2;
    return 0.5L * r * r * ang(A, P1) + 0.5L * (DB)cross(P1, P2)
        + 0.5L * r * r * ang(P2, B);
}
if (aIn && !bIn) {
    //  $A$  内  $B$  外, 段  $[0, 1]$  在  $[t1, t2]$  内的部分:  $t \geq 0$  (因为  $A$  内  $t1 < 0$ ), 取  $t2$ 
    Point<DB> P = A + d * t2;
    return 0.5L * (DB)cross(A, P) + 0.5L * r * r * ang(P, B);
}
// !aIn && bIn:  $A$  外  $B$  内,  $t1$  是入口
Point<DB> P = A + d * t1;
return 0.5L * r * r * ang(A, P) + 0.5L * (DB)cross(P, B);
}

// 简单多边形 (CCW) 与圆的交集面积. 累加每条边的 tri_disk_signed_area.
DB polygon_circle_intersect_area(Circle<DB> cir, const vector<Point<ll> > &poly) {
    DB res = 0;
    int n = poly.size();
    for (int i = 0; i < n; ++i) {
        Point<DB> A = poly[i] - cir.c;
        Point<DB> B = poly[(i + 1) % n] - cir.c;
        res += tri_disk_signed_area(A, B, cir.r);
    }
    return abs(res);
}

```

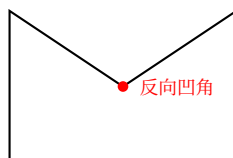
12.19 多边形凸性检验

允许的输入: **不允许自交**的多边形, 顺序可顺可逆, 函数会顺手把 poly 旋转 / 反向到「最左下点位 [0] + CCW 顺序」。

下面是三类不被认为是凸多边形的反例。

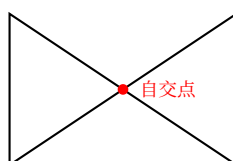
12.19.1 反例 1: 内角为反向凹角 (外角为锐角)

外角是锐角时 $\sin > 0$, 叉乘也 > 0 , 看似 CCW; 但其实形成的是**凹陷**。



图注: 标红顶点的内角 $> 180^\circ$ (反射角), isConvex 中第一遍 $\text{cross}(\text{poly}[i] - \text{poly}[0], \text{poly}[i+1] - \text{poly}[0]) \leq 0$ 触发返回 false。

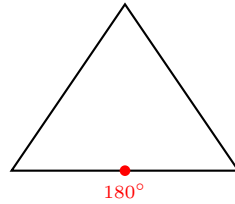
12.19.2 反例 2: 自交多边形 (蝴蝶结)



图注: 四个顶点排成蝴蝶结, 相对 $\text{poly}[0]$ 不再是单调 CCW 排列, 第一遍叉乘扫描即被排除。

12.19.3 反例 3：内角恰为 180°

三点共线（叉乘 = 0）也不允许，否则会让边数虚假增加。



图注：底边上多了一个 180° 顶点，第二遍 $\text{cross}(a, b) \leq 0$ 等号触发返回 false（严格大于零才算凸）。

实现：reorder_polygon 把多边形旋转 / 反向到「最左下点位 [0] + CCW 顺序」，再分两轮检查：(a) 所有顶点相对于 poly[0] 是 CCW 排列；(b) 每个内角的叉乘符号合规。

reorder_polygon 用整体 polygonSignedArea 判方向，**不要用**「前三点 cross」局部判向：含三点共线的多边形（如长方形 + 边密集共线）前三点 cross = 0，会被旧版 ≤ 0 误判为 CW 而强 reverse，下游 isConvex / 旋转卡壳全部失效（QOJ 784 实测）。

isConvex 加 strict 参数：

- strict = true（默认）：严格凸，拒绝三点共线 / 内角 = 180° ；
- strict = false：弱凸，允许内角 = 180° （题面用「凸」时常允许共线，要的就是这个口径）。

加宽接受范围有时是发现底层 bug 的好工具：本次就是因为加 strict = false 后 lax 模式 fail，才暴露出 reorder_polygon 的隐藏 bug——strict 模式因为 ≤ 0 提前 short-circuit 把它掩盖了。

```
// 依赖：
// struct Point
// sgn, cross (含三参数), polygonSignedArea
template<class T>
void reorder_polygon(vector<Point<T>> &poly) {
    ll n = poly.size();
    // 整体有向面积判向，避免「前三点共线」的局部退化 (QOJ 784 反例)
    if (sgn(polygonSignedArea(poly)) < 0) reverse(poly.begin(), poly.end());
    // 找到 y 最小、再 x 最小的点并循环左移到 [0]
    ll pos_mn = 0;
    for (int i = 1; i < n; ++i) if (poly[i].y < poly[pos_mn].y || (poly[i].y == poly[pos_mn].y &&
        poly[i].x < poly[pos_mn].x)) pos_mn = i;
    rotate(poly.begin(), poly.begin() + pos_mn, poly.end());
}

// 不允许自交的多边形，顺序都可以；会把 poly 变为 CCW 顺序
// AC https://qoj.ac/submission/2002121
// strict = true : 严格凸 (拒绝三点共线 / 内角 = 180°)
// strict = false: 弱凸 (允许三点共线 / 内角 = 180°), 题面允许共线时用
template<class T>
bool isConvex(vector<Point<T>> poly, bool strict = true) {
    ll n = poly.size();
    if (n <= 2) return false; // 0 / 1 / 2 点不是多边形
    reorder_polygon(poly);
    auto fail = [&](int s) { return strict ? s <= 0 : s < 0; };
    // 所有顶点相对 poly[0] 应按 CCW 整齐排列，否则是星形 / 乱序
    bool has_turn = false; // 至少一处真左转，排除全共线 (面积 = 0) 退化
    for (int i = 1; i < n - 1; ++i) {
        int s = sgn(cross(poly[0], poly[i], poly[i + 1]));
        if (fail(s)) return false;
        if (s > 0) has_turn = true;
    }
    // 弱凸下若全程 cross == 0 (全部点共线、面积 = 0) 也得拒；
    // strict 下 has_turn 在第一轮必为 true，此判等价空操作
    if (!has_turn) return false;
    // strict 时拒绝内角 = 180° (sgn = 0)，弱凸只拒绝凹 (sgn < 0)
    for (int i = 0; i < n; ++i) {
```

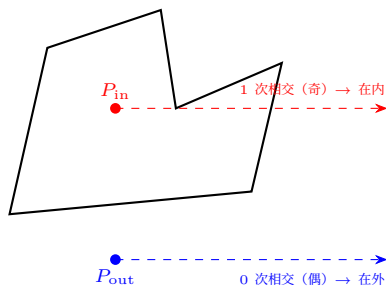
```

11 i1 = (i + 1) % n, i2 = (i + 2) % n;
    auto a = poly[i1] - poly[i], b = poly[i2] - poly[i1];
    if (fail(sgn(cross(a, b)))) return false;
}
return true;
}

```

12.20 点与多边形关系

多边形未必凸——「有向面积一定为正」的方法在凹多边形上失效，所以采用经典的射线奇偶法。



图注：从待判定点向任意方向打一条射线，统计与多边形边的相交次数。奇 = 在内，偶 = 在外。代码里射线方向用 `rng()` 随机，避开「射线恰好穿过顶点」的共线退化。

返回值约定：2 = 严格在内、1 = 点落在某条边上、0 = 严格在外。

```

// 依赖：
// struct Point, struct Line
// CCW(Point, Line), ON_SEGMENT
// isSegIntersect(Line, Line)
// mt19937_64 rng (基础模板已定义)
// 宏 SZ (用户自定义，等价于 (int)x.size())
template<class T>
int isInPolygon(Point<T> p, const vector<Point<T>> &poly) {
    ll res = 0;
    Line<T> l(p, p + Point<T>(2e18, rng()));
    for (int i = 0; i < poly.size(); ++i) {
        int i1 = (i + 1) % SZ(poly);
        Line<T> seg = Line<T>(poly[i], poly[i1]);
        int ccw = CCW(p, seg);
        if (ccw == ON_SEGMENT) return 1;
        if (isSegIntersect(l, seg)) ++res;
    }
    // 奇偶法则：射线相交奇数条边 = 在内
    return (res & 1) ? 2 : 0;
}

```

12.21 极角排序

按 `atan2(y, x)` 排序时，注意 `atan2` 浮点本身的精度问题。规避调用 `atan2` 的做法：先按「下半平面 / x 轴正向 / 上半平面」三档分桶，桶内再用叉乘符号判定相对方位。这样所有比较都用整数叉乘完成，没有浮点误差。

起点是 x 轴正半轴（含原点），逆时针环绕一周到 x 轴负半轴（不含）。

```

// 依赖：
// struct Point
// sgn, cross
template<class T>
int get_part_of_point(const Point<T> &p) {
    if (p.y < 0) return 0; // 下半平面 (-pi, 0)
    if (p.y == 0 && p.x >= 0) return 1; // 角度为 0
    return 2; // 上半平面 (0, pi]
}

```

```

// 按 atan2(y, x) 升序排，但用整数叉乘规避浮点
// AC https://qoj.ac/submission/2001485
template<class T>
void polar_angle_sort(vector<Point<T>> &poly) {
    sort(poly.begin(), poly.end(), [](const Point<T> &a, const Point<T> &b) -> bool {
        int part1 = get_part_of_point(a), part2 = get_part_of_point(b);
        if (part1 != part2) return part1 < part2;
        return sgn(cross(a, b)) == 1;
    });
}

// 重载：对有向直线按方向向量做极角排序（半平面交用）
// 同向（平行）的两条线，取「半平面内侧更靠右」的那条排前面——
// 等价判定：bl.p1 在 al 右侧（CLOCKWISE）时 al 在前
template<class T>
void polar_angle_sort(vector<Line<T>> &poly) {
    sort(poly.begin(), poly.end(), [](const Line<T> &a1, const Line<T> &b1) -> bool {
        auto a = a1.p2 - a1.p1, b = b1.p2 - b1.p1;
        int part1 = get_part_of_point(a), part2 = get_part_of_point(b);
        if (part1 != part2) return part1 < part2;
        int c = sgn(cross(a, b));
        if (c != 0) return c == 1;
        // 同向：bl 在 al 右侧，说明 al 的半平面把 bl 完全包住，al 更紧、应排前
        return CCW(b1.p1, a1) == CLOCKWISE;
    });
}

// polar_angle_sort 之后，把同向平行线去重，只保留每组最紧的那一条
// （配合上面的排序顺序，每组同向线的第一条即最紧）
template<class T>
void unique_line_polar_angle_sort(vector<Line<T>> &poly) {
    polar_angle_sort(poly);
    poly.resize(unique(all(poly), [](const Line<T> &a1, const Line<T> &b1) {
        auto a = a1.p2 - a1.p1, b = b1.p2 - b1.p1;
        return sgn(cross(a, b)) == 0;
    }) - poly.begin());
}

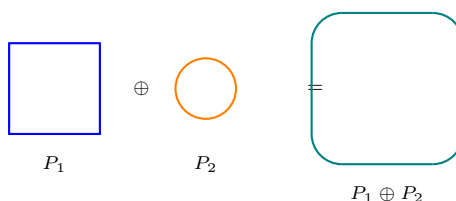
```

12.22 Minkowski 闵可夫斯基和

给定两个凸包 P_1 、 P_2 ，定义其闵可夫斯基和：

$$P_1 \oplus P_2 = \{a + b \mid a \in P_1, b \in P_2\}$$

结果仍是凸包。减法就等价于「加上对方关于原点的中心对称图形」，这是判断两凸包是否相交、求最近距离的常用技巧。



图注：正方形 + 圆 = 圆角矩形——四条直边长度等于原正方形的边长，四个角是半径等于圆半径的四分之一圆弧（这是几何上严格的 Minkowski 和）。

算法核心：先把两个凸包都 `reorder_polygon`（最左下点起、CCW 顺序），然后把它们的边向量按极角合并（类似归并排序的归并步）：每次比较两条候选边向量的叉乘符号，挑极角更小的那条接到答案末尾。起点设为 $P_1[0] + P_2[0]$ ，沿合并后的边向量逐条累加即得新凸包。

// 依赖：

```

// struct Point
// sgn, cross
// reorder_polygon (见多边形凸性检验一节)
// 参考 hh2048/XPC 的 mincowski AC https://qoj.ac/submission/2002545
template<class T>
vector<Point<T>> minkowski(vector<Point<T>> &P1, vector<Point<T>> &P2) {
    reorder_polygon(P1);
    reorder_polygon(P2);
    int n = P1.size(), m = P2.size();
    vector<Point<T>> V1(n), V2(m);
    for (int i = 0; i < n; i++) V1[i] = P1[(i + 1) % n] - P1[i];
    for (int i = 0; i < m; i++) V2[i] = P2[(i + 1) % m] - P2[i];
    vector<Point<T>> ans = {P1.front() + P2.front()};
    int i = 0, j = 0;
    while (i < n && j < m) {
        // cross(V1, V2) > 0 说明 V1 极角更小 (CCW 下), 先选小的
        Point<T> val = sgn(cross(V1[i], V2[j])) > 0 ? V1[i++] : V2[j++];
        ans.push_back(ans.back() + val);
    }
    while (i < n) ans.push_back(ans.back() + V1[i++]);
    while (j < m) ans.push_back(ans.back() + V2[j++]);
    return ans;
}

```

12.23 半平面交 (half_plane_cut)

把每条有向直线 $\text{Line}(p_1, p_2)$ 看作半平面——「沿 $p_1 \rightarrow p_2$ 方向走, 左手边」是合法区域。所有半平面的交集若非空则为一个凸多边形 (可能退化为无界、零面积等情形, 本模板返回 $\text{vector}<\text{Point}<\text{DB}>>$, 空表示交集为空 / 退化)。

算法核心: 双端队列扫描法。先 `unique_line_polar_angle_sort` 把直线按极角排序 + 同向去重, 然后逐条加入 deque q : 每次新直线 l 进来, 先用 l 反向清理队尾 (队尾两条线的交点若在 l 右侧, 说明该交点被 l 切掉, 对应的队尾线已经无贡献), 再清理队首 (对称地); 扫完一轮后还要做一次 **wrap-around**: 用队首 / 队尾互相再清一遍 (处理环形闭合时残留的失效线段)。

复杂度: $O(N \log N)$ (瓶颈是极角排序, 扫描本身均摊 $O(N)$)。

常见用法:

- **多个半平面求公共凸区域:** 每个不等式 $a_i x + b_i y \leq c_i$ 表示一个半平面, 加入对应的有向直线即可。
- **多边形并的总面积 (旋转切割型):** 经典题 [Uywu's Concert](#) (多次直线切凸多边形求剩余面积) / [洛谷 P4196 \[CQOI2006\] 凸多边形](#) (多个凸多边形求交)。
- **有界化:** 当输入只有半平面、没有边界 `box` 时, 可手动加 4 条「大正方形边界」直线 (如 $[-1e9, 1e9]$ 见方), 避免交集无界。

```

// 依赖:
// struct Point, struct Line
// CCW, CLOCKWISE
// getintersect, polygonArea
// unique_line_polar_angle_sort
// AC https://vjudge.net/solution/69843663 (Uywu's Concert)
template<class T>
vector<Point<DB>> half_plane_cut(vector<Line<T>> lines) {
    unique_line_polar_angle_sort(lines);
    deque<Line<T>> q;
    for (auto l : lines) {
        if (SZ(q) < 2) { q.push_back(l); continue; }
        // 清队尾: 队尾两条线交点若在 l 右侧 (被 l 切掉), 队尾线作废
        auto checkBack = [&]() {
            auto p = getintersect(q.rbegin()[0], q.rbegin()[1]);
            return CCW(p, l) != CLOCKWISE;
        };
        while (SZ(q) >= 2 && !checkBack()) q.pop_back();
    }
}

```



```

// 对称清队首
auto checkFront = [&]() {
    auto p = getintersect(q.begin()[0], q.begin()[1]);
    return CCW(p, 1) != CLOCKWISE;
};
while (SZ(q) >= 2 && !checkFront()) q.pop_front();
q.push_back(1);
}
// wrap-around: 用队首线再清一遍队尾
while (SZ(q) >= 3) {
    auto p = getintersect(q.rbegin()[0], q.rbegin()[1]);
    if (CCW(p, q.front()) != CLOCKWISE) break;
    q.pop_back();
}
// 对称: 用队尾线再清一遍队首
while (SZ(q) >= 3) {
    auto p = getintersect(q.begin()[0], q.begin()[1]);
    if (CCW(p, q.back()) != CLOCKWISE) break;
    q.pop_front();
}
if (SZ(q) < 3) return {};
vector<Point<DB>> poly;
for (int i = 0; i < SZ(q); ++i) {
    poly.push_back(getintersect(q[i], q[(i + 1) % SZ(q)]));
}
return poly;
}

```

使用范例 (Uyuw's Concert 框架: 给定 N 条有向直线 + 题目自带的 $[0, 10000]^2$ 边界, 求公共半平面交的面积):

```

ll N; cin >> N;
vector<Line<DB>> lines(N);
for (auto &l : lines) cin >> l;
// 手动补 4 条边界直线 (CCW 顺序: 左下 → 右下 → 右上 → 左上)
auto a1 = Point<DB>(0, 0), a2 = Point<DB>(10000, 0);
auto a3 = Point<DB>(10000, 10000), a4 = Point<DB>(0, 10000);
lines.push_back(Line<DB>(a1, a2));
lines.push_back(Line<DB>(a2, a3));
lines.push_back(Line<DB>(a3, a4));
lines.push_back(Line<DB>(a4, a1));
auto poly = half_plane_cut(lines);
cout << fsp(1) << polygonArea(poly) << "\n";

```

12.24 凸包 (Andrew 算法)

按 x 升序、 x 相同按 y 升序排序后, 从左到右扫一遍维护下凸包, 再从右到左扫一遍维护上凸包, 把两段拼起来即可。

strict 参数 决定共线点处理: strict = true 时 cross == 0 (共线) 也弹栈, **不保留共线点**; strict = false 时只在 cross < 0 (凹陷) 才弹, **保留共线点** (题目要求「凸包上所有点」时用)。

HullMode 参数: 三种返回都是 CCW 方向。HULL_FULL 完整凸包 (按 Y 排序, 从最低点 (y 最小、再 x 最小) 起 CCW 绕一圈; 这样剥层第一步就拿到最低点起步, 不必再调 reorder_polygon 找最低点); HULL_LOWER 下凸壳 (最左 → 最右); HULL_UPPER 上凸壳 (最右 → 最左, 从已建好的全凸包数组里切片, 不重跑单调栈)。易踩: HULL_UPPER 等价「直线上半包络」时调用前需 dedupe 同斜率 (strict 处理不了「同 x 不同 y 竖边」退化), 见 STL 节 sort + unique 模板。

```

// 依赖:
// struct Point
// cross(Point, Point) 与三参数 cross(Point, Point, Point)
enum HullMode { HULL_FULL, HULL_LOWER, HULL_UPPER };

```



```

// strict = true: 严格凸包 (不保留三点共线的中间点)
// strict = false: 弱凸包 (保留三点共线的中间点), 题目要求"凸包上所有点"时用
template<class T>
vector<Point<T>> make_convex_hull(vector<Point<T>> A, bool strict, HullMode hull_mode) {
    // Andrew 扫描. 各 mode 的排序键与返回 (顶点序列都是 CCW 序):
    // HULL_FULL: 按 Y 排序 (Y 同按 X), 返回整圈凸包, 从 (y 最小, 再 x 最小) 点起头.
    // HULL_LOWER: 按 X 排序 (X 同按 Y), 返回下凸链, 最左 -> 最右 (x 单调不减).
    // HULL_UPPER: 按 X 排序 (X 同按 Y), 返回上凸链, 最右 -> 最左 (x 单调不减).
    if (hull_mode == HULL_FULL) {
        sort(A.begin(), A.end(), [](const Point<T> &a, const Point<T> &b) {
            return a.y != b.y ? a.y < b.y : a.x < b.x;
        });
    } else {
        sort(A.begin(), A.end());
    }
    A.resize(unique(all(A)) - A.begin());
    size_t n = A.size();
    if (n <= 2) {
        // 退化: 不足 3 点直接返回排序后的点; UPPER 逆序成 x 不减, FULL/LOWER 保持排序序
        if (hull_mode == HULL_UPPER) reverse(A.begin(), A.end());
        return A;
    }
    vector<Point<T>> ans(n * 2);
    // strict 时 cross == 0 共线也弹; 弱凸只在 cross < 0 凹陷时弹
    auto fail = [&](ll s) { return strict ? s >= 0 : s > 0; };
    int now = -1;
    for (int i = 0; i < n; i++) {
        // 维护下凸包
        while (now > 0 && fail(sgn(cross(A[i], ans[now], ans[now - 1])))) {
            now--; // 弹出栈顶, 因为它会导致凹陷 (strict 时还会弹掉共线)
        }
        ans[++now] = A[i];
    }
    if (hull_mode == HULL_LOWER) {
        ans.resize(now + 1);
        return ans;
    }
    int pre = now;
    // 从右向左扫描所有点 (注意是从 n-2 开始, 因为 n-1 已经在栈顶了)
    for (int i = n - 2; i >= 0; i--) {
        // 维护上凸包
        while (now > pre && fail(sgn(cross(A[i], ans[now], ans[now - 1])))) {
            now--;
        }
        ans[++now] = A[i];
    }
    if (hull_mode == HULL_UPPER) {
        return vector<Point<T>>(ans.begin() + pre, ans.begin() + now + 1);
    }
    ans.resize(now);
    // HULL_FULL: 因上面按 Y 排序, 这里已天然「Y 最小点起头 + CCW」.
    // 退化处理: 输入点全共线时凸包退化成线段, 弱凸包会折返成重复序列 (如 A B C B);
    // 有向面积为 0 即退化, 此时 A 已按 Y 排好序且点互异, 就是沿线单程, 直接返回 (O(n), 不折返).
    if (sgn(polygonSignedArea(ans)) == 0 && !strict) {
        return A;
    }
    return ans;
}

```

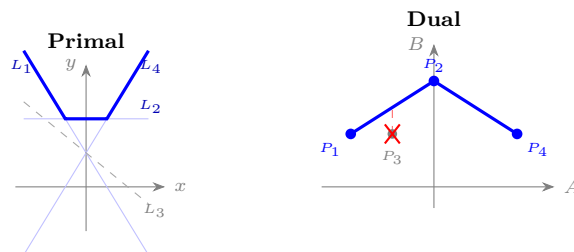
12.24.1 对偶点技巧: 直线上半包络 $\Leftrightarrow$ 对偶点上凸壳

洛谷 P3194 [HNOI2008] 水平可见直线: 给 N 条直线 $y = A_i x + B_i$, 从 $y = +\infty$ 俯视, 问哪些「有某段 x 露头」。

直线-点对偶：把 L_i 映射到对偶平面的点 $P_i = (A_i, B_i)$ ，则

L_i 可见 $\iff P_i$ 在所有对偶点的「严格上凸壳」上。

为什么：3 条 $a_1 < a_2 < a_3$ ， L_2 被 L_1, L_3 在某 x 处压下 $\iff \text{cross}(P_2 - P_1, P_3 - P_1) \geq 0$ (P_2 在 P_1P_3 下方或共线)。共线时三线共点、 L_2 只在一点露头不是 segment，strict 模式正好弹掉。



图注：左 Primal 4 条线，上半包络 = $L_1 \rightarrow L_2 \rightarrow L_4$ (粗蓝折线)， L_3 全程在下被压 (灰虚线)。右 Dual 4 个对偶点， $P_3 = (-1, 1)$ 严格落在弦 P_1P_2 下方 (红虚示意垂直差 0.5)，上凸壳 $P_1 \rightarrow P_2 \rightarrow P_4$ 不收 P_3 。这就是「 L_i 可见 $\iff P_i$ 在上凸壳上」的视觉对照。

落地：(1) 把 (A_i, B_i) 当点 (套 `Point<ll>` 的 `id` 字段绑原始下标)；(2) 同 A 多条只留 B 最大那条 (sort + unique 模板见 STL 节)；(3) `make_convex_hull(pts, true, HULL_UPPER)` 直接返回上凸壳点列，按 `id` 升序输出。 $|A|, |B| \leq 5 \times 10^5$ 时全程整数，`cross` 走 `__int128`。

12.25 凸多边形直径 (旋转卡壳)

QOJ 784 (凸多边形直径模板题)。

给一个 **逆时针顺序** 的 n 凸多边形 (允许三点共线)，求最远点对的距离。**旋转卡壳** 的标准技巧：把每条边 $A[r] \rightarrow A[r+1]$ 当成一条「卡尺」，对踵点 $A[l]$ 是离这条边最远的顶点。 r 沿 CCW 扫一圈时 l 单调 (不回退)，整体 $O(n)$ 。

关键判据： l 该不该往前推一步？看「以边 $(r, r+1)$ 为底、 $l+1$ vs l 为顶」的两个三角形面积——后者 $\geq$ 前者就推。**比直接比距离平方更鲁棒**：曲率非均匀的凸包 (如 spike 形 $r(\theta) = 1 + 0.1 \sin(7\theta)$) 下 $|A[l] - A[r]|^2$ 沿 r 不是单峰会卡死，但「以边为底的三角形面积」仍是单峰。

4 个不要踩的坑 (QOJ 784 session 实录，每个都有真实反例)：

1. **不要做点对踵**：以 $|A[l] - A[r]|^2$ 沿 r 找单峰只在「曲率均匀」凸包成立，cyaron 默认凸包形是盲区——必死于 spike 形；
2. l, r 角色不能反：必须「外层 r 是边、内层 l 是点」，反过来 `|edge|` 乘子会破坏单峰；
3. **ans 候选取两个**： $(A[l], A[r])$ 和 $(A[l], A[r+1])$ 都要更新，只取一个会漏掉「边端点本身就是最远对」的情形；
4. **推进用 $\leq$ 不是 $<$** ：plateau (两个候选三角形面积相等) 起点要继续推到 plateau 末端，严格 $<$ 会卡在起点。

对拍提醒：几何题对拍**必须用曲率非均匀的形状**(spike / heart / lemon / 椭圆)，cyaron `Polygon.convex_hull` 默认是近圆形，stress 5000+ case 全过会让你错误推断「算法对」。详见 cyaron-stress skill。

```
// 依赖：
// struct Point, sqrtL (基础模板已定义)
// cross (含三参数), isConvex (LOCAL 校验用，弱凸口径)
// 宏 SZ, all (用户自定义)

// 距离平方 (不开方)：旋转卡壳里 ans 反复更新，留到最后一次 sqrt
template<class T, class U>
auto distancePPNorm(const Point<T> &a, const Point<U> &b) {
    return (a - b).norm();
}

// L2 欧几里得距离 (sqrt 版，用 long double). 整型坐标走 isqrt + fraction 不丢精度.
template<class T, class U>
DB distancePP(const Point<T> &a, const Point<U> &b) {
    return (a - b).abs();
}
```

```

// L1 曼哈顿距离 = |dx| + |dy|. 返回类型继承 normL1 (整型 -> __int128_t, 浮点 -> T).
template<class T, class U>
auto distancePPL1(const Point<T> &a, const Point<U> &b) {
    return (a - b).normL1();
}

// Linf 切比雪夫距离 = max(|dx|, |dy|). 返回类型 T.
template<class T, class U>
T distancePPLinf(const Point<T> &a, const Point<U> &b) {
    return (a - b).normLinf();
}

// 以 CCW 顺序给定 n 点凸多边形 (允许三点共线), 求直径 (最远点对距离)
template<class T>
long double convex_diameter(const vector<Point<T>> &A) {
#ifdef LOCAL
    vector<Point<T>> B(all(A));
    assert(isConvex(B, false)); // 弱凸即可, 允许三点共线
#endif
    // 退化输入特判: n <= 2 时 area() 恒 0 会让 need_l_move 永真死循环
    if (SZ(A) <= 1) return 0;
    if (SZ(A) == 2) return sqrtL(distancePPNorm(A[0], A[1]));
    auto ans = (A[0] - A[0]).norm(); // 零值, 类型由 norm 继承
    // 以边 (r, r+1) 为底, A[l] 为顶的三角形 2 倍面积
    auto area = [&](ll l, ll r) {
        auto res = cross(A[r], A[(r + 1) % SZ(A)], A[l]);
        if (res < 0) res = -res;
        return res;
    };
    // l 是否还要向前推? 比较 (l, r) vs (l+1, r) 三角形面积
    // 用 <= 推到 plateau 末端, 严格 < 会卡在 plateau 起点
    auto need_l_move = [&](ll l, ll r) -> bool {
        return area(l, r) <= area((l + 1) % SZ(A), r);
    };
    for (int l = 0, r = 0; r < SZ(A); ++r) {
        while (need_l_move(l, r)) l = (l + 1) % SZ(A);
        ans = max(ans, distancePPNorm(A[l], A[r]));
        ans = max(ans, distancePPNorm(A[l], A[(r + 1) % SZ(A)])); // 第二候选
    }
    return sqrtL(ans);
}

```

12.26 最远点对 (旋转卡壳, 返回原始下标)

[Luogu P16223](#)。

convex_diameter 只返回距离; 要把最远点对的原始输入下标也输出回来, 靠 Point::id 字段: 读入时填 A[i].id = i, make_convex_hull 不做派生运算 (仅 sort + 拷贝原点入栈), 凸包顶点继承原始 id。

```

// 依赖:
// struct Point (带 id 字段)
// cross (三参数), distancePPNorm, make_convex_hull
// 宏 SZ (用户自定义)
template<class T>
pair<int, int> furthest_pair_of_point(vector<Point<T>> A) {
    A = make_convex_hull(A, true, HULL_FULL);
    // 退化短路: hull == 1 全重合; hull == 2 全共线 (area=0 死循环)
    if (SZ(A) == 1) return {A[0].id, A[0].id};
    if (SZ(A) == 2) return {A[0].id, A[1].id};
    i128 ans = 0;
    auto area = [&](ll l, ll r) {
        auto res = cross(A[r], A[(r + 1) % SZ(A)], A[l]);
        if (res < 0) res = -res;
    };
}

```

```

    return res;
};
auto need_l_move = [&](ll l, ll r) -> bool {
    return area(l, r) <= area((l + 1) % SZ(A), r);
};
pair<int, int> ans_pair;
for (int l = 0, r = 0; r < SZ(A); ++r) {
    while (need_l_move(l, r)) l = (l + 1) % SZ(A);
    if (distancePPNorm(A[l], A[r]) > ans) {
        ans = distancePPNorm(A[l], A[r]);
        ans_pair = {A[l].id, A[r].id};
    }
    int rn = (r + 1) % SZ(A);
    if (distancePPNorm(A[l], A[rn]) > ans) {
        ans = distancePPNorm(A[l], A[rn]);
        ans_pair = {A[l].id, A[rn].id};
    }
}
return ans_pair;
}

```

调用样板 (读入时给 id):

```

ll N; cin >> N;
vector<Point<ll>> A(N);
for (int i = 0; i < N; ++i) {
    cin >> A[i];
    A[i].id = i;
}
auto [l, r] = furthest_pair_of_point(A);
cout << l << " " << r << '\n';

```

12.27 最小矩形覆盖 (旋转卡壳, 4 指针)

Luogu P3187 [HNOI2007] 最小矩形覆盖。

给一个点集, 求面积最小、把所有点框住的矩形 (不必轴对齐)。**关键定理**: 最小覆盖矩形必有一条边与凸包某条边重合。所以枚举每条凸包边 e 当矩形底边, 三指针单调推进维护:

- up: 以 e 为底时离 e 最远的顶点 (决定矩形高 h);
- r / l : 在 e 方向上投影最大 / 最小的顶点 (决定宽 w 的右 / 左端)。

外层 edge 沿 CCW 扫一圈, 内层 up, r, l 各自单调 (不回退), 整体 $O(n)$ 。

比较时不要早除: 实现里用 $w = \text{dot}(e, A[r] - A[l])$ (长度乘子 $|e|$) 和 $h = \text{cross}(e, A[\text{up}] - A[\text{edge}])$ (同样乘子 $|e|$), 所以 $\text{area} = w * h$ 是真实矩形面积的 $|e|^2$ 倍。比较两条不同底边 e 给出的候选时, 正确做法是交叉相乘:

$$\text{area} \cdot |e_{\text{best}}|^2 \quad \text{vs.} \quad \text{mn_area} \cdot |e|^2$$

这样比较时只走整数 ($T = DB$ 下也是平方量级), 避免提前除 $|e|^2$ 丢精度。**还原 4 顶点**: 底 / 顶切线方向 $\parallel e$, 左 / 右切线方向 $\perp e$, 4 条切线两两相交即得。函数签名硬钉 `Point<DB>` (getintersect 走浮点除法 + 还原 4 顶点必然是非整坐标, 没有保留整数 T 的意义; 类型层一刀切防”误传 `vector<Point<ll>>`”沉默截断)。

```

// 依赖:
//   struct Point, struct Line
//   cross (含三参数), dot, getintersect
//   make_convex_hull(strict = true), Point::norm()
//   宏 SZ (用户自定义)
// 最小矩形覆盖: 返回 {面积, 4 个矩形顶点 (以底边 BL 起、CCW) }
// 前置: T = DB (reconstruct 走 getintersect 必须浮点)
pair<DB, array<Point<DB>, 4>> minimum_rectangle_cover(vector<Point<DB> > A) {
    A = make_convex_hull(A, true, HULL_FULL);
    int n = SZ(A);
    auto e_of = [&](int i) { return A[(i + 1) % n] - A[i]; }; // 第 i 条凸包边的方向向量

```

```

int up = 0, l = 0, r = 0;
using Type = decltype(dot(A[0], A[0]));
Type mn_area = 0;
do {
    auto e0 = e_of(0);
    for (int i = 0; i < SZ(A); ++i) {
        // 同一个底, 面积越大, 高自然越大
        if (cross(e0, A[i] - A[0]) > cross(e0, A[up] - A[0])) {
            up = i;
        }
        if (dot(e0, A[i] - A[0]) > dot(e0, A[r] - A[0])) {
            r = i;
        }
        if (dot(e0, A[i] - A[0]) < dot(e0, A[l] - A[0])) {
            l = i;
        }
    }
    auto w = dot(e0, A[r] - A[l]);
    auto h = cross(e0, A[up] - A[0]);
    mn_area = w * h;
} while (false);
int bestL = l, bestR = r, bestE = 0, bestUp = up;
for (int edge = 0; edge < SZ(A); ++edge) {
    Point<DB> e = e_of(edge);
    while (cross(e, A[(up + 1) % SZ(A)] - A[edge]) > cross(e, A[up] - A[edge]))
        up = (up + 1) % SZ(A);
    while (dot(e, A[(r + 1) % SZ(A)] - A[edge]) > dot(e, A[r] - A[edge]))
        r = (r + 1) % SZ(A);
    while (dot(e, A[(l + 1) % SZ(A)] - A[edge]) < dot(e, A[l] - A[edge]))
        l = (l + 1) % SZ(A);
    // 这个宽和高均乘以了 e 的长度
    auto w = dot(e, A[r] - A[l]);
    auto h = cross(e, A[up] - A[edge]);
    auto area = w * h;
    if (area * e_of(bestE).norm() < mn_area * e.norm()) {
        mn_area = area;
        // mn_area_edge = e.norm();
        tie(bestL, bestR, bestE, bestUp) = make_tuple(l, r, edge, up);
    }
}
// 还原 4 顶点 = 4 条切线两两相交 (4 条线方向: 底/顶 parallel e; 左/右 parallel e_perp)
Point e = e_of(bestE);
Point e_perp(-e.y, e.x); // CCW 90° 旋转
Line bottom(A[bestE], A[bestE] + e);
Line top_(A[bestUp], A[bestUp] + e);
Line right_(A[bestR], A[bestR] + e_perp);
Line left_(A[bestL], A[bestL] + e_perp);
array<Point<DB>, 4> corners = {
    getintersect(bottom, left_), // BL
    getintersect(bottom, right_), // BR
    getintersect(top_, right_), // TR
    getintersect(top_, left_), // TL
};
return {(long double) mn_area / e.norm(), corners};
}

```

12.28 平面最近点对

经典分治：把所有点按 x 排序后，递归求左右两半的最小距离 d ，再处理跨越中线、与中线 x 距离 $< \sqrt{d}$ 的「窄带」候选点。

难点在于把窄带内的候选点收集起来后再按 y 暴力扫一遍——技巧是从中线向两侧分别推进，一旦 $|\Delta x| \geq \sqrt{d}$ 立刻 break，避免把窄带退化成 $O(n)$ 。

```

// 依赖:
// struct Point (基础模板, 本节用 Point<ll>)
// dot(Point, Point)
// 宏 SZ, FOR, ROF, EB (用户自定义)
// 全局数组 Point<ll> A[MAXN]
Point<ll> A[MAXN];
struct Best {
    i128 d; // dot<ll>(v, v) 返回 i128, 得收得下, 否则 1e9 坐标平方就溢出 ll
    Point<ll> a, b;
    bool operator<(const Best &o) const { return d < o.d; }
};

// 暴力求平面最近点对 (按 y 排序后扫)
Best mindis(vector<Point<ll>> &a, Best res) {
    sort(all(a), [&](Point<ll> a, Point<ll> b) { return a.y < b.y; });
    FOR(i, 1, SZ(a) - 1) {
        ROF(j, i - 1, 0) {
            Point<ll> p = a[i], q = a[j];
            // y 距离平方已超出当前最优就早停
            if ((p.y - q.y) * (p.y - q.y) > res.d) break;
            Point<ll> v = p - q;
            res = min(res, {dot(v, v), p, q});
        }
    }
    return res;
}

Best dfs(int l, int r) {
    if (l >= r) return {(i128) INF, A[l], A[l]};
    if (r - l == 1) {
        Point<ll> v = A[r] - A[l];
        return {dot(v, v), A[l], A[r]};
    }
    int mid = r + l >> 1;
    Best ret = dfs(l, mid);
    ret = min(ret, dfs(mid + 1, r));
    ll x = A[mid + 1].x;
    vector<Point<ll>> wp;
    // 左半向左推进, 超出 sqrt(ret.d) 就停
    ROF(i, mid, l) {
        i128 d = abs(A[i].x - x);
        if (d * d < ret.d) wp.EB(A[i]);
        else break;
    }
    // 右半向右推进
    FOR(i, mid + 1, r) {
        i128 d = abs(A[i].x - x);
        if (d * d < ret.d) wp.EB(A[i]);
        else break;
    }
    return mindis(wp, ret);
}

```

12.29 两凸多边形最近距离 (旋转卡壳)

OpenJ_Bailian 3851 Bridge Across Islands。给两个不相交的 CCW 凸多边形 A, B , 求两边界最小距离。

初始 e_A 取 A 最低点、 e_B 取 B 最高点 (一对反向支撑线起步); 按 $\text{cross}(e_A, e_B) > 0$ 单调推进 e_B 、否则推进 e_A , 合计 $\leq n + m$ 步。每步取段-段距离 distanceSS (A 边长且 B 边整段在 A 边投影内时单纯点-边距离会漏, 必须段-段)。

必须双向各调一次: 单次 $\text{cal_min_dis_two_convex}(A, B)$ 的对偶覆盖不完整, 要 (A, B) 和 (B, A) 各跑一次取 $\min$ 才覆盖全。漏一次会在「 A 顶点 $\rightarrow B$ 边」和「 B 顶点 $\rightarrow A$ 边」非对称分布下系统性偏大 (实测反

例存在)。

```
// AC https://vjudge.net/solution/69793877
// 依赖:
// struct Point, struct Line
// reorder_polygon, cross, distanceSS
// 宏 SZ, all (用户自定义)
template<class T>
DB cal_min_dis_two_convex(vector<Point<T> > A, vector<Point<T> > B) {
    reorder_polygon(A);
    reorder_polygon(B);
    ll yMnA = min_element(all(A), [](const Point<T> &a, const Point<T> &b) {
        return a.y < b.y;
    }) - A.begin(), yMxB = max_element(all(B), [](const Point<T> &a, const Point<T> &b) {
        return a.y < b.y;
    }) - B.begin();
    DB ans = INFINITY;
    ll eA = yMnA, eB = yMxB;
    for (int _ = 0; _ < SZ(A) + SZ(B); ++_) {
        auto check = [&]() {
            auto vecA = A[(eA + 1) % SZ(A)] - A[eA];
            auto vecB = B[(eB + 1) % SZ(B)] - B[eB];
            return cross(vecA, vecB) > 0;
        };
        while (check()) ++eB, eB %= SZ(B);
        ans = min(ans, distanceSS(Line(A[(eA + 1) % SZ(A)], A[eA]),
            Line(B[(eB + 1) % SZ(B)], B[eB])));
        eA += 1;
        eA %= SZ(A);
    }
    return ans;
}

// 调用方: 必须双向各跑一次取 min
// DB ans = min(cal_min_dis_two_convex(A, B), cal_min_dis_two_convex(B, A));
```

12.30 常用公式

12.30.1 正多边形面积公式

设边数为 n 、边长为 l :

$$A = \frac{n \cdot l^2}{4 \cdot \tan\left(\frac{\pi}{n}\right)}$$

12.30.2 三角恒等式

和差角:

$$\begin{aligned}\sin(\alpha \pm \beta) &= \sin \alpha \cos \beta \pm \cos \alpha \sin \beta \\ \cos(\alpha \pm \beta) &= \cos \alpha \cos \beta \mp \sin \alpha \sin \beta \\ \tan(\alpha \pm \beta) &= \frac{\tan \alpha \pm \tan \beta}{1 \mp \tan \alpha \tan \beta}\end{aligned}$$

倍角:

$$\begin{aligned}\sin 2\theta &= 2 \sin \theta \cos \theta, & \cos 2\theta &= \cos^2 \theta - \sin^2 \theta = 1 - 2 \sin^2 \theta = 2 \cos^2 \theta - 1 \\ \tan 2\theta &= \frac{2 \tan \theta}{1 - \tan^2 \theta}\end{aligned}$$

降幂 (把 θ 整体升到 2θ):

$$\sin^2 \theta = \frac{1 - \cos 2\theta}{2}, \quad \cos^2 \theta = \frac{1 + \cos 2\theta}{2}, \quad \sin \theta \cos \theta = \frac{\sin 2\theta}{2}$$

辅助角 (线性叠加 $\rightarrow$ 单一正弦):

$$a \sin \theta + b \cos \theta = \sqrt{a^2 + b^2} \sin(\theta + \varphi), \quad \tan \varphi = \frac{b}{a}$$

极值即 $\pm\sqrt{a^2 + b^2}$, 把单变量旋转优化变成闭式。

积化和差:

$$\begin{aligned}\sin \alpha \cos \beta &= \frac{1}{2} [\sin(\alpha + \beta) + \sin(\alpha - \beta)] \\ \cos \alpha \cos \beta &= \frac{1}{2} [\cos(\alpha + \beta) + \cos(\alpha - \beta)] \\ \sin \alpha \sin \beta &= \frac{1}{2} [\cos(\alpha - \beta) - \cos(\alpha + \beta)]\end{aligned}$$

和差化积:

$$\begin{aligned}\sin \alpha + \sin \beta &= 2 \sin \frac{\alpha + \beta}{2} \cos \frac{\alpha - \beta}{2} \\ \sin \alpha - \sin \beta &= 2 \cos \frac{\alpha + \beta}{2} \sin \frac{\alpha - \beta}{2} \\ \cos \alpha + \cos \beta &= 2 \cos \frac{\alpha + \beta}{2} \cos \frac{\alpha - \beta}{2} \\ \cos \alpha - \cos \beta &= -2 \sin \frac{\alpha + \beta}{2} \sin \frac{\alpha - \beta}{2}\end{aligned}$$

半角:

$$\sin \frac{\theta}{2} = \pm \sqrt{\frac{1 - \cos \theta}{2}}, \quad \cos \frac{\theta}{2} = \pm \sqrt{\frac{1 + \cos \theta}{2}}, \quad \tan \frac{\theta}{2} = \frac{\sin \theta}{1 + \cos \theta} = \frac{1 - \cos \theta}{\sin \theta}$$

12.31 附录：C++14 版本基础模板

部分老评测机只开到 C++14 (POJ / HDU / UOJ 早期等), if constexpr、变量模板别名 is_xxx_v、inline constexpr 变量都不能用。下面给出与正文「点线基础模板」语义等价的 C++14 重写, 赛场上换评测机直接整段替换基础模板即可, 后续 make_convex_hull / minkowski / 半平面交 / Circle 等子节代码**无需改动**。

四点改造原则:(1) std::is_xxx_v 是 C++17 才有的别名, 手写一份放**全局命名空间**(不污染 std)——C++14 的 std 里没有这层别名, 所以 using namespace std; 之后下游代码可以原封不动写 is_floating_point_v<T>。(2) sqrtL / squared / cross / dot 等本来靠 if constexpr 在函数体内做整型/浮点分流的函数模板, 全部拆成两个独立模板, 用 enable_if_t<...> 写互补条件覆盖所有 T 组合。(3) 成员函数 Point::norm / Point::normL1 不能直接用 class 的 T 做 SFINAE (class 实例化时就会 hard error), 借 template<class TT = T, enable_if_t<...>> 把 SFINAE 推迟到成员函数自身的模板实例化阶段。(4) is_promotion_v 去掉 inline——C++14 已有变量模板, 但变量不能 inline (单 TU 内无 ODR 冲突, 多 TU 才需要 C++17 inline)。

```
// === C++14 版本基础模板 (POJ / HDU / UOJ 等老评测机) ===
// 1) std::xxx_v 是 C++17 才有的别名, 这里放全局命名空间手写一份;
//    C++14 std 里没这层别名, 与 using namespace std 不冲突
template<class T> constexpr bool is_floating_point_v = std::is_floating_point<T>::value;
template<class T> constexpr bool is_arithmetic_v      = std::is_arithmetic<T>::value;
template<class T> constexpr bool is_integral_v       = std::is_integral<T>::value;
template<class F, class T> constexpr bool is_same_v   = std::is_same<F, T>::value;
```

// 2) isqrt: 不依赖 C++17, 与正文版完全一致

```
i128 isqrt(i128 n) {
    if (n <= 0) return 0;
    i128 lo = 0, hi = (i128) 2e18;
    while (lo < hi) {
        i128 mid = lo + (hi - lo + 1) / 2;
        if (mid <= n / mid) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}
```

// 3) sqrtL: if constexpr 拆两个独立 SFINAE 模板, 互补条件覆盖所有 T

```
template<typename T, enable_if_t<is_floating_point_v<T>, int> = 0>
long double sqrtL(T n) { return sqrtL((long double) n); }
```

```
template<typename T, enable_if_t<!is_floating_point_v<T>, int> = 0>
```

```
long double sqrtL(T n) {
    i128 ni = (i128) n;
    if (ni <= 0) return 0;
    i128 i = isqrt(ni);
    long double id = (long double) i;
    long double rd = (long double) (ni - i * i);
    return id * sqrtL(1.0L + rd / (id * id));
}
```

// 4) squared: 同 sqrtL 拆法

```
template<class T, enable_if_t<is_floating_point_v<T>, int> = 0>
auto squared(T x) { return x * x; }
```

```
template<class T, enable_if_t<!is_floating_point_v<T>, int> = 0>
```

```
i128 squared(T x) { return (i128) x * (i128) x; }
```

// sgn: 不依赖 C++17, 照搬

```
template<class T>
int sgn(T x) {
    if (-eps < x && x < eps) return 0;
    return x < 0 ? -1 : 1;
}
```

// 5) Point: 成员函数 norm / normL1 借 TT = T 默认模板参数,

// 把 SFINAE 推迟到成员函数实例化, 避免 class 实例化时直接 hard error

```

// converting ctor / 标量乘除走 is_same_v<common_type_t<U,T>, T> 方向限制
template<class T>
struct Point {
    T x, y;
    int id = -1;

    explicit Point(T x_ = 0, T y_ = 0) : x(x_), y(y_) {}

    // Point<U> -> Point<T> 隐式 (common_type 单调升档方向, U -> T 必须无损):
    // common_type_t<U, T> == T 才放行 (例 ll -> DB 通; DB -> ll 拦, 防 truncation)
    template<class U, class = enable_if_t<is_same_v<common_type_t<U, T>, T>>>
    Point(const Point<U> &o) : x((T) o.x), y((T) o.y), id(o.id) {}

    Point &operator+=(Point p) & { x += p.x, y += p.y; return *this; }
    Point &operator-=(Point p) & { x -= p.x, y -= p.y; return *this; }
    Point &operator*=(T t) & { x *= t, y *= t; return *this; }
    Point &operator/=(T t) & { x /= t, y /= t; return *this; }

    Point operator-() const { return Point(-x, -y); }

    friend Point operator+(Point a, Point b) { return a += b; }
    friend Point operator-(Point a, Point b) { return a -= b; }

    // 标量乘除: 返回 Point<common_type_t<T, U>>; R==T 走 copy ctor, R!=T 走 converting ctor
    template<class U>
    friend auto operator*(Point a, U b) {
        Point<common_type_t<T, U>> r = a; r *= b; return r;
    }
    template<class U>
    friend auto operator*(U a, Point b) {
        Point<common_type_t<T, U>> r = b; r *= a; return r;
    }
    template<class U>
    friend auto operator/(Point a, U b) {
        Point<common_type_t<T, U>> r = a; r /= b; return r;
    }

    friend bool operator<(Point a, Point b) {
        int sx = sgn(a.x - b.x);
        if (sx != 0) return a.x < b.x;
        return a.y < b.y;
    }
    friend bool operator>(Point a, Point b) { return b < a; }
    friend bool operator==(Point a, Point b) {
        return sgn(a.x - b.x) == 0 && sgn(a.y - b.y) == 0;
    }
    friend bool operator!=(Point a, Point b) { return !(a == b); }
    friend istream &operator>>(istream &is, Point &p) { return is >> p.x >> p.y; }

    // 关键: TT = T 让 SFINAE 推迟到成员函数实例化阶段
    template<class TT = T, enable_if_t<!is_floating_point_v<TT>, int> = 0>
    auto norm() const { return (__int128_t) x * x + (__int128_t) y * y; }
    template<class TT = T, enable_if_t<is_floating_point_v<TT>, int> = 0>
    auto norm() const { return x * x + y * y; }

    DB abs() const { return sqrtL(norm()); }

    Point rot90() const { return Point(-y, x); }
    Point<DB> rot(DB ang) const {
        return Point<DB>(cosl(ang) * x - sinl(ang) * y,
                        sinl(ang) * x + cosl(ang) * y);
    }
}

```

```

template<class TT = T, enable_if_t<!is_floating_point_v<TT>, int> = 0>
auto normL1() const {
    auto ax = (x < 0 ? -x : x), ay = (y < 0 ? -y : y);
    return (__int128_t) ax + (__int128_t) ay;
}
template<class TT = T, enable_if_t<is_floating_point_v<TT>, int> = 0>
auto normL1() const {
    auto ax = (x < 0 ? -x : x), ay = (y < 0 ? -y : y);
    return ax + ay;
}

T normLinf() const {
    auto ax = (x < 0 ? -x : x), ay = (y < 0 ? -y : y);
    return ax > ay ? ax : ay;
}
};

// 7) cross (两参 / 三参): 两参拆两个 SFINAE 模板; 三参不需分流照搬
template<class T, enable_if_t<!is_floating_point_v<T>, int> = 0>
__int128_t cross(Point<T> a, Point<T> b) {
    return (__int128_t) a.x * b.y - (__int128_t) a.y * b.x;
}
template<class T, enable_if_t<is_floating_point_v<T>, int> = 0>
auto cross(Point<T> a, Point<T> b) {
    return a.x * b.y - a.y * b.x;
}

template<class T>
auto cross(Point<T> o, Point<T> a, Point<T> b) { return cross(a - o, b - o); }

// 8) dot: 与 cross 同构, 拆两个 SFINAE 模板 (T / U 可不同型)
template<class T, class U,
    enable_if_t<!is_floating_point_v<T> && !is_floating_point_v<U>, int> = 0>
__int128_t dot(Point<T> a, Point<U> b) {
    return (__int128_t) a.x * b.x + (__int128_t) a.y * b.y;
}
template<class T, class U,
    enable_if_t<is_floating_point_v<T> || is_floating_point_v<U>, int> = 0>
auto dot(Point<T> a, Point<U> b) {
    return a.x * b.x + a.y * b.y;
}

// 9) Line / gen_line_from_general / rng: 不依赖 C++17, 照搬正文「点线基础模板」即可
// (此处省略, 参见 § 计算几何/点线基础模板)

```

13 STL 及其使用

13.1 string::find 的起始位置参数与 string_view

find 可以指定「从哪个下标开始找」,但不能直接指定「查找到哪个下标结束」。

```
string s = "abacaba";
string t = "aba";

size_t pos = s.find(t, 2); // 从下标 2 开始找
if (pos == string::npos) { // string::npos 就是
    ↪ -1
    // 没找到
}
```

如果要限制在区间 $[l, r]$ 内查找,可以先从 l 开始找,再判断匹配是否完整落在区间内(`std::string_view` 是在 C++ 17 引入的)。

```
string_view sv = s;
auto sub = sv.substr(l, r - l + 1); // 只看 [l,
    ↪ r]
size_t p = sub.find(t);
if (p != string::npos) {
    size_t real_pos = l + p; // 映射回原串下标
}
```

13.2 mt19937_64 套 splitmix64 抗 BM 攻击

```
std::mt19937_64 rng(std::chrono::steady_clock::
    ↪ now().time_since_epoch().count());

// 把 rng() 输出再过一层非线性混合,挡住 F2 上的
    ↪ Berlekamp-Massey 攻击
uint64_t splitmix64(uint64_t x) {
    x ^= x >> 30; x *= 0xbf58476d1ce4e5b9ULL;
    x ^= x >> 27; x *= 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

uint64_t rnd() { return splitmix64(rng()); } //
    ↪ XOR hashing 等场景用这个,别用裸 rng()
```

攻击模型出处: *Hacking Incorrect XOR Hashing with mt19937_64* (Sugar_fan, CF blog)。mt19937_64 的 XOR、移位、稀疏 $\mathbb{F}_2$ 矩阵乘全部在 $\mathbb{F}_2$ 上线性, Berlekamp-Massey 拿前 2×19937 个输出位即可复盘最小多项式、预测后续输出,构造 XOR 多重集 hash 碰撞。换种子、加 seed_seq、混 random_device 全无效——线性结构和种子无关。

splitmix64 包装够不够: 本地 Apple M5 + GCC 15.2 上的 benchmark (gist) 实证——裸 mt19937_64 上 BM 在 5 个 bit lane 一致复盘出 degree = 19937、命中率 100%; 套上 splitmix64 后 BM 度数饱和到训练长度的一半、命中率锁在 50% (bit 要么 0, 要么 1, 基本就是闭眼瞎猜), 攻击失效。

13.3 unordered_map 自定义 hash & 结构体做 key

为何要自定义: GCC 整数默认 hash 是恒等 (`hash<ll>(x) == x`)、桶数又走内部写死的质数序列,二者都可离线预测。攻击者构造大量落同一桶的 key, 让查询退化 $O(n)$ 、整体 $O(n^2)$ TLE。防御 = 每次运行播一个 static 随机盐、混进 key 再过 splitmix64 打散 (复用上一节的 splitmix64):

```
struct custom_hash {
    size_t operator()(const ull &x) const {
        static ull R = rnd(); // 随机盐, rnd()
        ↪ 见上一节 mt19937
        return splitmix64(x ^ R); // 盐 + avalanche
    }
};
// 用 alias 一键注入, 声明处跟原生一样
template <class K, class V>
using hash_map = unordered_map<K, V,
    ↪ custom_hash>;
hash_map<ll, int> mp;
```

Me (zzy)

如果要存的东西里面有负数, 会造成问题吗?

Mentor (Claude)

不会, 负数 key 完全安全。关键在于 custom_hash 的形参是 `const ull &` (ull 即 64 位无符号整数): 往 `hash_map<ll, int>` 里塞一个负的 ll (如 -5) 时, 会先做一次 **signed** $\rightarrow$ **unsigned** 隐式转换再进哈希。C++ 标准规定这个转换是良定义的——结果 = 原值对 2^{64} 取模, 即 $-1 \rightarrow 2^{64} - 1$ 、 $-5 \rightarrow 2^{64} - 5$ 。

对 64 位的 ll, 这个转换是一个**双射** (2^{64} 个有符号值一一对应 2^{64} 个位模式, 无信息丢失), 于是:

- 相等的 key 转出相等的 ull $\Rightarrow$ 哈希一致, 一定查得到;
- 不同的 key 转出不同的 ull $\Rightarrow$ 负数不会跟某个正数「假撞」, 不会平白增加冲突。

后续 $x \wedge R$ 只是拿随机盐做按位异或 (连溢出都不涉及), 再交给 splitmix64 打散——它的设计就是吃满 64 位输入。再兜一层底: 即便两个 key 真落进同一个桶, unordered_map 还会用 `std::equal_to<ll>` 去比真实的 ll 值, -5 和 5 照样分得清。

唯一真会出事的不是「负数」, 而是「key 宽于 64 位」: 如果拿 `__int128` 当 key, 形参 ull 会截断高 64 位, 两个只差在高位的 key 才可能假撞。ll / int 的负数不涉及截断, 安全。

自定义结构体做 key 需要两样东西: (1) 一个 hash 把每个字段逐个混入 (禁止直接异或, (a,b) 与 (b,a)

会撞成同值); (2) operator== 供哈希冲突时比较。两个 operator 都要加 const, 否则 unordered_map 对 const key 调用时编译不过:

```
struct Point {
    ll x, y;
    bool operator==(const Point &o) const { // +
        ↪ const
        return x == o.x && y == o.y;
    }
};

struct PointHash {
    size_t operator()(const Point &p) const { // +
        ↪ const
        static const uint64_t R =
            chrono::steady_clock::now().time_since_epoch()
            ↪ ock().count();
        uint64_t h = splitmix64(p.x + R);
        return splitmix64(p.y + h); // 逐段混,
        ↪ 顺序敏感
    }
};

unordered_map<Point, int, PointHash> mp;
```

变长序列 (vector<ll>) 做 key: 思路一样, 只是字段数不定——用一个初值为随机盐 R 的 h, 把每个元素依次 fold 进去 (h ~ x 再过 splitmix64)。这样天然顺序敏感 + 长度敏感, 不会像裸异或那样让 {a,b} 和 {b,a} 撞成同值:

```
struct VecHash {
    size_t operator()(const vector<ll> &v) const {
        static const uint64_t R = rnd(); // 随机盐,
        ↪ rnd() 见上一节 mt19937
        uint64_t h = R;
        for (auto x : v) {
            h = splitmix64(h ^ (uint64_t)x); // 逐元素
            ↪ fold, 顺序/长度敏感
        }
        return h;
    }
};

unordered_set<vector<ll>, VecHash> st; // vector
↪ 直接做 key
```

坑: mp.reserve(n) / max_load_factor(0.25) 只是把桶数顶大 / 保持稀疏, 桶数仍然可预测——是弱防御 + 常数优化, 不能单独防 hack, 真正的防线永远是随机盐。追求速度可换 __gnu_pbds::gp_hash_table<K, V, custom_hash> (开放寻址, 常数更小), custom_hash 用法相同。

13.4 views::keys / views::values 遍历 map 的键 / 值 (C++20 ranges)

C++20 的 <ranges> 提供 views::keys / views::values, 把 map (或任何 pair 序列) 投影成「只有 key」/「只有 value」的惰性视图——省掉 for (auto &[k, v] : mp) 里那个用不到、却还得写的另一半。视图元素是原容器的引用, auto & 拿到就能原地改 (如对每个 value 排序):

```
#include <ranges> // 需要
namespace views = std::views; // 或 using
↪ namespace std::views;

unordered_map<ll, vector<array<ll, 2>>> mp;
// 只遍历 value: 免写 auto &[_, vec], 直接拿
↪ value 的引用, 能原地改
for (auto &vec : mp | views::values)
    ↪ sort(all(vec));
// 只遍历 key: 关联容器 key 是 const, 这里拿到
↪ const 引用
for (ll k : mp | views::keys) cout << k << " ";
```

同族常用视图 (都能用 | 串接) : views::filter(pred) 过滤、views::transform(f) 逐元素映射、views::reverse 反序、views::take(n) / views::drop(n) 取前 / 弃前 n 个。视图惰性求值、不拷贝底层数据, 遍历到才逐个算。

13.5 sort + unique 按 key 去重并保留指定值

std::unique 把相邻相等元素合并为一个, 保留每段的第一个 (不是最后)。配合 sort 可以做「按某个 key 分组、每组只留一个特定属性最大 / 最小的元素」。

标准套路: (1) 主 key 升序排 (保证同 key 元素挤在一起); (2) 次 key 反向排, 让“想留的”挤到每组最前面; (3) unique 用主 key 比较器去重, 留下每组第一个。

例 (洛谷 P3194 上半包络题 dedupe 步骤): 直线 $y = A_i x + B_i$, 同斜率 A 多条只留 B 最大那条 (其余被同斜率平行压死, 对偶点全部出现在上凸壳「同 x 不同 y 竖边」上 strict 也救不了)。

```
sort(poly.begin(), poly.end(), [](const
    ↪ Point<ll> &a, const Point<ll> &b) {
    if (a.x != b.x) return a.x < b.x; // 主 key
    ↪ 升序
    return a.y > b.y; // 次 key
    ↪ 降序: 想留的 (B 大) 在前
});
poly.erase(unique(poly.begin(), poly.end(),
    ↪ [](const Point<ll> &a, const Point<ll> &b) {
    return a.x == b.x; //
    ↪ 比较器只看主 key, 不看次 key
    }), poly.end());
```

坑: 次 key 写反成 a.y < b.y (升序), unique 留下的是每组 B 最小那条——和题意完全相反, 是高频低级错误。记忆口诀:「unique 留前 → 想留谁就 sort 把谁放最前」。

13.6 lower_bound / upper_bound 自定义比较器参数顺序相反

先厘清概念: lower_bound / upper_bound 的第三个参数是待查找的值 (拿来和元素的排序键比较的目标值), 不是下标 / 位置; 返回的迭代器才是「位置」。即输入

一个值，输出一个位置。下例数组 es 按 `node::val` 升序排好，`target` 是要找的目标值，`e.val` 是元素的排序键字段（也是个值，不是下标）。

带自定义比较器时，`lower_bound` 与 `upper_bound` 的比较器两个参数谁前谁是相反的——标准强制，记反轻则二分边界静默错、重则编译不过：

- `lower_bound(first, last, target, comp)` 内部按 `comp(*it, target)` 调用——数组元素在前，目标值在后，返回首个满足 `e.val ≥ target` 的位置。
- `upper_bound(first, last, target, comp)` 内部按 `comp(target, *it)` 调用——目标值在前，数组元素在后，返回首个满足 `e.val > target` 的位置。

场景 A ——找第一个 `e.val ≥ target` 的元素，用 `lower_bound`：

```
auto it = lower_bound(es.begin(), es.end(),
    ↪ target,
    [](const node &e, int t) { // 元素 node
        ↪ 在前，目标值 int 在后
        return e.val < t;
    });
```

场景 B ——找第一个 `e.val > target` 的元素，用 `upper_bound`：

```
auto it = upper_bound(es.begin(), es.end(),
    ↪ target,
    [](int t, const node &e) { // 目标值 int
        ↪ 在前，元素 node 在后
        return t < e.val;
    });
```

坑（编译错 vs 静默逻辑错）：比较器喂错函数的后果取决于元素类型与目标值类型是否同构。本例元素是 `node`、目标值是 `int`，二者异构且不可互转：把场景 B 的 `(int, node)` 比较器误传给 `lower_bound`，它要按 `comp(node, int)` 调，参数对不上又无 `node/int` 转换，直接编译失败。但若元素与目标值同类型（如拿 `node` 二分 `node`），顺序写反能编译通过，却是静默错的二分边界——更难查。

记忆口诀：「lower 找 ≥，元素在前；upper 找 >，目标在前」。

13.7 __int128 读入输出（重载运算符版）

`iostream` 原生不支持 `__int128`。重载 `<</>>` 后，`cin / cout` 就能像内建整型一样直接读写（也自动覆盖 `Point<i128>` 等模板类里转发的 `>> / <<<`）：

```
ostream &operator<<(ostream &os, __int128 x) {
    if (x < 0) os << '-', x = -x;
    if (x > 9) os << x / 10;
    return os << (char) ('0' + x % 10);
}
```

```
istream &operator>>(istream &is, __int128 &x) {
    string s;
    if (!(is >> s)) return is;
    x = 0;
    size_t i = 0;
    bool neg = false;
    if (s[0] == '-') {
        neg = true;
        i = 1;
    } else if (s[0] == '+') { i = 1; }
    for (; i < s.size(); ++i) x = x * 10 + (s[i]
        ↪ - '0');
    if (neg) x = -x;
    return is;
}
```

输出走递归除 10（深度 ≤ 39 层，无爆栈风险）；读入按字符串整体读进来再逐位累加，支持前导 - / +。

读入开头 `if (!(is >> s)) return is;` 的用意：让 `while (cin >> v)` 「多组输入读到 EOF 自动停」的写法对 `__int128` 同样成立。`is >> s` 返回流本身，流可以转 `bool`（语义是「上一次读取成功与否」）：EOF 时读不到内容、流被置上失败位、转出 `false`，这行就直接带着失败位原样返回且不动 `x`——与内建整型 `cin >> n` 读失败的行为对齐，外层循环条件拿到 `false` 正确退出。

坑：输出对负数先 `x = -x`，恰为 -2^{127} （`__int128` 最小值）时取负溢出是 UB——竞赛值域顶到 $\pm 1.7 \times 10^{38}$ 边界才会踩，正常使用不受影响。

13.8 next_permutation 枚举全排列

`next_permutation(first, last)` 把区间 `[first, last)` 原地重排成字典序的下一个排列并返回 `true`；当前已是最大排列（完全降序）时返回 `false`，并把区间重排回最小排列（完全升序）。标准套路——先 `sort` 到最小排列，再 `do-while`：

```
vector<int> p = {3, 1, 2};
sort(p.begin(), p.end()); // 必须从最小排列出发，
    ↪ 否则枚举不全
do {
    // 处理当前排列 p (do-while
    ↪ 保证初始排列本身也被处理)
} while (next_permutation(p.begin(), p.end()));
```

- 不 `sort` 就 `do-while` 是高频错误：只会从当前排列往字典序后面枚举，前面的排列全部漏掉。
- 复杂度：单次调用均摊 $O(1)$ （最坏 $O(n)$ ），枚举全部共 $n!$ 个—— $n = 10$ 约 3.6×10^6 随便跑， $n = 11$ 约 4×10^7 是边界， $n \geq 12$ 基本不可行。
- 有重复元素时自动去重：按「不同的排列」恰好各枚举一次（如 `{1, 1, 2}` 只出 3 种），这是它比手写 DFS 全排列省心的地方。
- 枚举组合的技巧：开一个 $n - k$ 个 0 + k 个 1 的 01 数组，`sort` 后用同一套 `do-while` 枚举它的全部排列，就是 $\binom{n}{k}$ 个组合的指示向量。

- 对称版本 `prev_permutation`: 字典序上一个, 完全升序时返回 `false`; `string` / 数组 / `vector` 都能用 (任何双向迭代器区间)。

14 常见问题与错误

14.1 -O2 下的 FMA 乘加融合踩崩浮点 ceil

2026 HDU 春季联赛 6「绩点查询」一题，坑的是一个很不起眼的浮点细节：同一份代码在 -O0 (CLion Debug 默认) 下跑样例全对，提交到判题机却 WA。最小复现如下：

```
ll s1 = 36, s2 = 96;
ll s = ceil(0.6L * s1 + 0.4L * s2);
// 数学真值: 3*36 + 2*96 = 300, 除 5 恰好 60, ceil
//   应为 60
// 但 -O2 下, s = 61
```

罪魁祸首是编译器在 -O2 默认启用的 FMA(fused multiply-add) 乘加融合。ARM64 上对应指令 fmadd, x86 上对应 vfmadd。把表达式 $a * b + c * d$ 合成一条带单次舍入的融合指令后，原本两次乘法各自产生的舍入误差无法互相抵消。

具体数值：0.6 * 36 真值 21.6，double 误差约 $-2.13e-15$ ；0.4 * 96 真值 38.4，double 误差约 $+5.68e-15$ 。分别计算再相加，两个反号误差近似抵消，落在精确的 60.0。一旦用 fmadd 融合，0.6 * 36 不再做中间舍入，加法也只做一次舍入，结果变成 60.0000000000000071，ceil 直接顶到 61。

实测不同优化等级下的行为：

编译参数	$0.6L \cdot 36 + 0.4L \cdot 96$	ceil
-O0 / -O1	60.0 精确	60 (对)
-O2 / -O3	60.0000000000000071	61 (错)
-O2 -ffp-contract=off	60.0 精确	60 (对)

教训：

1. **ceil(浮点表达式) 永远不要信**。只要表达式里有 $a * x + b * y$ 这种结构，-O2 的 FMA 合并就可能让结果比真值多出约 ϵ 的误差，刚好顶过整数边界。
2. **本地过、判题机挂**的经典场景：CLion / VSCode 的 Debug 构建是 -O0，不会触发 FMA；判题机普遍用 -O2。本地样例永远测不出来。
3. **long double 不是救命稻草**。Apple 平台 (macOS ARM64 / Intel) 的 long double ABI 就是 64-bit double；MinGW-w64 的 long double 宽度由发行版配置决定，不能指望一定是 80-bit。更要命的是 FMA 合并跟宽度无关，80-bit long double 也一样会被合并。

三种应对手段，优先级从高到低：

1. **首选——化成整数运算**。系数是有理数时，先通分成精确整数：

```
// 有理系数 0.6, 0.4 -> 3, 2, 分母 5
ll s = (3 * s1 + 2 * s2 + 4) / 5; // 整数
//   ceil
```

一行、零误差、对编译选项免疫。

2. **不得不用浮点时——关掉 FMA 合并**。用 #pragma 或 volatile 阻断合并：

```
// 函数级关 FMA (GCC)
__attribute__((optimize("fp-contract=off")))
-> ))
ll calc(ll s1, ll s2) {
    return ceil(0.6L * s1 + 0.4L * s2);
}

// 或者用 volatile 分步，强制内存来回，
//   编译器无法融合
volatile double a = 0.6 * s1;
volatile double b = 0.4 * s2;
ll s = (ll)ceil(a + b);
```

3. **真的需要更高精度时——上 __float128** (GCC / Clang 扩展，128-bit IEEE 754 quad precision, 精度 $\sim 10^{-34}$)：

```
// 注意：表达式里不要出现 double 字面量 0.6,
//   -> 0.4,
// 那会先以 double 存储 (带误差)，升到
//   float128 也救不回来。
// 所有系数都要用整数 / 有理构造：
__float128 v = (__float128)3 * s1 +
-> (__float128)2 * s2;
v /= 5;
ll s = (ll)v;
if ((__float128)s < v) ++s; // 手写 ceil
```

额外限制：0.6Q 这种 quad 字面量后缀在 -std=c++20 下默认被禁用，要加 -fext-numeric-literals 才认，而判题机场景下你无法指定编译参数。__float128 也没有 std::ostream 重载，不能直接 cout <<，只能把结果转回整数或 long double 再输出。

所以 __float128 能用的“安全姿势”几乎等价于整数法，反而更啰嗦。除非中间必须保留非整数有理值（比如小数系数来自输入），才值得上。

14.2 __float128 使用要点

- **类型本身**：GCC / Clang 扩展，x86_64 与 ARM64 均支持。sizeof(__float128) = 16，尾数 112-bit， $\epsilon \approx 2^{-112} \approx 1.9 \times 10^{-34}$ 。
- **字面量后缀 Q**：需要 -fext-numeric-literals 编译参数，在严格 -std=c++20 / c++17 下被禁用。
- **头文件 <quadmath.h> 与 -lquadmath**：ceilq、sqrtq、quadmath_snprintf 等函数依赖这个库，判题机不一定链接。若只做基础算术 (+ - * / 与比较)，不需要任何额外库，就能直接用。

- 没有流 I/O: `cin >> __float128` 和 `cout << __float128` 都不存在, 必须手动转成 `double / long double` / 整数再输出。
- 陷阱: `(__float128)0.6` 这种写法不是在 `__float128` 里存 0.6——0.6 先以 `double` 字面量存储(带 $5.5\text{e-}17$ 的误差), 再升到 `__float128`, 误差直接继承。想保证精度要么用 `0.6Q` (需要上面那个 flag), 要么干脆把系数换成整数比例构造。

14.3 DP 不可达状态必须用 INF/-INF 显式标识

线性 DP 形如 $f[r+1] = \max\{f[l] + 1 : (l, r) \text{ 合法转移}\}$ 时, 不可达的前缀 (即 $a[0..l-1]$ 没有合法划分的状态) 必须用 `-INF` 标记, 不能图省事用 `vector<ll> dp(N + 1)` (默认初值 0)。

反例: [CF 1094C Median Partition](#) 写成

```
vector<ll> dp(N + 1);    // 全 0, BUG!
dp[0] = 0;
for (int r = 0; r < N; ++r)
    for (int l = 0; l <= r; ++l)
        if (checkLr(l, r))
            dp[r + 1] = max(dp[r + 1], 1 +
                               ↪ dp[l]);
```

会让”不可达的 $a[0..l-1]$ ”被误读成”0 段已成功”, 从而 $dp[r+1] = 1 + 0 = 1$ ——这等价于凭空跳过前面无法划分的部分、把 $a[l..r]$ 当作整个数组的合法第一段。最大段数被高估。

正解:

```
vector<ll> dp(N + 1, -INF); // 不可达 = -INF
dp[0] = 0;                  // 空前缀是唯一
                             ↪ base
// 转移时 dp[l] = -INF 自动让 1 + dp[l] 落败
```

记忆口诀: base 之外一律 `-INF` (求 `max`) / `INF` (求 `min`)。0 只能用于”求计数 / 默认无贡献”语义, 不能用于”段数 / 长度”语义——后者 0 是合法答案值, 与”不可达”撞车。

14.4 大网格 DFS 遇到障碍要跳过, 不要继续标记递归

2026 陕西省赛 M *Group Effect* 一题, $n \cdot m = 4 \times 10^6$ 。如果为了实现方便, 遇到障碍不跳过而是继续标记已访问递归, 会爆栈。这里 RE 不是代码写错了, 是 C++ 爆栈了。

15 debug 技巧

15.1 除了打印之外，善用 assert

我们在头文件中已经定义了 `dbg` 宏，用来在本地把变量、容器打印到 `cerr`。但打印并不是调试的唯一手段，**配合 `assert` 把关键的「不变量」写成断言**，往往能更快定位错误。

`assert` 的好处是：一旦条件不成立，程序会立刻崩溃在出错的那一行，而不是让错误悄悄蔓延到很久之后再体现为 WA 或段错误。这对于 DP、图论、状压之类有很多前置约束的算法尤其有用，能帮我们把「这里应该满足某个条件」这种口头约定，变成程序能自检的硬约束。

下面是一段 Color Coding 的代码（随机染色 + 状压 DP，求包含 K 个不同颜色的最短路），我们同时使用了 `dbg` 打印和 `assert` 断言。

```
void execColorCoding() {
    for (int i = 1; i <= N; ++i) {
        dp[i][1 << color[i]] = {0, A[i]};
        timeId[i][1 << color[i]] = curT;
    }
    for (int sta = 1; sta < (1 << K); ++sta) {
        for (int u = 1; u <= N; ++u) {
            if (!check(u, sta)) continue;
            for (auto [to, w] : g[u]) {
                if (sta & (1 << color[to])) continue;
                ll toSta = sta | (1 << color[to]);
                if (!check(to, toSta)) {
                    dp[to][toSta] = {INF, -1};
                    timeId[to][toSta] = curT;
                }
            }
            #ifdef LOCAL
                assert(check(u, sta));
                assert(check(to, toSta));
            #endif
            // 省略与 debug 无关的正常转移
        }
    }
    dbg(dp);
    dbg(timeId);
    cEnd;
}
```

这道题目是一个**随机化算法**，每次跑出来的结果大概率是错的，**常规的这个 `dbg` 打印其实效果不是很好**。因为我们不知道预期输出是什么。在这种情况下，我们就不得不采用这个 **`assert` 来进行 debug**。

这道题目当中，我们使用这个 `assert` 来判断来源状态，即**即将进行转移的这个状态是否已经被初始化过了**。

15.1.1 增量 debug

如果一份代码原本是对的，而你后来**只新加了两样东西**，随后程序就挂了，那么一个很自然的想法就是做**增量 debug**。

具体来说，不要一上来就在新代码里到处打印，而是先把其中**一处改动暂时注释掉**，只保留另一处，然后重新运行：

- 如果问题消失了，说明刚刚注释掉的那部分很可疑。
- 如果问题还在，说明更可能是剩下那部分导致的。

这样就能把「两个新增改动共同成锅」的局面，快速缩小成「大概率是哪一个改动出了问题」。如果还有更多新增内容，也可以继续按这个思路一段一段地回退，逐步缩小可疑范围。

这个方法本质上是在做**控制变量**。很多时候，真正高效的 debug 不是打印更多信息，而是一次**只改一个东西**，让错误来源自己暴露出来。

15.2 `std::string` 条件断点的陷阱

在调试器（GDB、LLDB、CLion 等）里设置**条件断点**时，如果条件写成：

```
ops == "?00?00?00"
```

你会发现断点**根本不按预期停**——它可能在条件完全不成立时就触发，退化成了无条件断点。

原因：调试器的表达式求值器在求值条件时，需要在运行时调用 `std::string::operator==`。但这个运算符是一个重载函数，调试器往往**无法可靠地解析并调用它**，导致比较结果始终为真（或行为未定义），断点每次都停。这一问题在 JetBrains、VS Code、GDB 等主流工具的 issue tracker 上均有记录（另见 [gist 整理](#)）。

正确写法：改用成员函数 `compare`，调试器能正确调用它：

```
ops.compare("?00?00?00") == 0
```

或者用 C 风格的 `strcmp`（部分调试器更容易解析 C 函数符号）：

```
strcmp(ops.c_str(), "?00?00?00") == 0
```

16 环境测试

16.1 试机赛与纪律

- 检查身份证件：护照、学生证、胸牌以及现场所需通行证。
- 确认什么东西能带进场。特别注意：智能手表、金属（钥匙）等等。
- 测试鼠标、键盘、显示器和座椅。如果有问题，立刻联系工作人员。
- 测试比赛提交方式。如果有 submit 命令，确认如何使用。
- 设置自动保存、自动备份。

16.2 编译器版本与扩展头

测试编译器版本。C++20 cin >> (s + 1); C++17 auto [x, y] : a; C++14 [](auto x, auto y); C++11 auto; bits/stdc++.h; pb_ds。

```
#include <ext/rope>
using namespace __gnu_cxx;
rope<int> R; R.insert(y, x); R[x]; R.erase(x,
↪ 1);

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> s;
s.insert(1); s.find_by_order(0);
↪ s.order_of_key(5);
```

16.3 __int128, __float128, long double 实测

long double 不是跨平台稳定类型：在 x86-64 Linux + GCC (CF / AtCoder / QOJ 评测机) 上是 80-bit x87 扩展精度 (尾数 64 bits、约 18-19 位有效、 $LDBL_EPSILON \approx 1.08 \times 10^{-19}$)，但 Apple arm64 / MSVC 上 long double == double (尾数 53 bits、约 15 位、 $\epsilon \approx 2.22 \times 10^{-16}$)。本机过 ≠ 评测机过。

```
#include <bits/stdc++.h>
// pb_ds / rope 是否可用：能编过即 OK
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/rope>

// long double 是不是真扩展精度 (CE = 退化成
↪ double 或编译器异常)
static_assert(sizeof(long double) >
↪ sizeof(double), "ld == double");
static_assert(LDBL_MANT_DIG >= 64, "ld mantissa
↪ < 64 bits");

int main() {
    std::cout << "sizeof(long double)=" <<
↪ sizeof(long double) << '\n';
```

```
std::cout << "LDBL_MANT_DIG=" <<
↪ LDBL_MANT_DIG << '\n';
// QOJ x86-64 Linux 实测 sizeof=16 (10B
↪ 80-bit 数据 + 6B 对齐 padding), mant=64
// 相对精度 LDBL_EPSILON ~ 2-63 ~ 1.08e-19
↪ (vs double 2.22e-16, 多 ~3 位有效数字)
// __int128 是否可用 (不可用直接 CE)。assert
↪ 必须在函数体内, namespace scope 不合法
__int128 x = (__int128)1 << 100; assert(x >
↪ 0);
// __float128 是否可用 (GCC 扩展, MSVC 无;
↪ 不可用 CE)
// 1.0Q 字面量需 -fext-numeric-literals 才认;
↪ 用 cast 跨编译选项更稳
__float128 y = (__float128)1; assert(y ==
↪ 1);
}
```

注意 __int128 的 cin/cout 不支持，需手写 read/print；__float128 是软件模拟，慢一档。重载 operator<< / operator>> 后 cin >> x / cout << x 即可直接吃 __int128：

```
ostream &operator<<(ostream &os, __int128 x) {
    if (x < 0) os << '-', x = -x;
    if (x > 9) os << x / 10;
    return os << (char) ('0' + x % 10);
}

istream &operator>>(istream &is, __int128 &x) {
    string s;
    if (!(is >> s)) return is;
    x = 0;
    size_t i = 0;
    bool neg = false;
    if (s[0] == '-') {
        neg = true;
        i = 1;
    } else if (s[0] == '+') { i = 1; }
    for (; i < s.size(); ++i) x = x * 10 +
↪ (s[i] - '0');
    if (neg) x = -x;
    return is;
}
```

16.4 评测机限制与编译选项

- 测试代码长度限制，尝试触发 NO-OUTPUT、OUTPUT-LIMIT、RUN-ERROR。
- 测试 #pragma GCC optimize / #pragma GCC target 是否会被判为 CE (本机自带一半答案，详见下面「#pragma 开优化 / SIMD 实测」一节)。
- 测试 clarification：如果问不同类型的愚蠢问题，得到的回复是否不一样？
- 测试 -fsanitize=address、-fsanitize=undefined、-D_GLIBCXX_DEBUG 是否可用。

- 测试 `/usr/bin/time -v ./a.out` 是否能显示内存占用。

16.5 #pragma 开优化 / SIMD 实测

开 O3 三条路径，可叠加；**源码 pragma 优先级高于命令行**（同 TU 内覆盖 -O flag）。

```
// 命令行：本机训练默认；-march=native 评测机别带
// g++ -O3 -std=c++20 a.cpp

// optimize 放 include 之前；target 必须放
// include 之后（见下方踩坑 d）
#pragma GCC optimize("O3,unroll-loops")
#include <bits/stdc++.h>
#ifdef __x86_64__
#pragma GCC target("avx2,bmi,bmi2")
#endif
int main() { std::vector<int> v(5); v[0] = 1; }
// 自检要触发 STL 实例化
```

实测 ($N = 2^{20}$ 浮点 mul-add 200 轮，5 次中位数)：

编译方式	时间
-O2 (评测机默认)	120 ms
-O3	117 ms
-O3 -funroll-loops	113 ms
#pragma optimize("O3,unroll-loops")	114 ms

评测机默认 -O2，所以 `#pragma optimize("O3")` 的实际收益是 **O3 比 O2 快约 3%、加 unroll-loops 共约 6%**——不是“几倍加速”。其更主要价值是兜底：评测机若被设成 -O0/-O1（少见但确实有），pragma 自动把模板抬回 O3。

踩坑：(a) `target("avx2")` 在 arm64 硬 CE，模板里此类代码默认假设 x86 评测机。(b) pragma 路径下 `__OPTIMIZE__` 不定义（后端才 apply），不能拿来自检。(c) Mac 系统 g++ 是 clang shim、不认这套 pragma 也找不到 `bits/stdc++.h`，本地必须显式调 `g++-15`。(d) **#pragma GCC target 必须放在 #include <bits/stdc++.h> 之后**：放之前会让 `libstdc++` 头里 `always_inline` 模板（如 `_allocator / _Vector_impl`）带上 SIMD target，CF 用的 winlibs MinGW GCC 上 user 代码用 `vector<>` 等触发 inline 时报 `target specific option mismatch` CE（Linux GCC 不触发）。

评测机自检：把上段提交到任意评测机——CE 就是 pragma 被拒，AC 就是 pragma 全收。**自检源码必须实例化 STL**（如 `vector<int> v(5)`），空 main 不触发模板 inline、是假绿灯。

16.6 性能基准（map 跑分）

跑分覆盖 **STL 容器 + 随机访存 + 比较器内联**这套 CP 真实热点。`std::map` 的插入查找把红黑树指针追逐 + L2/L3 miss + `operator<` 内联效果都能压出来。计时用 `chrono::steady_clock`（wall-clock，纳秒分辨率，跨平台语义一致）而不是 `clock()`（Windows MSVC 上是 wall-clock、Linux glibc 上是 CPU 时间，跨评测机数字没法直接比）。标定 1s 能跑多少次：

```
#include <bits/stdc++.h>
using namespace std;

// 文件作用域：高精度 steady_clock 种子 +
// splitmix64 非线性混合（同 §10.4）
mt19937_64 rng(chrono::steady_clock::now().time)
// _since_epoch().count());
uint64_t splitmix64(uint64_t x) {
    x ^= x >> 30; x *= 0xbf58476d1ce4e5b9ULL;
    x ^= x >> 27; x *= 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

int main() {
    const int N = 1 << 20;
    vector<uint64_t> keys(N);
    for (int i = 0; i < N; i++) keys[i] =
        // splitmix64(rng());
    auto t0 = chrono::steady_clock::now();
    map<uint64_t, int> mp;
    for (int i = 0; i < N; i++) mp[keys[i]] =
        // i; // 插入
    long long s = 0;
    for (int i = 0; i < N; i++) s +=
        // mp[keys[i]]; // 查找
    assert(s != 0);
    // 防 DCE + 弱正确性
    auto dt = chrono::steady_clock::now() - t0;
    cout << chrono::duration_cast<chrono::milli>
        // seconds>(dt).count() << "ms\n";
}

// 参考 (N=2^20 一轮)：本机 440ms; CF ~2434ms
// (judge Used ~1988ms); QOJ 1024-1558ms
// (连发越跑越快，冷启动 +50%)
```

17 常用 Python 命令

17.1 启动交互解释器

启动命令因环境而异，依次试：python3 / python / py。

- **Linux 赛场机**：一般只有 python3（裸 python 可能不存在或指向 Python 2）。
- **Windows**：官方安装器装的是 py launcher，敲 py；从 Microsoft Store 装的或勾了 PATH 的则 python 直接可用。
- 退出：exit()（或 Unix Ctrl-D / Windows Ctrl-Z + Enter）。

17.2 当计算器：科学计数法速估

赛场想快速估算（阶乘 / 组合数 / log / 大整数幂会不会爆 11）时，进解释器先敲这两行：

```
>>> from math import *
>>> def p(x): print(f"{x:.2e}")
...
>>> p(factorial(20))
2.43e+18
>>> p(comb(30, 15))
1.55e+08
>>> p(2**64)
1.84e+19
>>> log2(1e9)
29.897352853986263
```

p(x) 按科学计数法两位小数打印，配合 Python 天然无界的大整数，判「爆不爆 11（上限 $\approx 9.22e18$ ）/ 复杂度量级多大」一眼出结果。

18 常用技巧（卡常，对拍，打表，调试等）

18.1 快读快写

下面这个模板来自 wida 的 GitHub 仓库并补充了一些功能，适合需要高性能读写的场景。

```
// 来自wida github仓库
→ https://github.com/hh2048/XCPC/tree/main
→ 加了一些新功能
//46 ms https://acm.hdu.edu.cn/contest/view-code?cid=1179&rid=14912 读入字符getc(a) 的AC记录
// AC https://vjudge.net/solution/63197697
→ 一道测试读写的codechef题目
namespace QuickRead {
    // 快读
    char buf[1 << 21], *p1 = buf, *p2 = buf;

    inline int getc() {
        return p1 == p2 && (p2 = (p1 = buf) +
            → fread(buf, 1, 1 << 21, stdin), p1 ==
            → p2) ? EOF : *p1++;
    }

    template<typename T>
    void Cin(T &a) {
        T ans = 0;
        bool f = 0;
        char c = getc();
        for (; c < '0' || c > '9'; c = getc()) {
            if (c == '-') f = 1;
        }
        for (; c >= '0' && c <= '9'; c = getc()) {
            ans = ans * 10 + c - '0';
        }
        a = f ? -ans : ans;
    }

    void getc(char &a) {
        char c = getc();
        while (isspace(c)) {
            c = getc();
        }
        a = c;
    }

    template<typename T, typename... Args>
    void Cin(T &a, Args &... args) {
        Cin(a, Cin(args...));
    }

    // ===== 输出 =====
    static constexpr size_t OUT_CAP = 1 << 21; //
    → 大概是2MB
    char obuf[OUT_CAP];
    size_t op = 0;

    inline void flush() {
        if (op) {
            fwrite(obuf, 1, op, stdout);
            op = 0;
        }
    }
}
```

```
inline void putc(char c) {
    if (op == OUT_CAP) flush();
    obuf[op++] = c;
}

template<typename T>
void write(T x) {
    // 注意，这里输出不带换行
    if (x < 0) putc('-'), x = -x;
    if (x > 9) write(x / 10);
    putc(x % 10 + '0');
}

struct Flusher {
    // 析构时自动刷新
    ~Flusher() { flush(); }
} flusher;
} // namespace QuickRead
using namespace QuickRead;
```

18.1.1 读入，输出科学计数法小数

正常来说是用不到这个东西的。

```
// https://www.luogu.com.cn/record/230330447
→ 科学计数法小数
template<typename T>
void Cinf(T &a) {
    // 新增：读取浮点数，支持科学计数法
    T ans = 0;
    bool f = 0; // 负号
    char c = getc();
    // 跳过非数字和小数点前的空白/符号
    while ((c < '0' || c > '9') && c != '.') {
        if (c == '-') f = 1;
        c = getc();
    }
    // 读取整数部分
    while (c >= '0' && c <= '9') {
        ans = ans * 10 + (c - '0');
        c = getc();
    }
    // 如果有小数点，读取小数部分
    if (c == '.') {
        c = getc();
        T frac = static_cast<T>(1);
        while (c >= '0' && c <= '9') {
            frac /= 10;
            ans += (c - '0') * frac;
            c = getc();
        }
    }
    a = f ? -ans : ans;
    // 处理科学计数法部分
    if (c == 'E' || c == 'e') {
        c = getc();
        bool exp_neg = false;
        if (c == '+') {
            c = getc();
        } else if (c == '-') {
            exp_neg = true;
        }
    }
}
```



```

        c = getc();
    }
    int exp = 0;
    while (c >= '0' && c <= '9') {
        exp = exp * 10 + (c - '0');
        c = getc();
    }
    T multiplier = pow(10.0, exp_neg ? -exp :
    ↪ exp);
    a *= multiplier;
}

template<typename T>
void writef(T x) {
    // 新增: 输出浮点数, 默认输出6位小数,
    ↪ 支持四舍五入
    if (x < 0) putchar('-'), x = -x;
    using ll = long long;
    int prec = 6; // 默认精度 6, 可根据需要调整
    // 添加圆整因子
    T round_add = static_cast<T>(5);
    for (int i = 0; i < prec + 1; ++i) {
        round_add /= 10;
    }
    x += round_add;
    ll int_part = static_cast<ll>(x);
    write(int_part); // 输出整数部分
    putchar('.');
    T frac_part = x - int_part;
    for (int i = 0; i < prec; ++i) {
        frac_part *= 10;
        int digit = static_cast<int>(frac_part);
        putchar(digit + '0');
        frac_part -= digit;
    }
}

```

18.2 开大栈空间

如何开大栈空间

正常应该是不需要选手去干这件事情的。

杭电 OJ 暂不支持调整栈空间大小。如果使用 G++ 提交代码,且代码中递归层数较深,可能会返回 Runtime Error (STACK_OVERFLOW)。可以在 main() 开头加入内联汇编手动切换到更大的栈空间。注意必须用 exit(0) 退出,不能 return,否则会访问已失效的旧栈帧。

x86_64 (Codeforces、洛谷、AtCoder 等绝大多数 OJ):

```

int main() {
    int size(256 << 20); // 256M
    __asm__ volatile("movq %0, %%rsp" ::
    ↪ "r"((char *)malloc(size) + size));

    // 关同步流代码
    // 主程序代码

    exit(0);
}

```

AArch64 / ARM64 (Apple Silicon 本地调试等):

```

int main() {
    int size(256 << 20); // 256M
    asm volatile("mov sp, %0" :: "r"((char
    ↪ *)malloc(size) + size));

    // 关同步流代码
    // 主程序代码

    exit(0);
}

```

18.3 C++ 中 struct 自定义比较器与 std::array 性能对比

在作为 std::set 和 std::priority_queue 的元素时,使用自定义的 struct (配合自定义比较器) 通常比使用 std::array (无论是自定义比较器还是默认比较器) 具有更好的性能。根据基准测试,在 std::set 中,struct 的操作耗时比 std::array 显著更少(快约 30% 到 50%)。在 std::priority_queue 中两者性能相近,但整体上 priority_queue 远快于 set。因此,在对常数要求较高的题目中,建议使用 struct 替代 std::array 或 std::tuple,并优先考虑使用 priority_queue。

详细的基准测试代码与输出结果记录 ([gist 链接](#))。

18.4 计时

```

#include <chrono>
#include <iostream>

int main() {
    auto start = std::chrono::high_resolution_clock::now();

    // ... 你的代码 ...

    auto end = std::chrono::high_resolution_clock::now();
    ↪ ck::now();

    // 转换为毫秒
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
    ↪ std::chrono::milliseconds>(end - start);
    std::cout << "耗时: " << duration.count()
    ↪ << " ms\n";

    // 转换为微秒
    auto us = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
    ↪ hrono::microseconds>(end - start);
    std::cout << "耗时: " << us.count() << "
    ↪ us\n";

    // 或者直接用浮点数秒
    std::chrono::duration<double> sec = end -
    ↪ start;
    std::cout << "耗时: " << sec.count() << "
    ↪ s\n";
}

```

18.5 对拍

对拍程序大致框架 (windows) (不写 .exe 也是可以的)

```
#include <bits/stdc++.h>

int main() {
    // For Windows
    // 对拍时不开文件输入输出
    // 当然, 这段程序也可以改写成批处理的形式
    while (true) {
        system("gen > test.in"); //
        ↪ 数据生成器将生成数据写入输入文件
        system("mySol.exe < test.in > a.out"); //
        ↪ 获取程序1输出
        system("std.exe < test.in > b.out"); //
        ↪ 获取程序2输出
        if (system("fc a.out b.out")) {
            // 该行语句比对输入输出
            // fc返回0时表示输出一致, 否则表示有不同处
            system("pause"); // 方便查看不同处
            return 0;
            // 该输入数据已经存放在 test.in 文件中,
            ↪ 可以直接利用进行调试
        }
    }
}
```

```
正在比较文件 a.out 和 B.OUT
***** a.out
0
***** B.OUT
4
*****
请按任意键继续. . .
```

fc 函数的输出结果如上。

Linux 可以像下面这么写。(其实就是把 fc 改成了 diff, 其他其实差不多)

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    // 对拍时不开文件输入输出
    // 当然, 这段程序也可以改写成批处理的形式
    int idx=0;
    while (true) {
        cerr<<"running on "<<++idx<<"
        ↪ test"<<"\n";
        // 下面的这些都是可执行程序
        // (相对) 路径也是以这个可执行程序位置为准
        system("./D2_Tree_Orientation_Gen >
        ↪ test.in"); //
        ↪ 数据生成器将生成数据写入输入文件
```

```
system("./D2_Tree_Orientation_Hard_Vers ]
    ↪ ion < test.in > a.out"); //
    ↪ 获取希望检验输出
system("./D1_Tree_Orientation_Easy_Vers ]
    ↪ ion < test.in > b.out"); //
    ↪ 获取暴力程序输出
if (system("diff a.out b.out")) {
    // 该行语句比对输入输出
    return 0;
    // 该输入数据已经存放在 test.in 文件中,
    ↪ 可以直接利用进行调试
}
}
```

```
zzy@zzysever:~/myCppProgram$ clang++ bruteCMP.cpp -o bruteCMP
zzy@zzysever:~/myCppProgram$ ./bruteCMP
1c1
< 16
---
> 8
sh: 1: pause: not found
zzy@zzysever:~/myCppProgram$
```

18.5.1 Special Judge (答案不唯一) 题目怎么对拍

答案不唯一时 (「输出任意一组方案」/ Yes 后跟构造 / 浮点误差), diff 逐字节比对会把「两个都对但长得不一样」误报成不一致。做法: 把 diff a.out b.out 换成自己写的 checker, 通过返回 0、发现错返回非 0——仍然沿用 system() 返回值判断, 框架其余部分完全不变:

```
system("./gen > test.in");
system("./smart < test.in > a.out");
// 有 checker 的题一般没有 brute: checker 读
↪ test.in + a.out 直接验合法性
if (system("./checker test.in a.out")) {
    system("cat test.in"); // Windows 下把 cat
    ↪ 换成 type
    cout << "\n\n";
    system("cat a.out");
    return 0; // 反例数据已留在 test.in / a.out
}
```

抓到反例时顺手把 test.in / a.out cat 到终端, 肉眼直接看现场 (Windows 下没有 cat, 换成 type, 与打表一节相同)。

checker 骨架 (argv 传文件名即可, 本地对拍不必上 OJ 官方 checker 用的 testlib.h):

```
// checker.cpp: 任一步失败 => cerr 打出原因,
↪ return 1
int main(int argc, char **argv) {
    ifstream in(argv[1]); // 输入数据
    ifstream ans(argv[2]); // smart 输出 (待验)
    ifstream ref(argv[3]); // (可选) brute 输出
    ↪ -- 仅比对最优值时才传
    // 1. 从 in 读题目数据
    // 2. 从 ans 读方案, 按题面规则逐条验证合法性
    // (格式 / 越界 / 重复 / 违反约束 -> return
    ↪ 1)
    // 3. (可选) 算出该方案的目标值, 与 ref
    ↪ 的最优值比对
```



```
return 0; // 全部通过
}
```

按答案形态选比对口径:

- **方案不唯一、目标值唯一** (「输出任意一组最优解」): brute 只输出最优值 (不必输出方案); checker 验证 smart 的方案合法, 且其目标值恰等于 brute 的最优值。
- **Yes/No + 构造**: 可行性是唯一的, 先比两边 Yes/No 是否一致; smart 答 Yes 时再验它给出的构造确实满足题面约束。
- **浮点答案**: 不逐字节比, 按 $|a - b| \leq 10^{-6} \cdot \max(1, |b|)$ 判等 (绝对或相对误差任一达标即过, 和多数 OJ 口径一致)。

两条铁律: (1) checker 只按**题面规则**验证, 禁止复用 smart 里的函数 / 思路——共用逻辑会「同错互过」, 对拍就失去意义了; (2) 「验证」往往比「求解」简单得多, 默认就按上面框架用 gen + smart + checker 三件套拍合法性、**不写 brute**; 只有「目标值唯一、需要比对最优值」的题 (口径一) 才补一份只输出最优值的 brute, 此时多跑一行 `system("./brute < test.in > b.out")` 并给 checker 多传一个 b.out。

18.6 打表

用 `system()` 调用编译好的生成器和暴力程序, 批量跑数据并输出结果, 方便观察规律。

```
struct Solve {
    // ll N;

    Solve() {
        // cin >> N;
        for (int i = 1; i <= 100; ++i) {
            system("./Good_Array_1_gen > in.txt");
            system("cat in.txt");
            system("./Good_Array_1_brute < in.txt >
                out.txt");
            cout << "\n"; // 这里需要加一个换行,
                // 才能更好地看清 out.txt
            system("cat out.txt");
            cout << "\n-----\n";
        }
    };
};
```

Windows 注意: Windows 下没有 cat 命令, 需要换成 type。

重定向 cerr 到文件: 如果程序的输出走的是 cerr (比如调试信息), 普通的 `>` 只能捕获 stdout。需要用 `2>` 或 `2>&1` 进行重定向:

```
# 只捕获 cerr (stderr) 到 out.txt
./brute < in.txt 2> out.txt

# 同时捕获 cout (stdout) 和 cerr (stderr) 到
out.txt
./brute < in.txt > out.txt 2>&1
```

不过一般我们只需要在 brute.cpp 中在 cerr 输出调试信息, 所以不需要这个。

重定向后, 被重定向的流就不会再实时显示在终端里, 而是写入文件。比如用了 `2> out.txt` 之后, cerr 会消失在终端; 用了 `> out.txt 2>&1` 之后, cout 和 cerr 都会消失在终端。

这里为什么不直接用 brute.cpp 程序的标准输出流, 而是写入文件? 因为如果直接用标准输出流, 那么 brute.cpp 的输出和这个 in.txt 的 cat 输出会混在一起, 不方便观察。

18.6.1 打表中需要输出的东西 (答案)

Binary Construction 这个是一道典型的打表题目, 只需要使用暴力输出答案, 即可一眼看出规律。

```
1
01
011
1001
10001
100001
1000001
.....
100000000000001
100000000000001
1000000000000001
1000000000000001
```

但是, 由于我们采用了放弃大脑思考, 以对拍代替大脑思考的策略, 显然, 我们不止在打表题目进行打表, 而是在很多题目上都打表, 那么如何打表才能够更好地观察规律, 找到解法, 帮助我们解决问题呢?

18.6.2 打表中需要输出的东西 (前缀答案)

Good Array 1 输出前缀答案, 可以帮助我们看出来这道题目是不是前缀 dp。

```
1
8
2 7 1 4 6 6 4 4
[dp] = [0, 0, 0, 0, 0, 1, 1, 2]

*****

2

*****

1
7
4 3 3 1 4 5 6
[dp] = [0, 0, 1, 1, 1, 1, 1]

*****

1
```

不过其实不是每道题目都是适合这个打表的, 对于一些题目, 头脑风暴或者其他方法可能更加适合。

18.7 凸包数据生成

对拍几何题（凸包、旋转卡壳、闵可夫斯基和等）需要快速生成 CCW 序凸多边形。每次随机选椭圆长短轴比 $\in [1, 3]$ + 随机旋转角，覆盖近圆 / 椭圆 / 倾斜椭圆等曲率非均匀形状（破除「近圆形 monotonicity 盲区」），然后随机 N 个角度排序映射上去。T 模板参数：T = ll 走 llround 钉整数（题面整数坐标）、T = DB 保留 cos/sin 原浮点（精度/边界更细）；坐标范围 $x, y \in [-R, R]$ （椭圆 $R \cos \theta, \frac{R}{asp} \sin \theta$ 内嵌半径 R 圆盘，旋转 φ 不变半径，llround 边界 ± 1 不溢出）。allow_collinear 切换严格凸（cross > 0）和允许三点共线（cross ≥ 0 ，但仍要求至少一处真左转保证多边形面积非零）。可行域实测： $R = 10^9$ 时 $N \leq 2000$ 安全（ $N = 5000$ 严格模式 75% 失败）； $N/R \gtrsim 0.3$ 严格模式 retry 显著上升， $\gtrsim 0.5$ 不可行（如 $R = 100$ ， $N = 50$ 100% 卡死）。要求 $N \geq 3$ ，超出可行域时自旋等死（外面套 retry 上限自己加）。

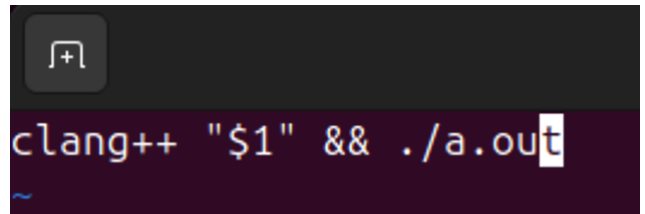
```
mt19937_64 rng(chrono::steady_clock::now().time
    ↪ _since_epoch().count());

// T = ll : 整数坐标 (llround)    /    T = DB :
    ↪ 浮点坐标 (保留原值)
// 坐标范围 |x|, |y| <= R
template<class T>
vector<Point<T>> gen_convex_polygon(int N, bool
    ↪ allow_collinear = false,
                                ll R =
                                ↪ 1'000'0
                                ↪ 00'000)
    ↪ {
    // 退化 case (N <= 2): 主循环对 N <= 2
    ↪ 必死循环
    // (N=1: j=(0+1)%1=0=i, A[i]==A[j] 永真;
    ↪ N=2: k=(0+2)%2=0=i, cross 三点退化恒 0).
    // 直接撒 N 个不重点返回，跳过 CCW 校验.
    if (N <= 2) {
        set<pair<T, T>> S;
        while ((int) S.size() < N) {
            if constexpr
                ↪ (is_floating_point_v<T>) {
                S.insert({uniform_real_distribut
                    ↪ tion<T>(-(T) R, (T) R)(rng),
                    uniform_real_distribut
                    ↪ tion<T>(-(T) R,
                    ↪ (T) R)(rng)});
            } else {
                S.insert({uniform_int_distribut
                    ↪ ion<T>(-(T) R, (T) R)(rng),
                    uniform_int_distribut
                    ↪ ion<T>(-(T) R,
                    ↪ (T) R)(rng)});
            }
        }
        vector<Point<T>> A;
        for (auto [x, y] : S)
            ↪ A.push_back(Point<T>(x, y));
        return A;
    }
    DB phi = uniform_real_distribution<DB>(0, 2
    ↪ * M_PI)(rng);
```

```
DB asp = uniform_real_distribution<DB>(1.0,
    ↪ 3.0)(rng); // 1=圆, >1=椭圆
DB cp = cos(phi), sp = sin(phi);
while (true) {
    vector<DB> th(N);
    for (auto& t : th) t =
        ↪ uniform_real_distribution<DB>(0, 2
        ↪ * M_PI)(rng);
    sort(th.begin(), th.end());
    vector<Point<T>> A(N);
    for (int i = 0; i < N; ++i) {
        DB x = R * cos(th[i]), y = R / asp
            ↪ * sin(th[i]); // 椭圆
        DB rx = cp * x - sp * y, ry = sp *
            ↪ x + cp * y; // 旋转 phi
        if constexpr
            ↪ (is_floating_point_v<T>) {
            A[i] = Point<T>(rx, ry);
        } else {
            A[i] = Point<T>((T)
                ↪ llround(rx), (T)
                ↪ llround(ry));
        }
    }
    bool ok = true, has_turn = false; //
    ↪ has_turn: 至少一处真左转
    for (int i = 0; i < N && ok; ++i) {
        int j = (i + 1) % N, k = (i + 2) %
            ↪ N;
        if (A[i] == A[j]) { ok = false;
            ↪ break; }
        auto c = cross(A[i], A[j], A[k]);
        if (allow_collinear ? c < 0 : c <=
            ↪ 0) { ok = false; break; }
        if (c > 0) has_turn = true;
    }
    // allow_collinear 下若全程 cross == 0
    ↪ 则点全共线、面积 = 0, 得重抽
    if (ok && !has_turn) ok = false;
    if (ok) return A;
}
}
```

18.8 编译 shell

```
clang++ "$1" && ./a.out
```



```
zzy@zzysever:~/myCppProgram$ ./cr gen.cpp
1
5 4
2 1
3 1
4 1
5 1
```